Оглавление

Введение 2
Основы языка программирования – Kotlin 4
Инструменты разработки 7
Система контроля версий 29
Облачный бэкенд и хранение данных – Supabase 41
Базовые компоненты и UI Android 44
Архитектура и паттерны презентационного слоя 49
Требования к реализации лабораторных работ 50
Лабораторная работа №1 51
Теоретическая часть 51
Техническое задание 53
Контрольные вопросы 58
Лабораторная работа №2 60
Теоретическая часть 60
Техническое задание 64
Контрольные вопросы 74
Лабораторная работа №3 76
Теоретическая часть 76
Техническое задание 80
Контрольные вопросы 86
Лабораторная работа №4 87
Теоретическая часть 87
Техническое задание 89
Контрольные вопросы 98
Лабораторная работа №5 99
Теоретическая часть 99
Техническое задание 100
Контрольные вопросы 108

Введение

Android - мобильная платформа с открытым исходным кодом, это означает, что значительная часть системы и инструментов доступна разработчикам для анализа, модификации и создания собственных решений. Платформа развивается уже много лет, и за это время вышло множество версий.

Android-разработка опирается на одну из самых массовых мобильных платформ, поэтому количество Android-устройств напрямую формирует устойчивый спрос на разработчиков. Экосистема постоянно расширяется, появляются новые модели смартфонов, планшетов и других устройств, которым нужны современные приложения. Вход в эту область сравнительно прост: инструменты разработки распространяются бесплатно, используются открытые технологии и библиотеки с открытым исходным кодом. Это позволяет быстро перейти от учебных примеров к реальным проектам и понимать, как всё устроено изнутри.

Сегодня Android установлен на миллиардах активных устройств по всему миру, поэтому написанное нами приложение потенциально выходит на глобальную аудиторию. Экосистема Google Play обеспечивает централизованную витрину для распространения приложений, где ежемесячно активны огромные объёмы пользователей, постоянно генерирующие спрос на новые сервисы.

Android не ограничивается только смартфонами, это единая платформа для самых разных устройств. Те же технологии используются в телевизорах, планшетах, умных часах, автомобильных системах и других категориях электроники. Для разработчика это означает возможность переносить знания и архитектурные решения между разными форм-факторами, адаптируя интерфейс и логику под конкретный экран и сценарий использования. Одна кодовая база может работать сразу на нескольких типах устройств.

В современной Android-разработке основной упор сделан на язык Kotlin, он официально поддерживается Google и считается предпочтительным для создания нативных приложений. Android Studio, официальные библиотеки и большинство новых API проектируются с учётом особенностей Kotlin, поэтому инструменты разработки, документация и примеры кода изначально ориентированы именно на него. Язык полностью совместим с существующим Java-кодом, что позволяет использовать проверенные временем библиотеки и постепенно обновлять архитектуру проектов без жёстких миграций.

Платформа Android даёт разработчику возможность довольно быстро переходить от идеи к работающему продукту. Даже с базовыми знаниями можно собрать приложение, запустить его на устройстве и подключить к реальному серверу. Набор стандартных компонентов и шаблонов проектов в Android Studio сокращает объём рутины, поэтому основной фокус можно держать на логике и пользовательских сценариях. При этом платформа не ограничивает готовыми решениями: систему интерфейсов можно глубоко настраивать, создавать собственные элементы, анимации и темы оформления под требования конкретного продукта. Помимо смартфонов, Android широко используется на специализированных устройствах, таких как терминалы оплаты, сканеры штрихкодов и мобильные терминалы сборщиков заказов в магазинах и на складах. В таких случаях приложения должны надёжно интегрироваться с кассовыми системами, периферийным оборудованием вроде принтеров чеков и внутренними сервисами компании. Важное развитие получает направление кроссплатформенных приложений на Kotlin Multiplatform, где общая бизнес-логика реализуется единым кодом и затем подключается к проектам под разные операционные системы.

Теоретические разделы учебно-методического материала по дисциплине «Разработка мобильных приложений» охватывают основы языка Kotlin, инструменты разработки и контроль версий (Android Studio и Git), основы облачного бэкенда и хранения

данных (на примере Supabase), а также базовые компоненты Android и подходы к UI (Activity, XML-верстка Views и обзор Jetpack Compose). Практическая часть включает 5 лабораторных работ, посмотрим какие темы в них будут:

1 - SplashScreen, проверка наличия интернета в приложении, ViewPager2.

2 - Выбор даты/времени, взаимодействие с хранилищем устройства.

3 - SearchView, RecyclerView, нижнее навигационное меню.

4 - Room и SharedPreferences.

5 - Spinner и множественный выбор фотографий из галереи пользователя.

Каждая лабораторная работа сопровождается теоретическим разделом, направленным на закрепление ключевых понятий, и практическим заданием, ориентированным на воспроизведение типовых сценариев разработки Android-приложений, а также включает контрольные вопросы, предназначенные для самопроверки усвоения материала и целенаправленной подготовки к защите выполненного задания. Освоение материала позволит студентам не только закрепить базовые приемы разработки на Kotlin, но и сформировать целостное представление о взаимосвязи основных компонентов Android-экосистемы. Итогом выполнения лабораторных работ должно стать умение проектировать структуру приложения, реализовывать пользовательские сценарии и подготавливать результат к демонстрации и защите по заданным требованиям.

Kotlin - основной язык разработки современных Android-приложений, созданный компанией JetBrains. Он проектировался как практичный язык для реальных промышленных задач, с упором на простоту чтения и понятный синтаксис. Конструкции языка позволяют писать меньше служебного кода, чем обычно требуется в Java, при этом сохраняется привычная структурная модель программы. Kotlin компилируется в тот же байт-код виртуальной машины Java, поэтому использует ту же платформу исполнения и экосистему библиотек. Это означает, что Kotlin и Java могут свободно работать в одном проекте, а существующий Java-код может постепенно дополняться и расширяться кодом на Kotlin. Богатая стандартная библиотека уже включает средства для работы со строками, коллекциями и другими типичными задачами, поэтому многие операции выполняются одной-двумя строками.

Kotlin относится к строго типизированным языкам программирования: тип каждой переменной и выражения определяется явно или выводится компилятором на этапе компиляции и не может быть произвольно изменён во время выполнения программы. Это повышает надёжность кода и позволяет обнаруживать значительное число ошибок ещё до запуска приложения. При этом Kotlin поддерживает механизм вывода типов, благодаря которому разработчику не требуется каждый раз явно указывать тип переменной, если он однозначно определяется из контекста. Язык также поощряет безопасную работу с данными за счёт встроенной поддержки null-безопасности, что снижает вероятность возникновения ошибок типа NullPointerException и способствует написанию более устойчивых и предсказуемых программ.

Пример объявления переменных в Kotlin:

```
var userAge: Int = 20 // Переменная, значение которой можно изменять
userAge = 21 // Значение переменной можно переопределить
// Переменная только для чтения, значение задаётся один раз
val userName: String = "Иван"
// Тип можно не указывать явно, он будет выведен компилятором
val isAuthorized = true
```

В данном примере var используется для изменяемых переменных, тогда как val предназначен для значений, которые не должны изменяться после инициализации. Такой подход делает код более предсказуемым и облегчает его сопровождение.

Kotlin поддерживает четыре целочисленных типа с фиксированной разрядностью: Byte, Short, Int и Long, которые различаются объёмом занимаемой памяти и, как следствие, диапазоном представимых значений. Тип Byte использует 8 бит и применяется для хранения малых числовых величин, тогда как Long имеет разрядность 64 бита и предназначен для работы с существенно большими числами. Наличие строгой числовой типизации способствует более рациональному использованию памяти и обеспечивает корректность арифметических вычислений.

Помимо целочисленных типов, в языке Kotlin предусмотрены типы Double и Float, предназначенные для представления вещественных чисел. Тип Double используется по умолчанию при работе с десятичными значениями и обладает 64-битной разрядностью, что обеспечивает точность порядка 16–17 значащих цифр. Тип Float имеет разрядность 32 бита и позволяет хранить около 6–7 значащих цифр, обеспечивая экономию памяти за счёт снижения точности вычислений. Символьный тип Char (16 бит) предназначен для хранения одного символа в кодировке Unicode. Для представления логических значений в Kotlin используется тип Boolean, который может принимать одно из двух состояний: true или false. Логические операции (||, &&, !) реализованы с использованием механизма ленивых вычислений, при котором вычисление выражения прекращается, если его итоговое значение может быть определено без оценки всех операндов.

В языке Kotlin строка определяется как упорядоченная последовательность символов, заключённая в двойные кавычки. Для управления форматированием строк могут применяться специальные управляющие последовательности, например \n, используемая для обозначения переноса строки. Кроме того, Kotlin поддерживает так называемые raw-строки (многострочные строки), которые записываются с использованием тройных кавычек (""""). Такой способ задания строк позволяет включать произвольный текст без экранирования символов, в том числе сохранять исходные переносы строк и форматирование.

Одной из ключевых особенностей языка Kotlin является встроенный механизм null-safety, направленный на предотвращение ошибок, связанных с обращением к отсутствующему значению (null). В Kotlin типовая система на уровне компиляции различает значения, которые могут принимать null, и значения, для которых null запрещён. По умолчанию любая переменная не может содержать null, если это явно не указано в её типе. Для объявления nullable-типа используется оператор ?, добавляемый к имени типа. Такой подход позволяет обнаруживать потенциальные ошибки ещё на этапе компиляции и существенно снижает вероятность возникновения NullPointerException во время выполнения программы.

Для безопасной работы с nullable-значениями Kotlin предоставляет набор языковых конструкций. Оператор безопасного вызова ?. позволяет обращаться к свойствам и методам объекта только в том случае, если он не равен null. Оператор Элвиса ?: используется для задания значения по умолчанию, если выражение слева равно null. В случаях, когда разработчик логически уверен, что значение не может быть null, может применяться оператор принудительного утверждения !!, однако его использование считается небезопасным и допускается только в исключительных ситуациях, поскольку при ошибке он приводит к выбросу NullPointerException.

```
// Переменная non-nullable: значение null недопустимо
val userName: String = "MTUCI"
// Переменная nullable: допускает как строку, так и null
val middleName: String? = null
// Безопасный вызов: метод length будет вызван,
// только если middleName не равен null
val length: Int? = middleName?.length
// Оператор Элвиса: если middleName равно null,
// будет использовано значение по умолчанию
val safeLength: Int = middleName?.length ?: 0
// Принудительное утверждение non-null.
// Если значение окажется null, возникнет исключение
val forcedLength: Int = middleName!!.length
```

В языке Kotlin функции и классы являются базовыми конструкциями, используемыми для структурирования программы и описания её логики. Функции предназначены для инкапсуляции законченных фрагментов поведения и позволяют многократно использовать один и тот же алгоритм в различных частях приложения. Объявление функции в Kotlin начинается с ключевого слова fun, за которым указывается имя функции, список параметров с их типами и, при необходимости, тип возвращаемого значения. Явное указание возвращаемого типа повышает читаемость кода и упрощает его анализ, хотя в ряде случаев компилятор способен вывести тип автоматически. Функции могут не возвращать значение - в этом случае используется тип Unit, который может быть опущен в объявлении.

Классы в Kotlin применяются для описания объектов предметной области и объединяют данные и поведение в единую логическую сущность. Объявление класса осуществляется с помощью ключевого слова class. Одной из особенностей Kotlin является наличие первичного конструктора, который позволяет сразу определить свойства класса в

заголовке объявления. Это существенно сокращает объём шаблонного кода по сравнению с традиционными объектно-ориентированными языками. Классы могут содержать свойства, методы, а также дополнительные конструкторы и блоки инициализации, что обеспечивает гибкость при создании и настройке объектов.

```
// Простая функция без возвращаемого значения
fun printGreeting(name: String) {
    println("Hello, $name")
}
// Функция с возвращаемым значением
fun sum(a: Int, b: Int): Int {
    return a + b
}
// Класс с первичным конструктором и свойствами
class User(
    val id: Int,
    var name: String
) {
    // Метод класса
    fun getUserInfo(): String {
        return "User(id=$id, name=$name)"
    }
}
// Создание объекта класса и вызов его методов
val user = User(id = 1, name = "MTUCI")
printGreeting(user.name)
val total = sum(3, 5)
val info = user.getUserInfo()
```

Инструменты разработки

Android Studio - официальная интегрированная среда разработки (IDE) для создания приложений под платформу Android, разработанная компанией Google совместно с JetBrains на основе IntelliJ IDEA. Первая стабильная версия Android Studio (1.0) была выпущена в декабре 2014 года, а с 2015 года она используется как основная и рекомендуемая среда разработки для Android-проектов. Android Studio объединяет в одном инструменте редактор исходного кода, средства навигации и рефакторинга, систему подсказок, инструменты статического анализа и отладки. Для запуска и проверки приложений используется встроенный эмулятор Android, позволяющий создавать и использовать виртуальные устройства с различными версиями операционной системы и конфигурациями оборудования. Сборка проекта и управление зависимостями выполняются с помощью системы Gradle, которая автоматически обрабатывает конфигурации сборки, варианты (build variants) и зависимости библиотек. Среда обеспечивает полную нативную поддержку языка Kotlin, включая автодополнение, проверку типов, рефакторинг и отладку.

Android Studio покрывает полный цикл разработки мобильного приложения - от написания и тестирования кода до подготовки релизной версии. В рамках одной среды выполняется модульное тестирование, запуск автоматизированных и ручных тестов на эмуляторах и физических устройствах, а также анализ поведения приложения. Встроенные профилировщики позволяют отслеживать использование памяти, процессора и сети, выявлять задержки и узкие места производительности. На этапе сборки Android Studio формирует установочные артефакты APK и Android App Bundle (AAB) на основе настроек Gradle, после чего из интерфейса среды можно подписать сборку, подготовить релиз и загрузить его в Google Play Console. Таким образом, Android Studio выступает не только как редактор кода, но и как комплексный инструмент, объединяющий разработку, тестирование, профилирование, сборку и публикацию Android-приложений в рамках единого рабочего процесса.

Для начала работы с Android-проектом требуется установленная среда разработки Android Studio. Официальные дистрибутивы доступны на сайте developer.android.com и устанавливаются с помощью стандартного мастера установки. После первого запуска IDE предлагает выполнить начальную настройку, в ходе которой загружаются Android SDK и дополнительные инструменты: платформенные компоненты, средства сборки и образы систем для эмуляторов. Этот этап является обязательным, так как без установленного SDK среда не сможет собирать и запускать приложения. После завершения загрузки и настройки пользователь попадает на стартовый экран Android Studio, где может создать первый проект, выбрав подходящий шаблон и базовые параметры приложения.

При создании нового проекта одним из ключевых параметров является Package Name - уникальный идентификатор Android-приложения в экосистеме платформы. Package Name задаётся в виде строки, напоминающей доменное имя в обратном порядке (например, com.example.myapp), и используется системой для однозначной идентификации приложения. Этот идентификатор применяется при установке, обновлении и удалении приложения, а также отображается в Google Play и используется во внутренней структуре проекта, включая манифест и пространство имён исходного кода. Уникальность Package Name гарантирует, что два разных приложения не могут конфликтовать между собой, поэтому его выбор на этапе создания проекта является важным архитектурным решением и, как правило, сохраняется на протяжении всего жизненного цикла приложения.

Android SDK (Software Development Kit) представляет собой официальный набор библиотек, инструментов и вспомогательных компонентов, необходимых для разработки приложений под платформу Android. В состав SDK входят API операционной системы, средства сборки и подписи приложений, инструменты командной строки, а также образы системы для эмуляторов и отладочные драйверы. Android Studio использует SDK для компиляции проекта, установки приложения на устройство или эмулятор и выполнения отладки. Фактически при создании проекта разработчик выбирает, с какими версиями

Android SDK будет совместимо приложение, что напрямую определяет доступные возможности платформы и набор API, которые можно использовать в коде.

Для каждой версии Android используются несколько связанных обозначений, отражающих разные аспекты релиза платформы. Codename - это внутреннее кодовое имя версии, которое традиционно применялось в документации и обсуждениях. Version - числовой номер релиза Android (например, Android 12 или Android 14), отображаемый в настройках устройства и в магазинах приложений. API Level - целочисленный идентификатор, который описывает конкретный набор возможностей Android SDK для данной версии системы. Именно по номеру API Level инструменты сборки определяют, какие классы, методы и функции доступны в проекте. При настройке приложения выбор минимального и целевого API Level задаёт диапазон версий Android, на которых приложение может быть установлено и корректно работать, и является важным параметром совместимости и архитектуры проекта.

Таблица 1 – Версии Android

Codename	Version	Api level/NDK release	Year
Android 16	16	API level 36	2025
Android 15	15	API level 35	2024
Android 14	14	API level 34	2023
Android 13	13	API level 33	2022
Android 12L	12	API level 32	2022
Android 12	12	API level 31	2021
Android 11	11	API level 30	2020

Для проверки и отладки Android-приложений в процессе разработки используются программный эмулятор или физическое устройство. Эмулятор представляет собой виртуальное Android-устройство, запускаемое непосредственно на компьютере и имитирующее экран, аппаратные характеристики и основные функции смартфона или планшета. В Android Studio можно создавать несколько конфигураций эмуляторов с разными версиями Android, размерами экранов и параметрами устройств, что удобно для тестирования интерфейсов и поведения приложения в различных условиях. Альтернативным вариантом является запуск приложения на реальном Android-устройстве, подключённом по USB или по сети. В этом случае приложение работает в условиях, максимально приближённых к реальному использованию, а все логи и отладочная информация доступны через инструменты Android Studio вне зависимости от выбранного способа запуска.

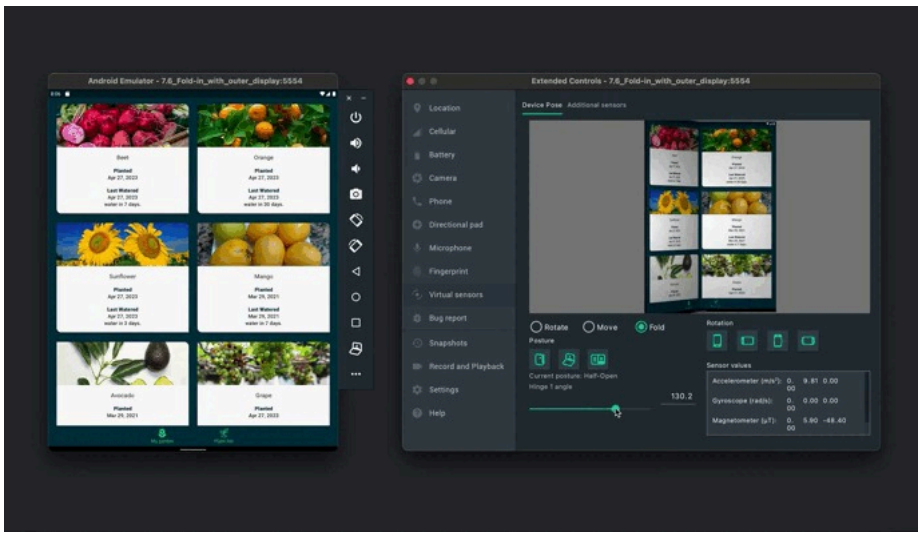


Рисунок 1 - Эмулятор

Для запуска и отладки приложения на физическом устройстве необходимо включить режим разработчика и активировать отладку по USB. Режим разработчика включается в настройках устройства путём многократного (обычно семикратного) нажатия на пункт с номером сборки системы, после чего в настройках появляется соответствующий раздел. В нём активируется параметр USB debugging, позволяющий Android Studio устанавливать и запускать приложения на устройстве, а также получать отладочную информацию. После подключения устройства система запрашивает подтверждение доверия компьютеру, и при успешном соединении устройство становится доступным для выбора в списке целевых устройств Android Studio. Это позволяет запускать приложение напрямую из среды разработки и пошагово отлаживать его поведение на реальном оборудовании.

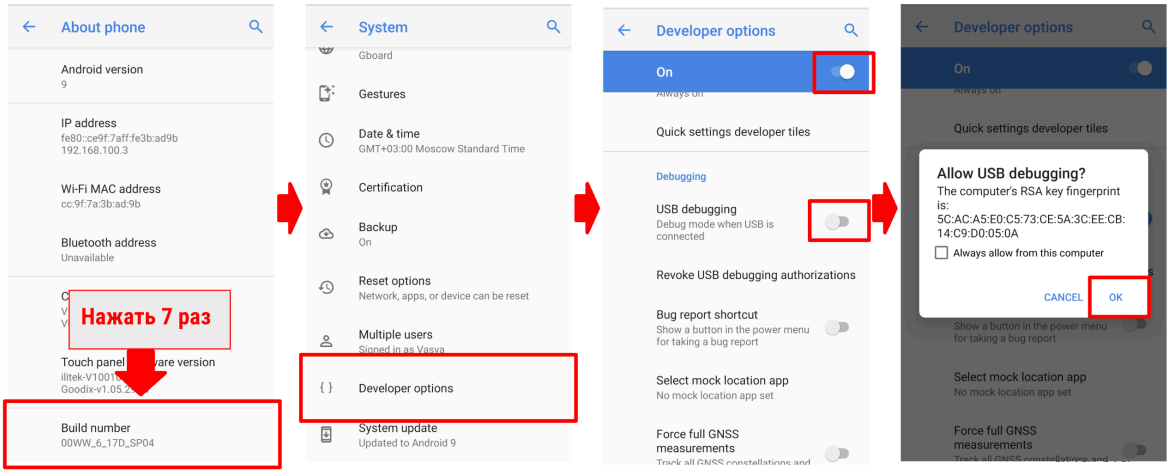


Рисунок 2 - Включение режима разработчика

В процессе сборки и распространения Android-приложений используются два основных формата установочных артефактов: APK и Android App Bundle (AAB). APK (Android Package) - это готовый скомпилированный пакет приложения, содержащий код, ресурсы и манифест, который может быть установлен непосредственно на устройство. Такой формат применяется для локального тестирования, установки приложения вручную и запуска на эмуляторах или физических устройствах без участия Google Play.

Android App Bundle (AAB) представляет собой формат модульной сборки, предназначенный для публикации приложений в Google Play. В отличие от APK, AAB не устанавливается напрямую на устройство: он загружается в Google Play, где система автоматически формирует оптимизированные APK под конкретную конфигурацию

устройства пользователя. Это позволяет уменьшить размер загружаемого приложения и исключить ненужные ресурсы, повышая эффективность распространения и установки. В современной Android-разработке формат AAB является рекомендуемым и обязательным для публикации новых приложений в Google Play.

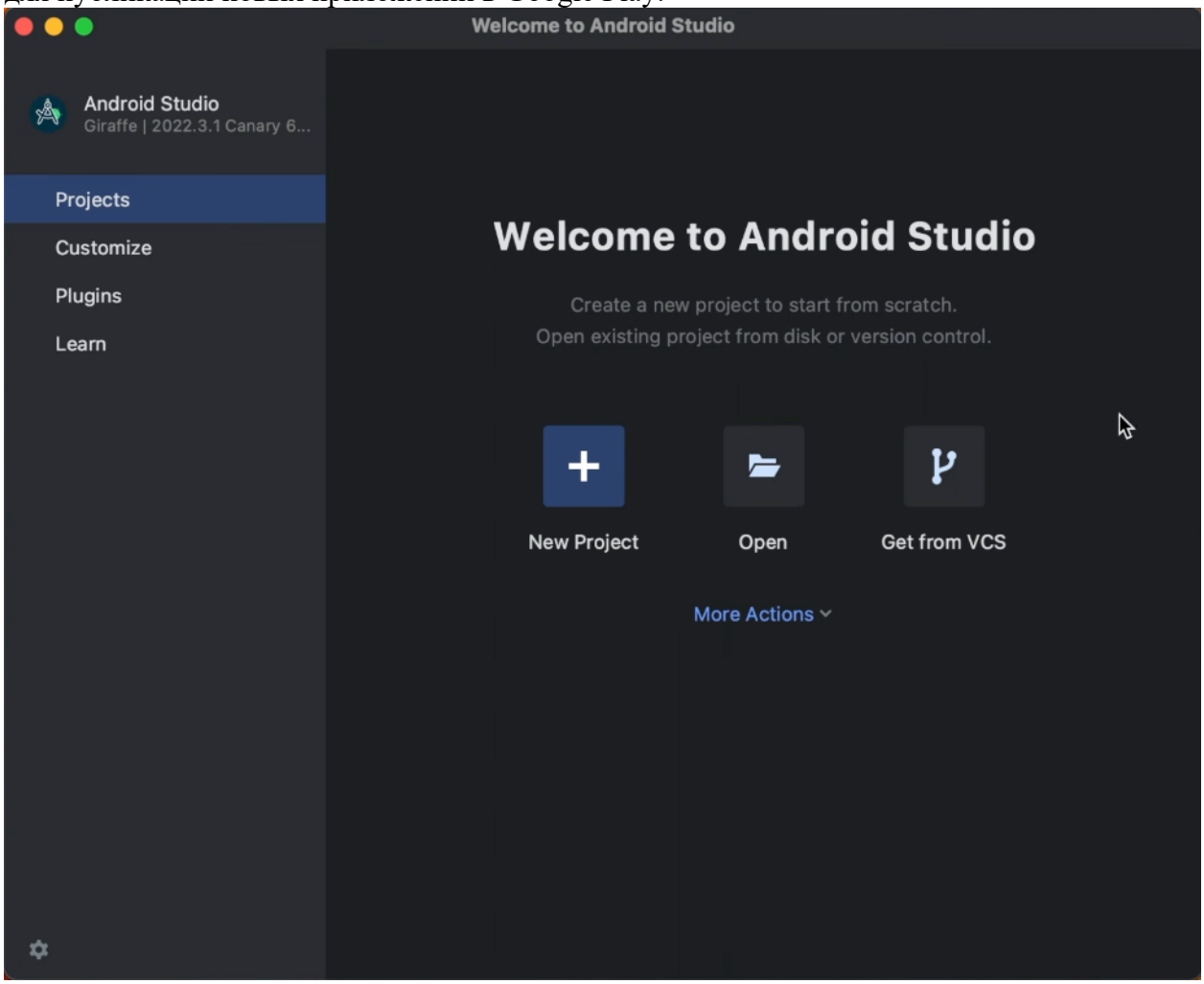


Рисунок 3 - Стартовое окно

При запуске Android Studio открывается стартовое окно (Welcome screen), предназначенное для быстрого начала работы с проектами: из него можно создать новый проект, открыть уже существующий проект с диска, либо получить проект из системы контроля версий (Get from VCS) с последующим импортом и настройкой. В этом же окне доступны разделы для первоначальной настройки среды (например, параметры интерфейса), управления расширениями (Plugins) и перехода к справочным материалам (Learn), а также дополнительные действия через меню More Actions, где обычно сосредоточены основные служебные инструменты и настройки среды разработки.

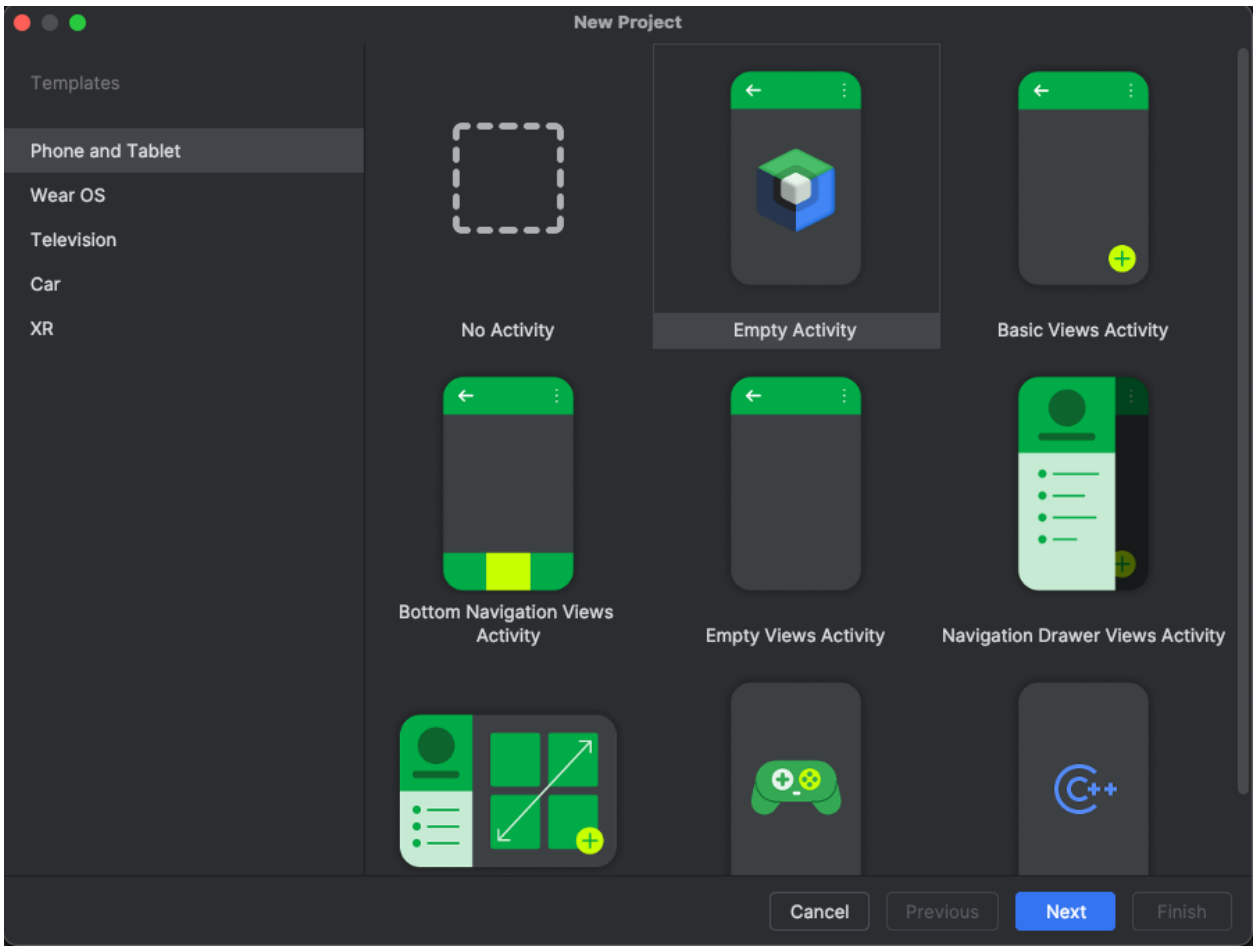


Рисунок 4 – Выбор шаблона проекта

На этом изображении показан первый шаг мастера создания проекта в Android Studio - выбор категории устройства и стартового шаблона приложения. Слева расположен список целевых платформ (например, Phone and Tablet, Wear OS, Television, Car, XR), где выбирается тип устройства, под который будет сформирована базовая конфигурация проекта. В центральной части отображается набор доступных шаблонов: каждый вариант представлен карточкой с названием и предварительным просмотром стартового экрана (например, No Activity, Empty Activity, Basic Views Activity, Bottom Navigation Views Activity, Empty Views Activity, Navigation Drawer Views Activity, а также шаблон с поддержкой C++). После выбора подходящего шаблона переход к следующему шагу выполняется кнопкой Next внизу окна.

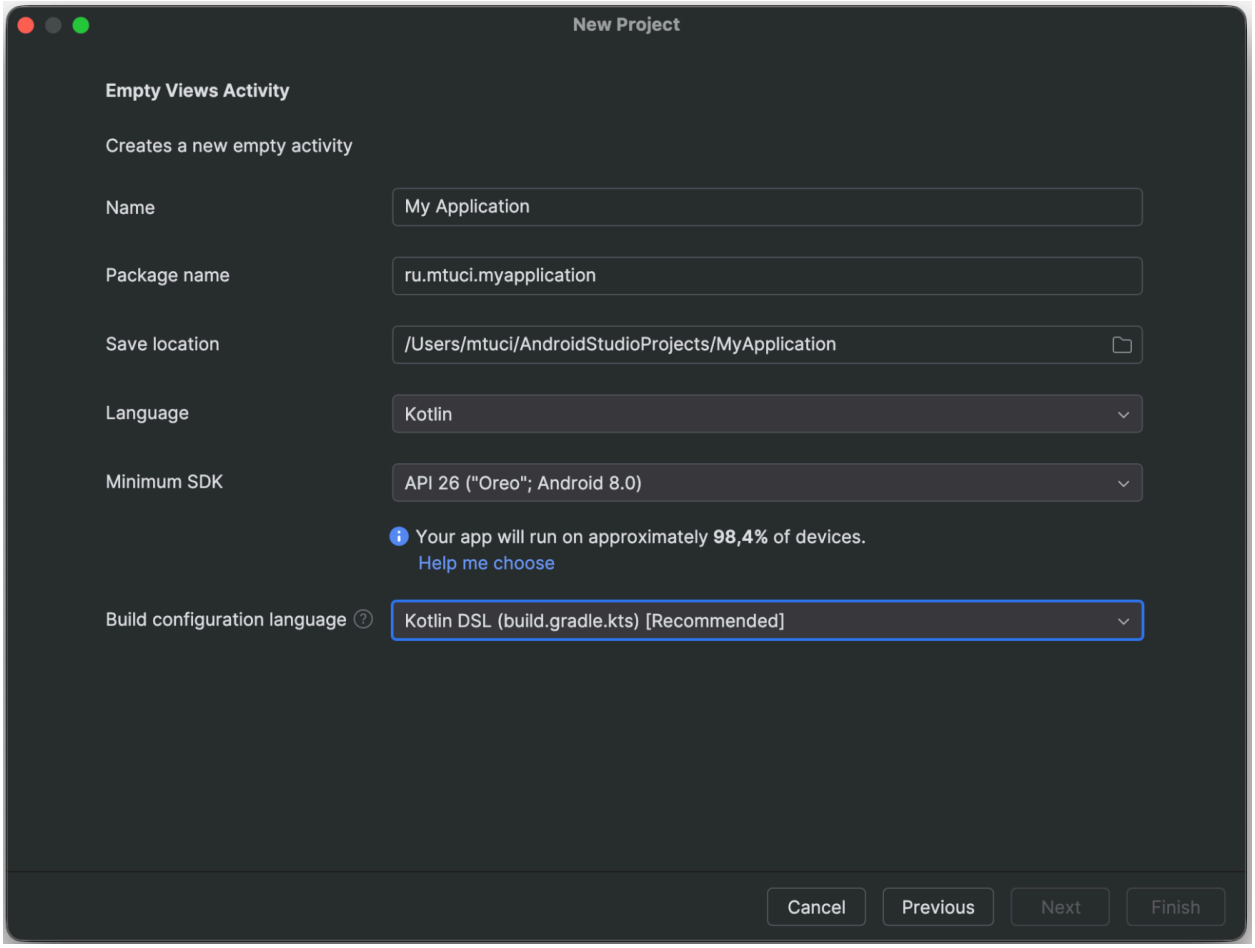


Рисунок 5 - Создание проекта

Здесь указывается имя проекта (Name) и уникальный идентификатор приложения (Package name), выбирается путь сохранения (Save location), язык разработки (Language) и минимальная поддерживаемая версия платформы (Minimum SDK). Параметр Minimum SDK определяет, на каких версиях Android приложение сможет запускаться, и влияет на доступность отдельных API, также отображается ориентировочная доля устройств, которые соответствуют выбранному минимальному уровню. Дополнительно выбирается язык конфигурации сборки Gradle (Build configuration language), где обычно рекомендуется Kotlin DSL.

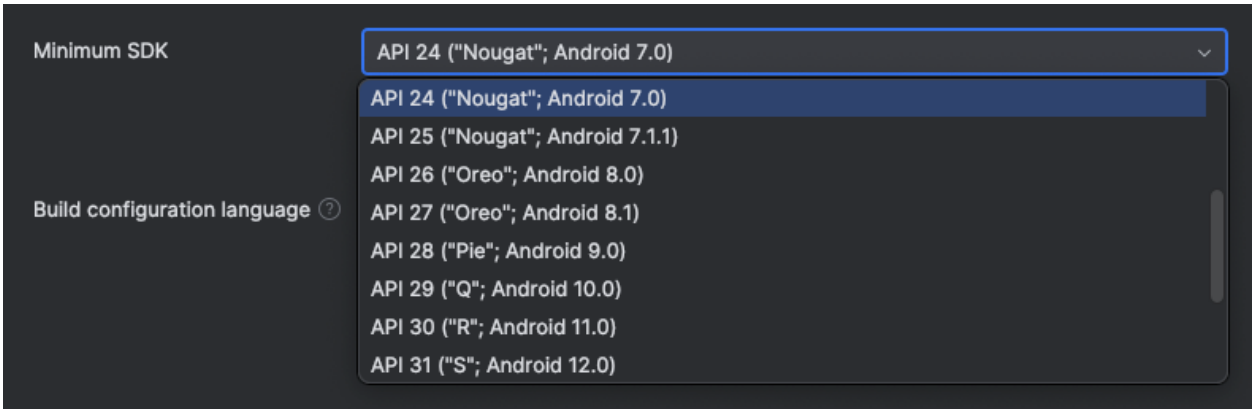


Рисунок 6 - Выбор версии SDK

В поле Minimum SDK доступен выпадающий список: при нажатии раскрывается перечень вариантов, из которых выбирается минимальная поддерживаемая версия Android. Элементы списка отображаются в нескольких представлениях одновременно: уровень API (API Level), кодовое имя релиза (Codename) и номер версии Android (Version), что упрощает сопоставление выбранного значения с версией платформы и возможностями SDK.

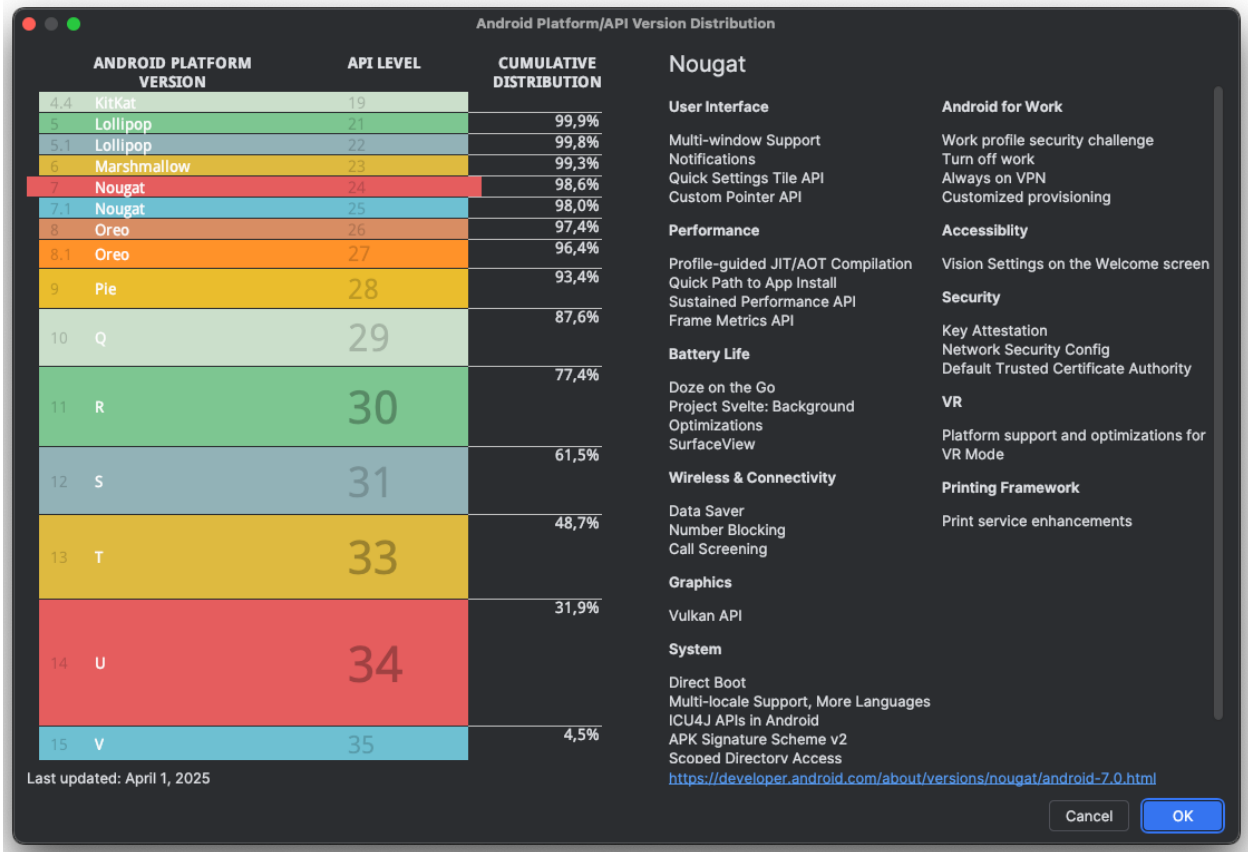


Рисунок 7 - Статистика версий

Окно Android Platform/API Version Distribution открывается по нажатию на строку «Help me choose». В нём отображается сводная статистика распространённости версий Android: слева приведён перечень релизов с долей устройств и накопленной (cumulative) долей, что помогает оценить, какую аудиторию покрывает выбранная конфигурация проекта. При выборе конкретной версии (на рисунке выбран Nougat) справа показывается краткая выжимка по возможностям и изменениям выбранного релиза, чтобы сопоставить целевую версию с доступными платформенными функциями. Внизу окна указана дата последнего обновления данных (Last updated) и приведены ссылки на источники статистики для проверки актуальности и деталей распределения.

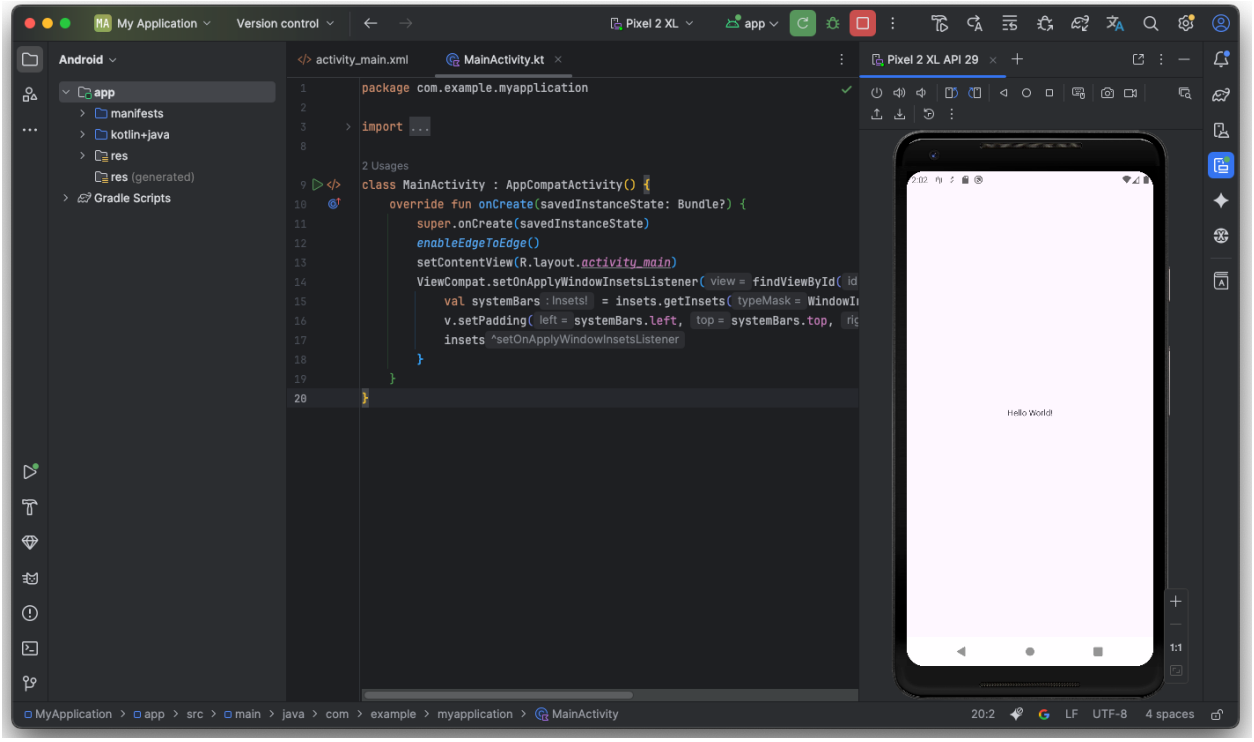


Рисунок 8 - Рабочее окно

Рабочее окно Android Studio организовано как единое пространство для разработки и проверки приложения: в центральной области расположен редактор, где открываются файлы проекта и выполняется основная работа с кодом (в примере - файл Activity на Kotlin). Слева отображается дерево проекта, через которое выполняется навигация по модулям, исходникам и ресурсам, а также поиск нужных файлов. Справа показано целевое устройство для запуска - встроенный эмулятор с установленным приложением, позволяющий сразу увидеть результат изменений без переключения на отдельные окна. Верхняя панель объединяет действия, связанные с конфигурацией запуска и сборкой проекта (выбор устройства, запуск, отладка), а нижняя часть среды обычно используется для служебного вывода (сборка, логи, сообщения инструментов), что поддерживает типовой цикл работы «изменение кода → сборка/запуск → проверка результата».

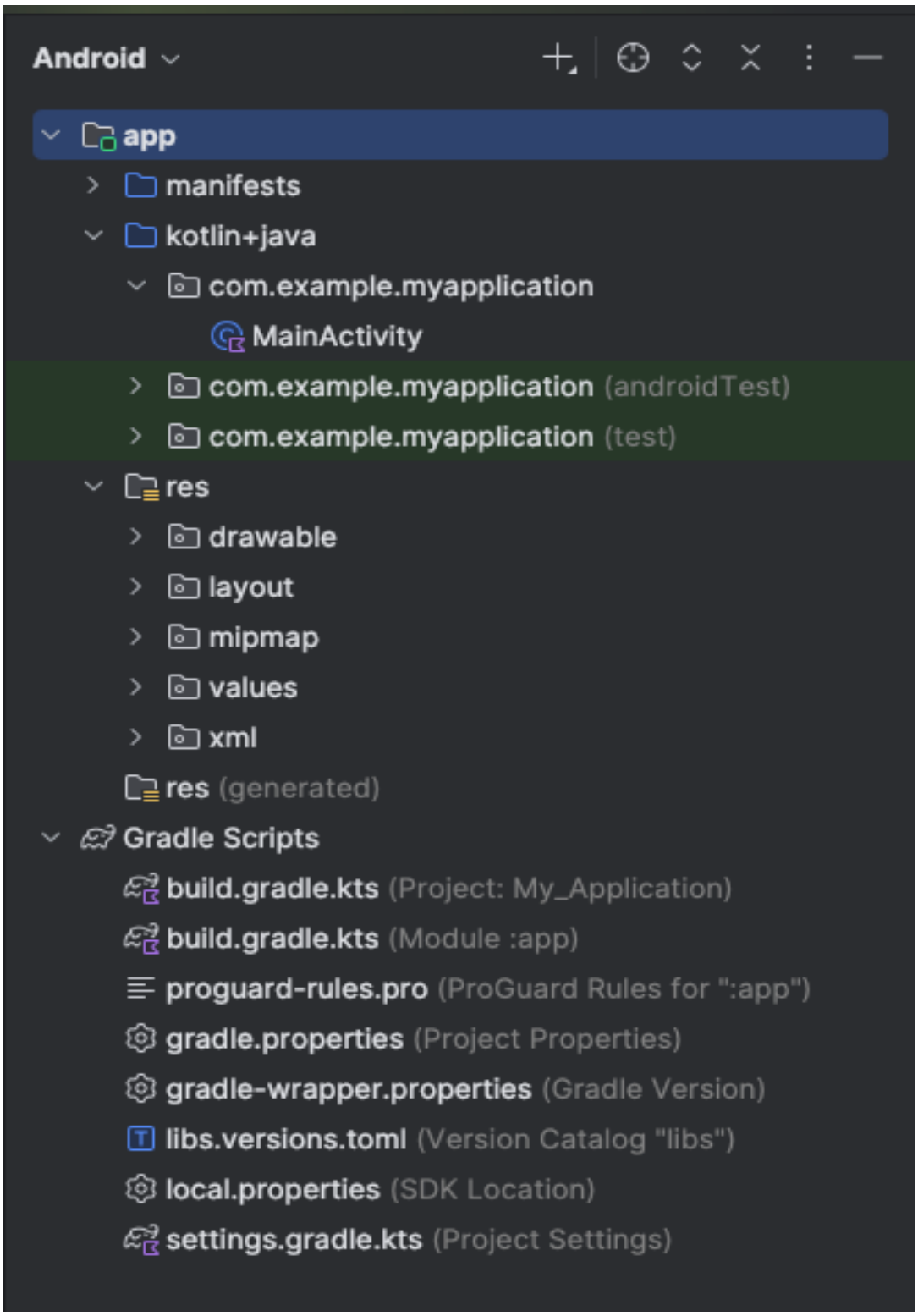


Рисунок 9 - Файловый вид Android

Дерево проекта в Android Studio - это панель навигации по файлам и структуре проекта. На рисунке выбран режим представления Android, в котором содержимое проекта сгруппировано «по смыслу» и показано в наиболее удобном для повседневной разработки виде: исходный код, ресурсы, манифест и файлы сборки вынесены в отдельные логические блоки. Такой режим упрощает ориентирование в проекте, но отображает не все реальные каталоги и файлы на диске - часть структуры скрыта или объединена, чтобы не перегружать разработчика лишними деталями.

В этом представлении обычно выделяют несколько основных разделов: manifests (файлы манифеста приложения), kotlin+java (исходный код, включая основное пространство имён и каталоги тестов), res (ресурсы приложения: разметки, изображения, строки, стили и др.), а также Gradle Scripts (скрипты и настройки сборки Gradle проекта и модуля). В верхней части панели расположены элементы управления: выпадающий список с выбором режима отображения структуры (например, Android/Project и другие варианты), команды для работы с деревом (быстрое раскрытие/сворачивание узлов, переход к нужному элементу) и действия, связанные с отображением/добавлением элементов в структуру проекта.

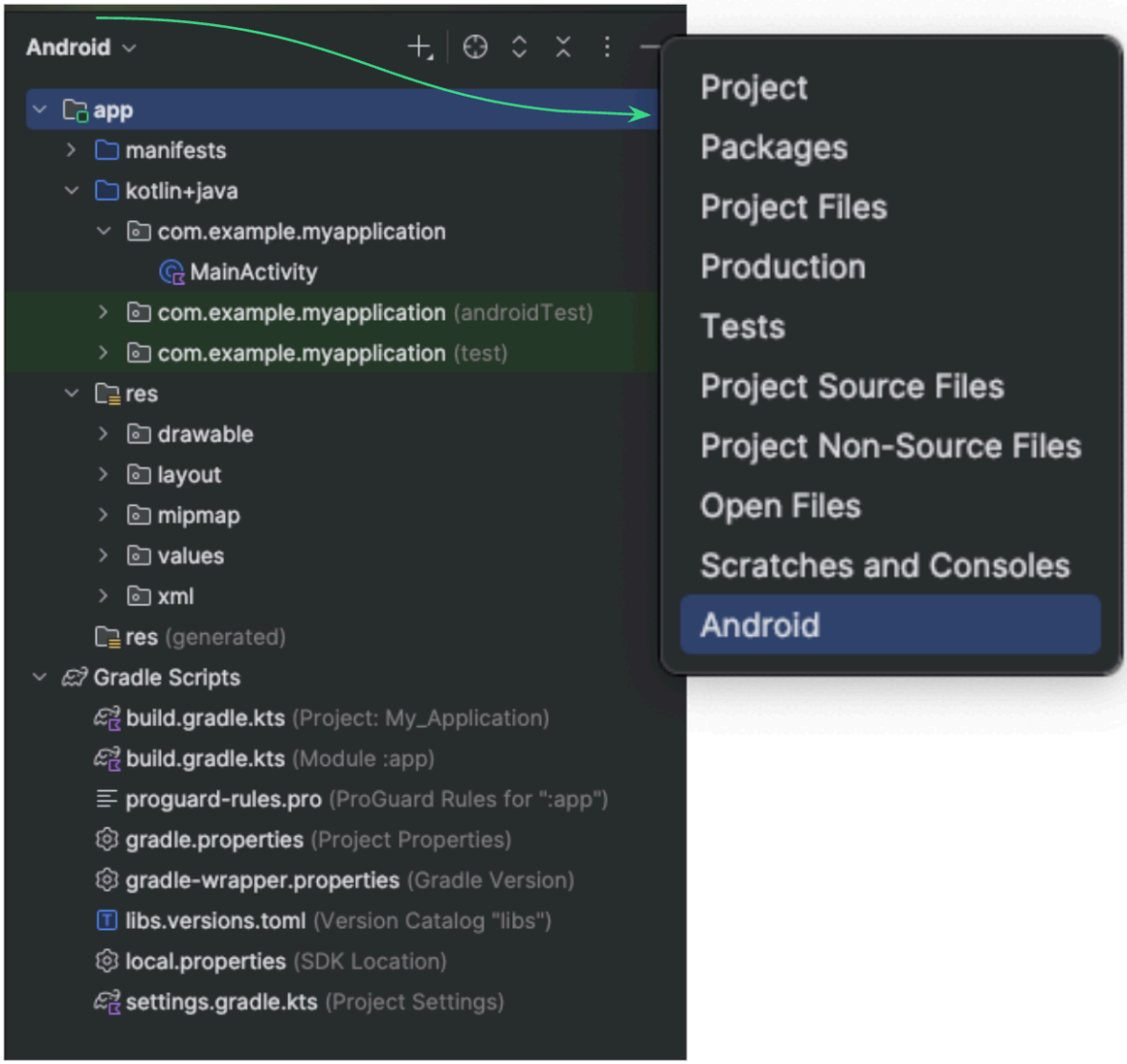


Рисунок 10 - Настройка режима структуры проекта

По нажатию на выпадающий список в верхней части панели можно переключать режим представления структуры проекта. Эти режимы меняют способ группировки и фильтрации файлов, чтобы удобнее решать конкретные задачи.

- Android - логическое представление, ориентированное на Android-разработку: группировка по манифесту, исходникам, ресурсам и файлам сборки.
- Project - представление структуры «как на диске», далее рассматривается отдельно.
- Packages - представление, сфокусированное на пакетах исходного кода: удобно для навигации по namespace и классам без акцента на физические каталоги.
- Project Files - более «файловый» режим, близкий к структуре на диске, с акцентом на реальные папки и файлы проекта.
- Production - показывает материалы, относящиеся к основному (production) коду и ресурсам приложения, без тестовых компонентов.
- Tests - отображает тестовые части проекта (например, unit-тесты и android-тесты), чтобы работать с тестами отдельно от основного кода.
- Project Source Files - фильтрованный вид, где остаются только исходные файлы (код), без несвязанных служебных и конфигурационных элементов.

- Project Non-Source Files - противоположный режим: показывает в основном «не исходники» (конфиги, скрипты, настройки, ресурсы и прочие вспомогательные файлы).
- Open Files - список файлов, которые сейчас открыты в редакторе, для быстрого переключения между ними.
- Scratches and Consoles - временные файлы и консоли для экспериментов, заметок и коротких фрагментов кода, не входящих в проект как часть репозитория.

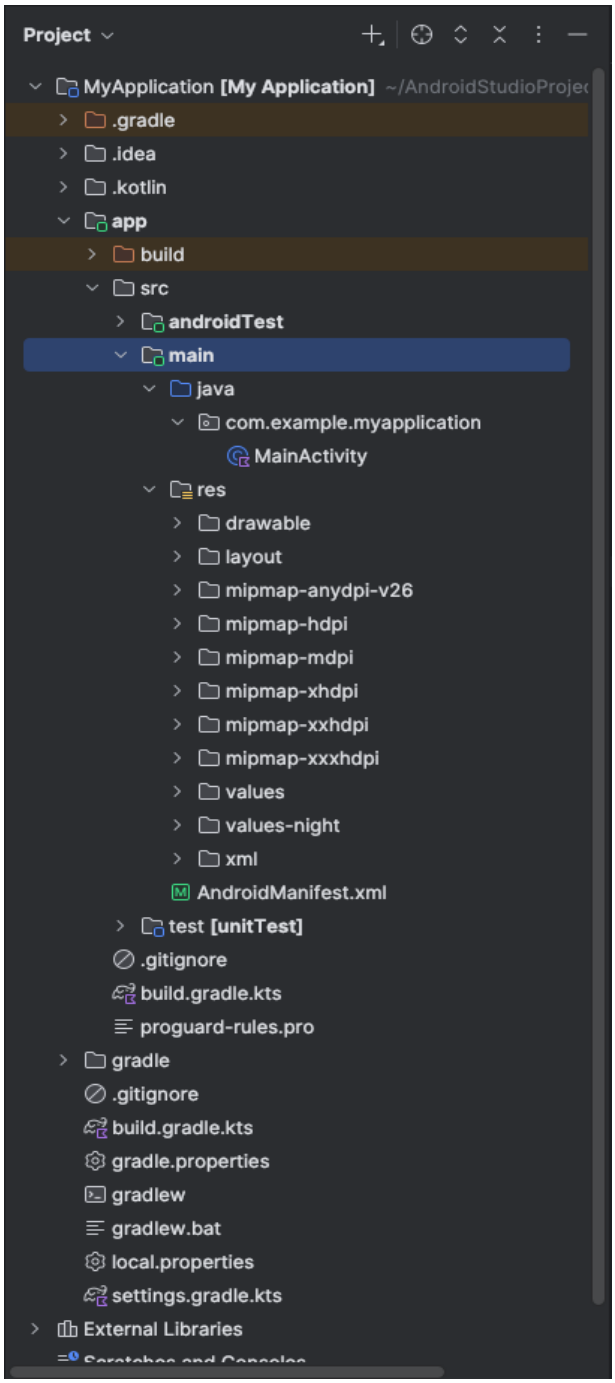


Рисунок 11 - Файловый вид Project

В режиме Project структура отображается в «файловом» виде - так, как проект реально организован на диске. В отличие от представления Android, где элементы сгруппированы по логическим разделам, здесь видны фактические каталоги и служебные файлы проекта: корневые папки, конфигурации Gradle, файлы настроек, а также полная иерархия модулей (например, app), исходников (src/main/java), ресурсов (src/main/res) и

тестов. Такое представление даёт более полную картину состава проекта и помогает понимать, какие именно файлы и каталоги участвуют в сборке и работе приложения, но при этом содержит больше технических деталей, чем «упрощённый» Android-вид.

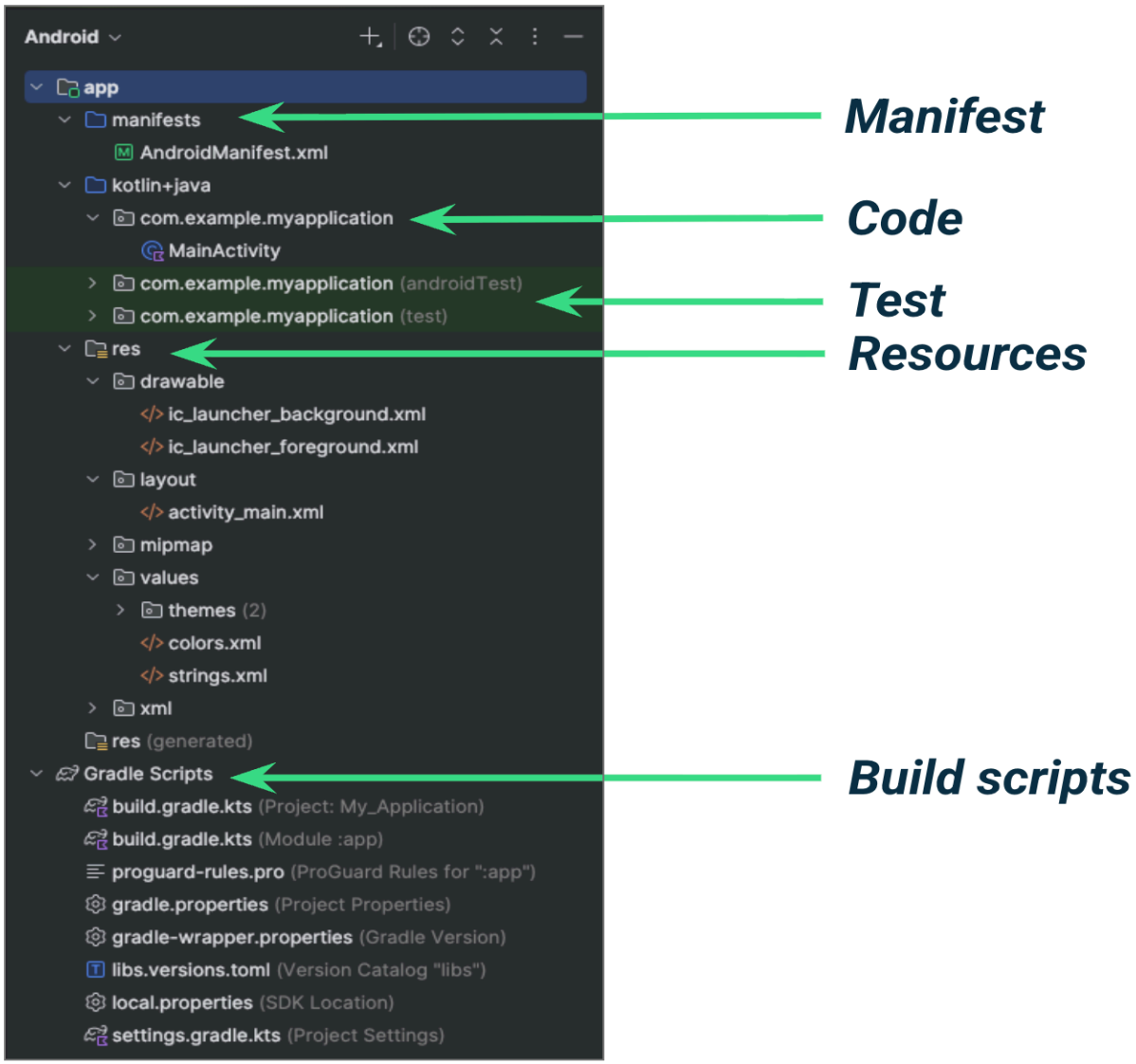


Рисунок 12 - Основные пакеты

AndroidManifest.xml (Manifest) - основной конфигурационный файл приложения, который описывает его «паспорт» для операционной системы Android. В манифесте указываются компоненты приложения, их параметры и точки входа (в том числе стартовый экран), требования к устройству и системе, а также разрешения (permissions), необходимые приложению для работы (например, доступ в интернет). На основе манифеста система понимает, какие части приложения можно запускать, как они связаны между собой и какие ограничения/возможности нужно учитывать при установке и выполнении приложения.

Code (kotlin+java) - раздел с исходным кодом приложения. Внутри пространства имён (например, `com.example.myapplication`) размещаются классы Kotlin/Java, из которых строится логика приложения. В типовом проекте здесь находится стартовый класс `MainActivity` и другие компоненты, которые добавляются в процессе разработки. Код относится к модулю app (основному модулю Android-приложения), который собирается в установочный пакет и содержит всю клиентскую часть проекта.

Resources (res) - раздел ресурсов приложения, где хранятся файлы интерфейса и вспомогательные данные, используемые кодом: разметки экранов (layout), изображения и графика (drawable, mipmap), строки, цвета и стили (values) и другие ресурсные файлы. Эти материалы подключаются к коду через механизм ресурсов Android и используются для построения UI и настройки внешнего вида приложения.

Build scripts (Gradle Scripts) - файлы конфигурации сборки, которые управляют тем, как проект компилируется и упаковывается. Ключевыми являются два скрипта: `build.gradle.kts` уровня проекта (задаёт общие настройки сборки, репозитории и плагины для всего проекта) и `build.gradle.kts` уровня модуля `app` (описывает параметры конкретного приложения: зависимости, версии SDK, типы сборок, настройки упаковки и прочие параметры, влияющие на результат сборки).

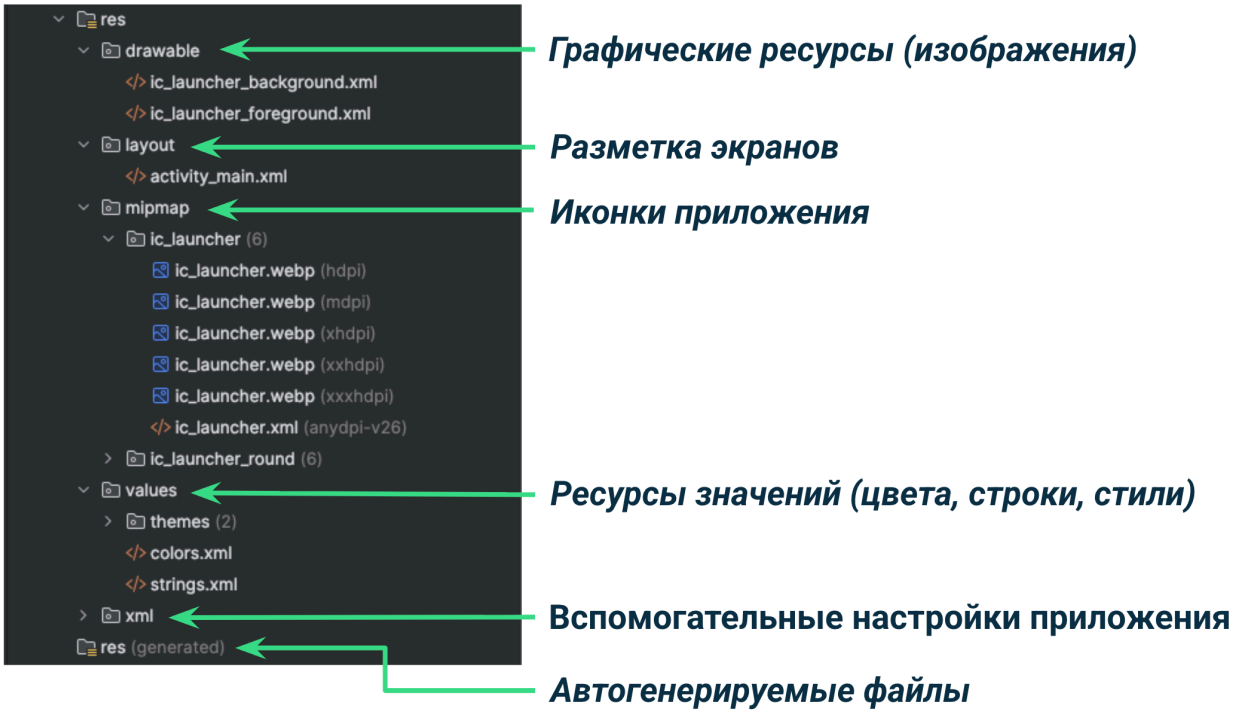


Рисунок 13 - Навигация по ресурсам проекта

Теперь подробнее рассмотрим папку res, где хранятся ресурсы приложения. В drawable размещаются графические ресурсы (изображения, а также drawable-ресурсы в виде XML, например, формы и векторная графика). В layout находятся XML-файлы разметки экранов и отдельных UI-компонентов (например, `activity_main.xml`). Папка mipmap предназначена для иконок приложения (launcher icon) и обычно содержит варианты для разных плотностей экрана. В values хранятся ресурсы значений: строки (`strings.xml`), цвета (`colors.xml`), стили и темы (`themes.xml`), а также другие типы значений (например, размеры и атрибуты). В xml размещаются вспомогательные XML-конфигурации приложения (например, настройки, конфиги провайдеров, правила и другие служебные описания). Отдельно может присутствовать раздел res (generated) - автогенерируемые файлы ресурсов, которые создаются инструментами сборки и подключаемыми библиотеками.

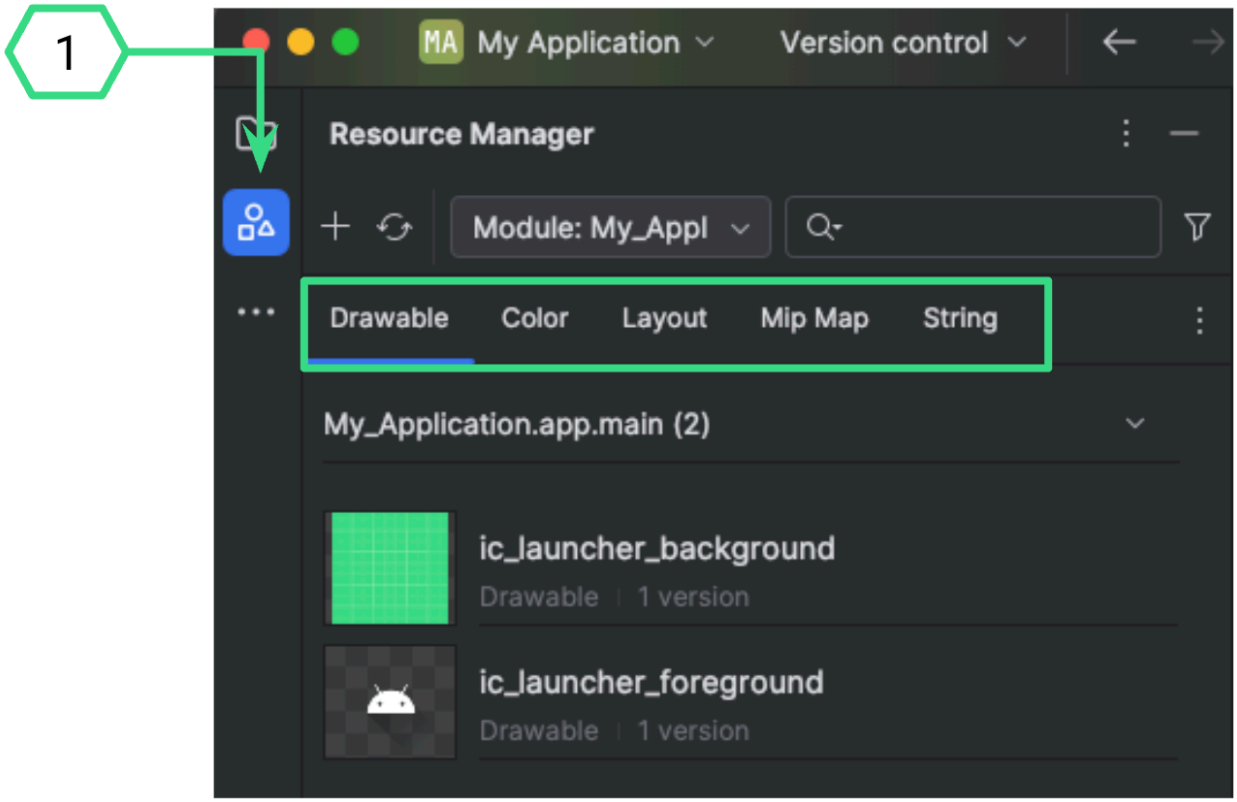


Рисунок 14 - Открытие менеджера ресурсов

Кнопка Resource Manager открывает встроенный менеджер ресурсов Android Studio, который позволяет в удобном виде просматривать содержимое ресурсов проекта. В верхней части окна можно выбрать модуль, а затем переключаться между типами ресурсов (например, Drawable, Color, Layout, Mip Map, String), чтобы быстро увидеть нужную категорию без ручного поиска по папкам res. Это особенно полезно в крупных проектах: вместо того чтобы открывать файлы по одному и постоянно перемещаться по структуре каталогов, можно просмотреть весь набор ресурсов выбранного типа в одном месте, сразу видя список элементов и их миниатюры/предпросмотр, что ускоряет навигацию и снижает вероятность ошибок при выборе нужного ресурса.

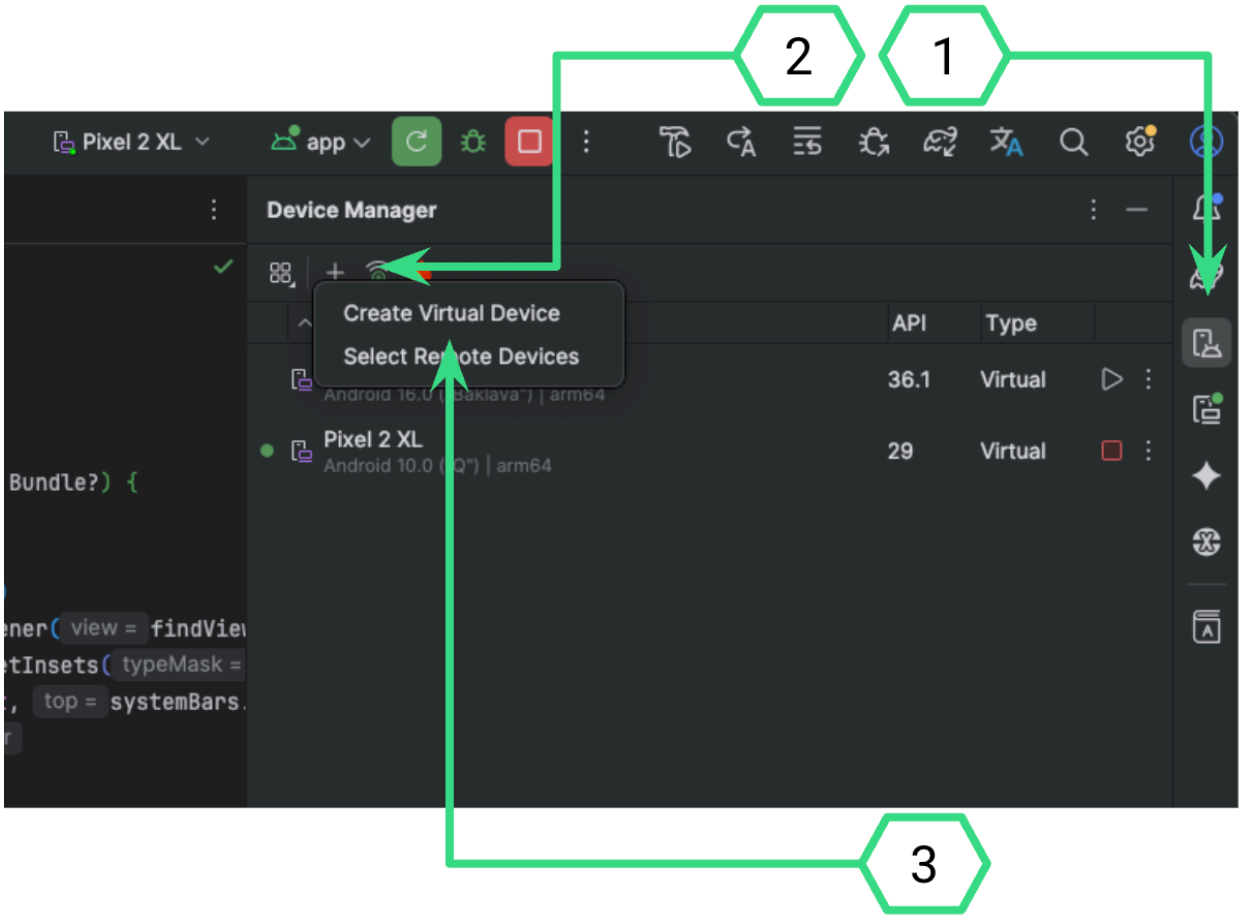


Рисунок 15 - Создание виртуального устройства

После знакомства со структурой проекта можно перейти к настройке эмулятора для запуска и проверки приложения. Для создания эмулятора используются шаги, показанные на рисунке: сначала открывается Device Manager (панель управления устройствами) на правой боковой панели среды. Затем нажимается кнопка «+» для добавления устройства, после этого в меню выбирается вариант Create Virtual Device (создание виртуального устройства) либо Select Remote Devices (выбор удалённого устройства для подключения и запуска).

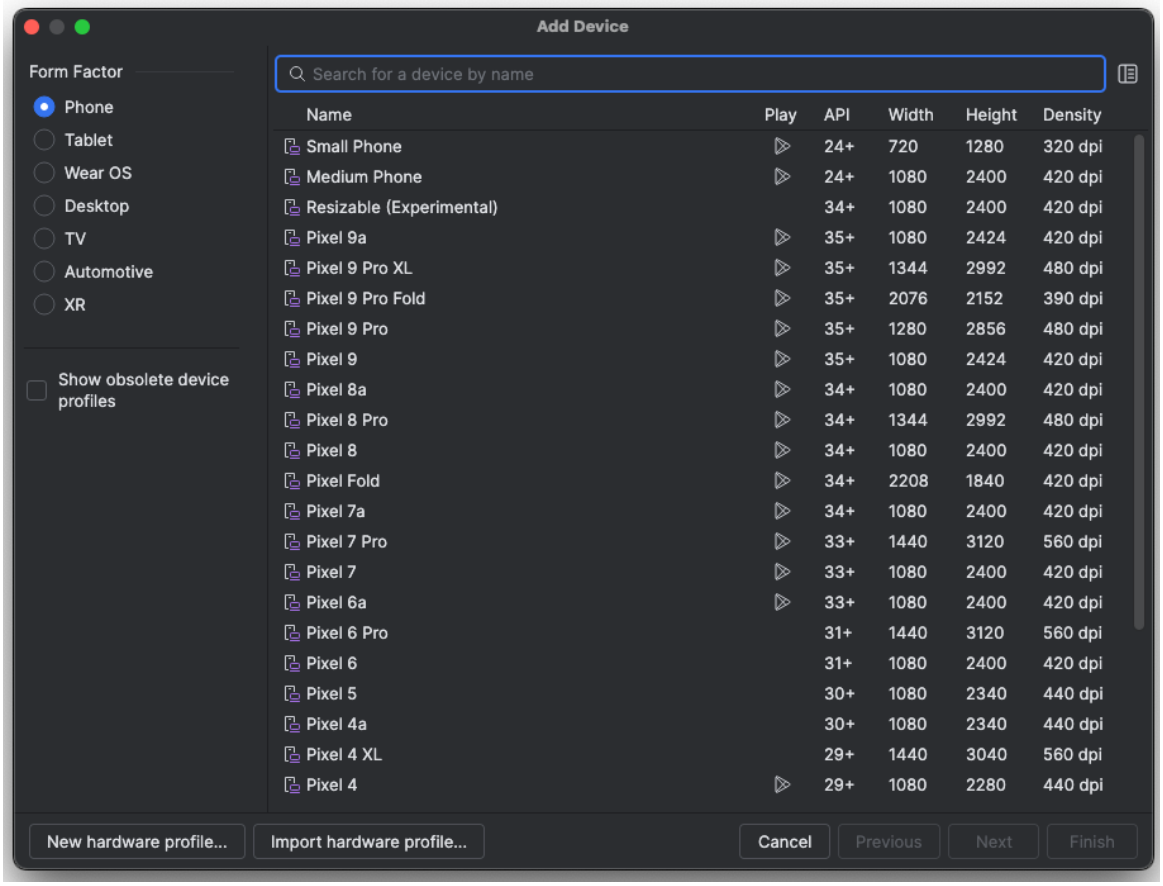


Рисунок 16 - Выбор устройства

В окне Add Device сначала выбирается форм-фактор устройства: Phone (смартфоны), Tablet (планшеты), Wear OS (умные часы), Desktop (настольный режим/эмуляция под большие экраны), TV (Android TV), Automotive (автомобильные системы), XR (устройства расширенной/смешанной реальности). Далее отображается список готовых профилей устройств - в типовой конфигурации это преимущественно модели линейки Google Pixel и несколько базовых «обобщённых» профилей (например, Small/Medium Phone). Для каждого профиля показываются характеристики: Play - наличие образа с Google Play/Google-сервисами (то есть сборка, ориентированная на использование сервисов Google), API - уровень Android API для выбранного образа системы, Width/Height - разрешение экрана в пикселях, Density - плотность пикселей (dpi), влияющая на масштаб интерфейса.

Дополнительно доступны две команды для работы с профилями устройств: New Hardware Profile… - создание собственного профиля с более детальной настройкой параметров виртуального устройства, и Import Hardware Profile… - импорт готового профиля устройства из внешнего источника.

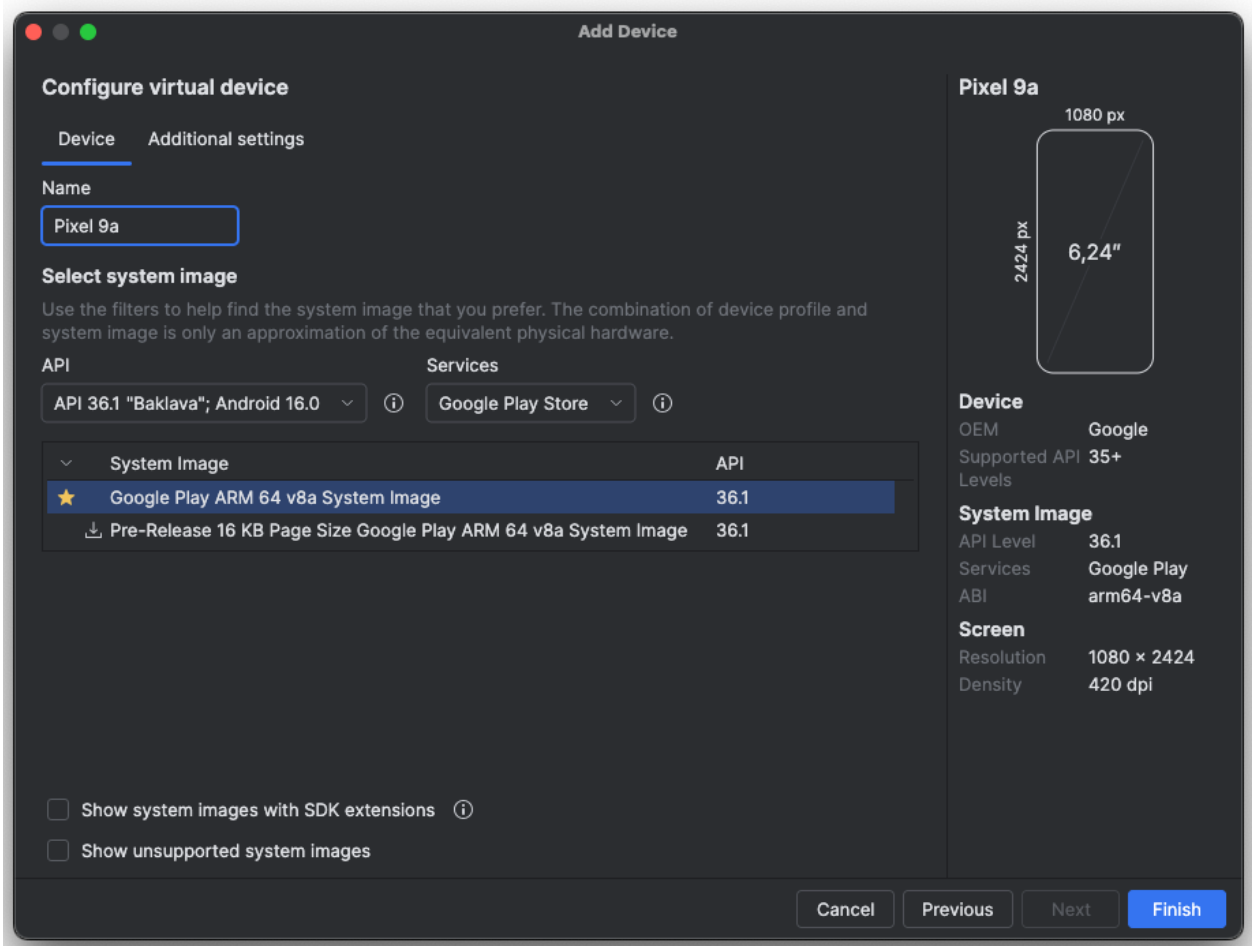


Рисунок 17 - Настройка виртуального устройства

В этом окне выполняется настройка виртуального устройства перед созданием эмулятора. В поле Name задаётся имя профиля - именно так эмулятор будет отображаться в списке устройств. Ниже выбирается System Image (образ системы), то есть конкретная сборка Android, на которой будет запускаться виртуальное устройство: образы различаются уровнем API, типом сборки и наличием сервисов Google. В списке могут встречаться варианты с Google Play (как на рисунке — Google Play Store), которые включают Google-сервисы и магазин, а также образы без Google Play (обычно AOSP/Google APIs), используемые для других сценариев тестирования.

Правая часть окна показывает итоговые характеристики выбранного профиля и образа: модель устройства, сведения об уровне API и выбранном образе системы, а также параметры экрана - разрешение (в пикселях), плотность (dpi) и диагональ, что помогает заранее понимать, в каких условиях будет проверяться интерфейс и поведение приложения на эмуляторе.



Рисунок 18 - Запуск эмулятора

После завершения создания виртуального устройства оно появляется в Device Manager в списке доступных эмуляторов (вместе с ранее созданными профилями). Чтобы запустить нужный эмулятор, выбирают соответствующую строку и нажимают кнопку запуска (значок треугольника) в правой части этой строки - после этого Android Studio запускает выбранное виртуальное устройство.

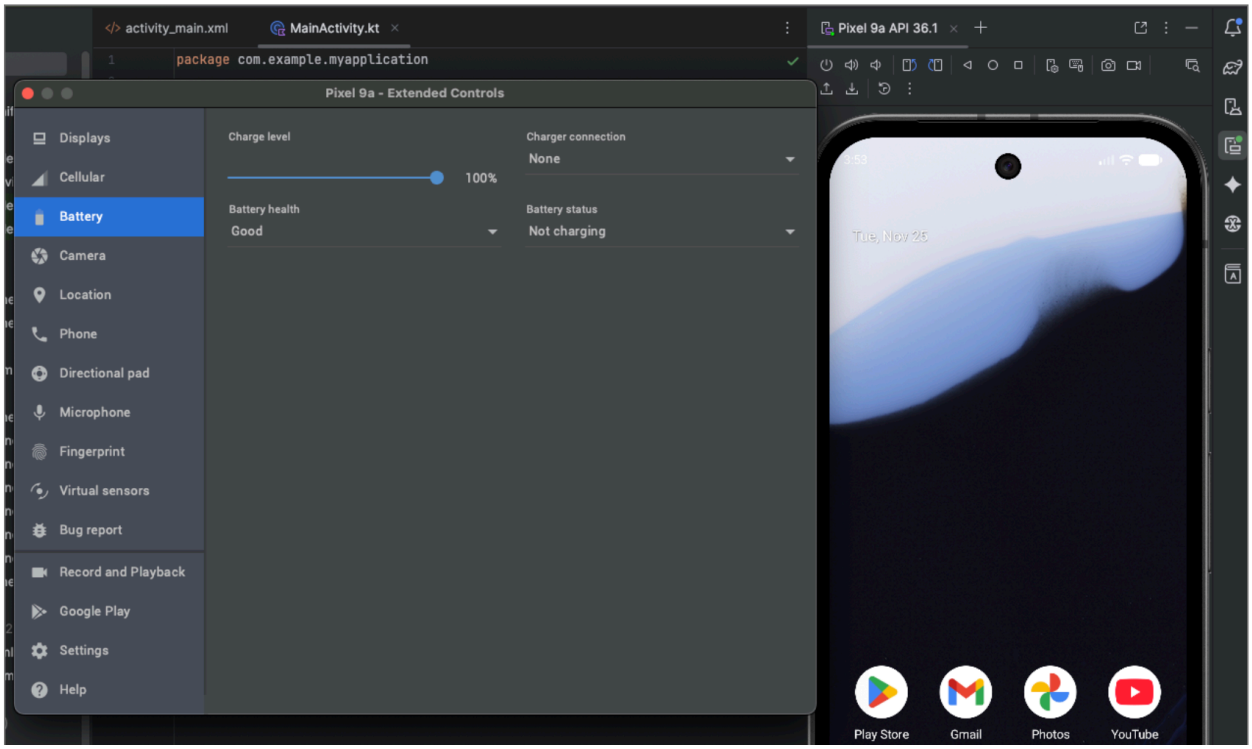


Рисунок 19 - Окно расширенных настроек эмулятора

Запущенный эмулятор по умолчанию отображается в правой части окна Android Studio и работает как виртуальный смартфон, на котором можно устанавливать и запускать приложение, проверять интерфейс и поведение на выбранной версии Android. Сверху от экрана эмулятора расположена панель управления устройством: она включает действия,

которые чаще всего нужны при тестировании - имитацию кнопки питания (включение/выключение и блокировка), управление громкостью, поворот экрана (смена ориентации), а также навигационные кнопки Android (Back, Home, Recents) для проверки сценариев переходов и поведения приложения при сворачивании. В этой же панели обычно доступны быстрые действия вроде создания скриншота и записи экрана, что удобно для отчётов по лабораторной работе и фиксации ошибок.

Отдельно в эмуляторе предусмотрено окно расширенных настроек и инструментов (Extended controls), где можно моделировать параметры устройства и внешние условия. Наиболее полезные для практики пункты: Battery (уровень заряда и режим зарядки), Cellular/Network (состояние сети и ограничения подключения), Location (задание координат GPS для проверки функций геолокации), Camera/Microphone (подключение источников для проверки доступа к камере и микрофону), а также Fingerprint (имитация биометрии на поддерживаемых конфигурациях). Эти настройки позволяют воспроизводить типовые ситуации, которые сложно постоянно проверять на реальном устройстве, и быстрее находить ошибки в поведении приложения.

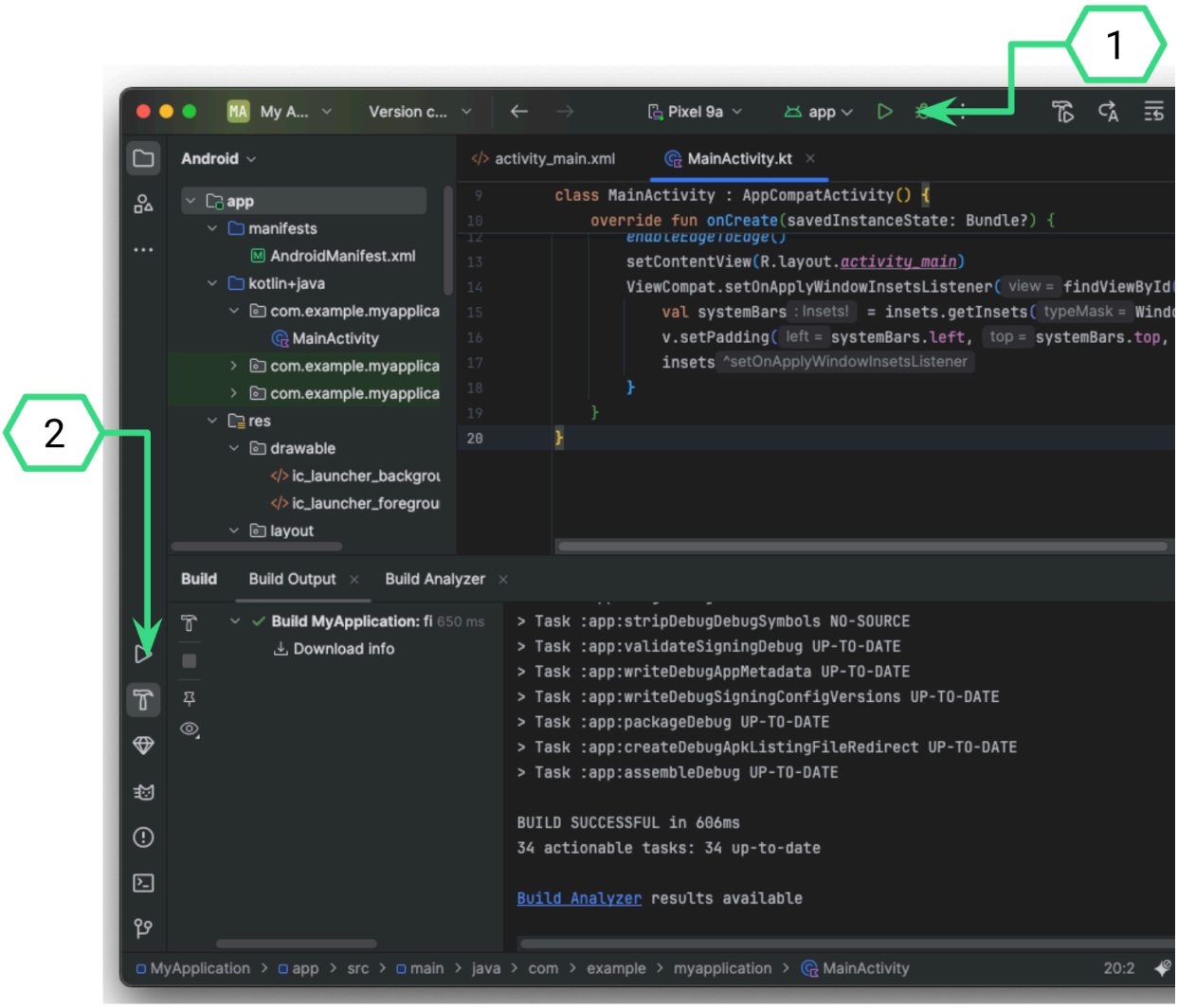


Рисунок 20 - Сборка приложения

Чтобы проверить работу приложения после внесённых изменений в код, в верхней панели Android Studio используется кнопка запуска (зелёная стрелка Run). После нажатия начинается процесс сборки и запуска: в нижней части окна автоматически открывается вкладка Build Output, где отображается журнал сборки проекта.

В Build Output выводятся этапы сборки и служебные сообщения системы Gradle: какие задачи выполняются (Gradle tasks), какие модули собираются, какие действия происходят (компиляция, упаковка, проверка ресурсов и т. п.), а также возможные предупреждения и ошибки. В правой части журнала обычно виден список выполняемых Gradle-задач и их статусы, что помогает понять, на каком шаге находится сборка или где она остановилась при ошибке. При успешной сборке в конце появляется итоговое сообщение вида BUILD SUCCESSFUL, а также указывается время выполнения сборки (сколько миллисекунд или секунд занял процесс).

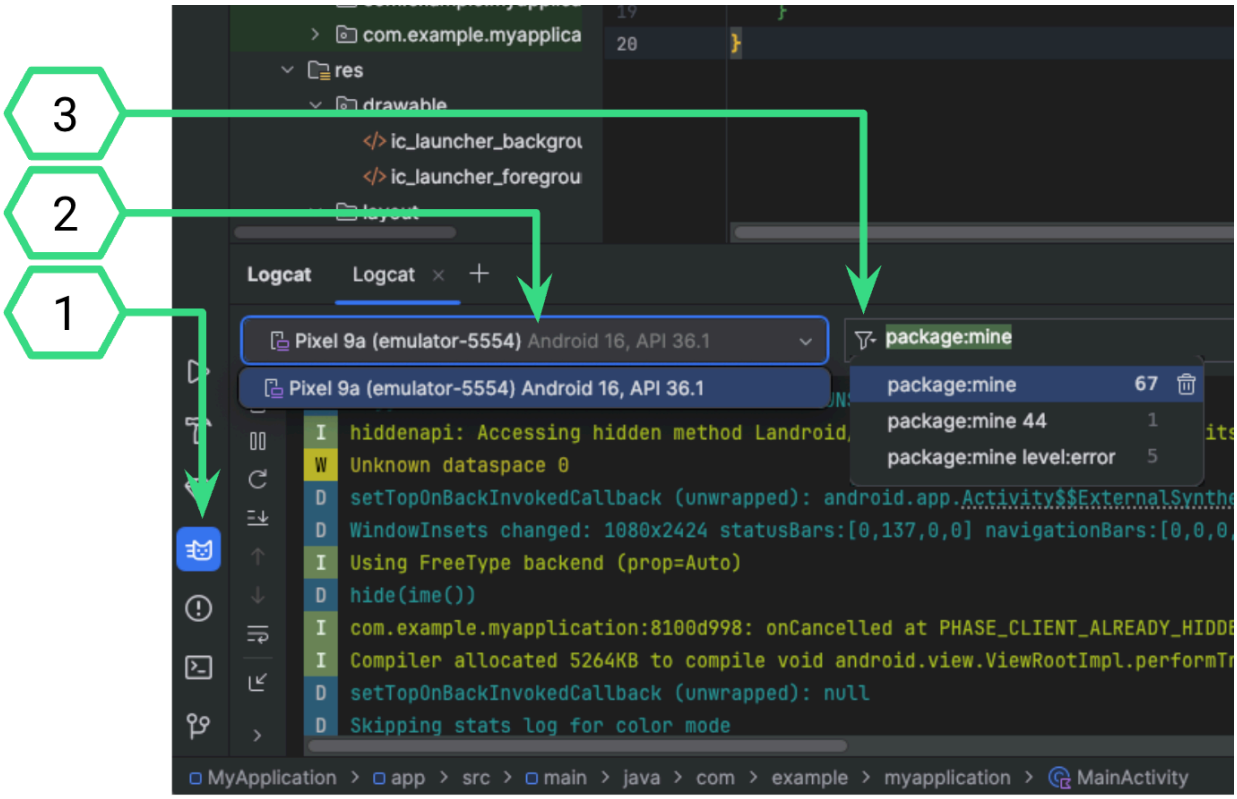


Рисунок 21 - Logcat

Logcat - это инструмент для просмотра системных и прикладных логов, то есть текстовых сообщений, которые во время работы приложения выводят сама операционная система Android, среда выполнения и код приложения. Логи используются для диагностики: по ним удобно понимать, что происходит в приложении во время запуска, навигации, сетевых запросов, обработки ошибок и исключений. В частности, при падении приложения (crash) Logcat показывает стек вызовов и причину ошибки, а при отладке - сообщения, которые разработчик специально выводит в код (например, чтобы проверить, что метод вызвался и какие значения пришли).

На рисунке 21 показан типовой порядок работы с Logcat:

1. В нижней панели Android Studio открывается вкладка Logcat (через соответствующую кнопку в левом нижнем блоке);
2. В верхней части окна Logcat выбирается активное устройство, с которого читаются логи (это важно, потому что одновременно могут быть подключены несколько устройств/эмуляторов, и у каждого будет свой поток сообщений);
3. Для удобства поиска используется фильтрация: можно ограничить вывод по приложению (по package name), по тексту сообщения или по тегам/уровням важности.

В Logcat чаще всего используют фильтрацию по уровню логов: Verbose, Debug, Info, Warn, Error, Assert - это основные категории, которые помогают быстро отделить служебные сообщения от предупреждений и ошибок. Дополнительно часто фильтруют по тегу (Tag), который задаётся источником сообщения: его могут выводить системные компоненты или сам разработчик (через указание тега в логировании), поэтому по тегам удобно выделять сообщения конкретного модуля или экрана приложения.

Система контроля версий

Git - распределённая система контроля версий, предназначенная для ведения истории изменений в исходном коде и связанных файлах проекта. Она фиксирует состояние проекта во времени и позволяет отслеживать, кто и когда внёс изменения, какие именно строки были изменены и по какой причине. За счёт распределённой модели каждый разработчик работает с полноценной локальной копией репозитория и его истории, поэтому проект не зависит от единственного «центрального» компьютера: даже без подключения к сети можно сохранять изменения и анализировать историю. Git применяется для надёжного хранения результатов работы, упрощения совместной разработки и снижения рисков: при ошибках можно вернуться к предыдущему состоянию, а при параллельной работе нескольких участников - синхронизировать изменения и поддерживать единый актуальный код проекта.

В базовом рабочем цикле с Git чаще всего используются три команды, которые понадобятся и при выполнении лабораторных работ: git add, git commit и git push. Команда git add добавляет выбранные изменённые файлы в индекс (staging area) - то есть помечает, какие именно изменения должны попасть в ближайшую фиксацию. Команда git commit сохраняет эти подготовленные изменения в локальной истории репозитория в виде отдельного коммита и сопровождает его комментарием (сообщением), по которому затем удобно понимать смысл внесённых правок. Команда git push отправляет сделанные локально коммиты в удалённый репозиторий, чтобы изменения стали доступны на сервере и могли быть использованы для хранения, проверки и совместной работы.

Ветвление в Git - это механизм, который позволяет вести разработку параллельными линиями, не затрагивая основную историю проекта: вы можете изолированно вносить изменения, проверять гипотезы и развивать отдельную задачу, сохраняя «базовую» версию кода неизменной. Обычно ветки создают под конкретный тип работы: например, feature/* (разработка новой функциональности), bugfix/* (исправление ошибки), hotfix/* (срочное исправление), либо любые другие названия, принятые в команде. Создать новую ветку можно командой git branch <name>, а перейти на неё - командой git checkout <name>. Такой подход помогает упорядочивать изменения, разделять задачи и поддерживать проект в предсказуемом состоянии, когда работа ведётся последовательно или параллельно несколькими участниками.

Pull request (PR) - это запрос на включение изменений из одной ветки в другую (чаще всего - в основную ветку проекта) через интерфейс удалённого репозитория. Он используется как контролируемая «точка входа» изменений в общий код: перед слиянием команда может просмотреть, обсудить и проверить правки, чтобы снизить риск ошибок и сохранить единые стандарты разработки. В pull request обычно видно, какие коммиты и файлы входят в набор изменений, какие строки были изменены, а также автоматически формируется сравнение (diff) с возможностью оставлять комментарии к конкретным строкам кода. Как правило, перед слиянием PR проходит код-ревью другими разработчиками: они могут запросить правки, предложить улучшения или подтвердить готовность изменений к интеграции.

В pull request настраиваются и фиксируются ключевые параметры совместной работы: выбираются ветка-источник и целевая ветка слияния, задаются заголовок и описание (что сделано и зачем), при необходимости добавляются метки (labels), назначаются ответственные участники (reviewers/assignees), а также привязываются задачи

или тикеты. В большинстве репозиториев PR связан с автоматическими проверками: сборкой, запуском тестов и статическим анализом, результаты которых отображаются прямо в запросе и влияют на возможность слияния. Такой процесс делает интеграцию изменений предсказуемой: команда заранее видит содержание правок, обсуждает спорные моменты, подтверждает качество и только затем объединяет изменения с основной кодовой базой.

GitHub - это веб-платформа для размещения и совместной работы с Git-репозиториями, которая выступает как удалённое хранилище кода и предоставляет инструменты для командной разработки: просмотр истории изменений, управление доступом, обсуждение правок и работу с pull request через интерфейс браузера. В данной методической рекомендации GitHub используется как основной пример удалённого репозитория, потому что он широко распространён и удобен для учебных проектов, однако это не является ограничением: при выполнении лабораторных работ студент может использовать любую другую платформу, поддерживающую Git (например, GitLab, GitFlic и т.д.), если она обеспечивает удалённое хранение кода и обмен изменениями.

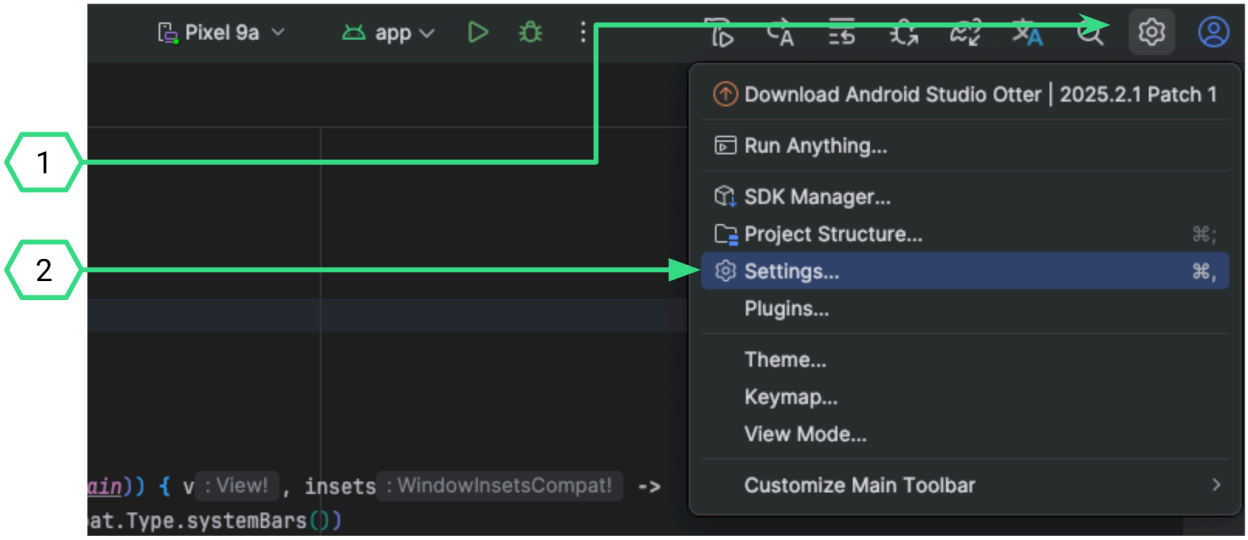


Рисунок 22 - Открытие настроек IDE

Чтобы добавить учётную запись GitHub в Android Studio, откройте меню настроек среды: в верхней панели нажмите на главное меню (кнопка с дополнительными действиями/меню приложения), затем в выпадающем списке выберите пункт Settings….

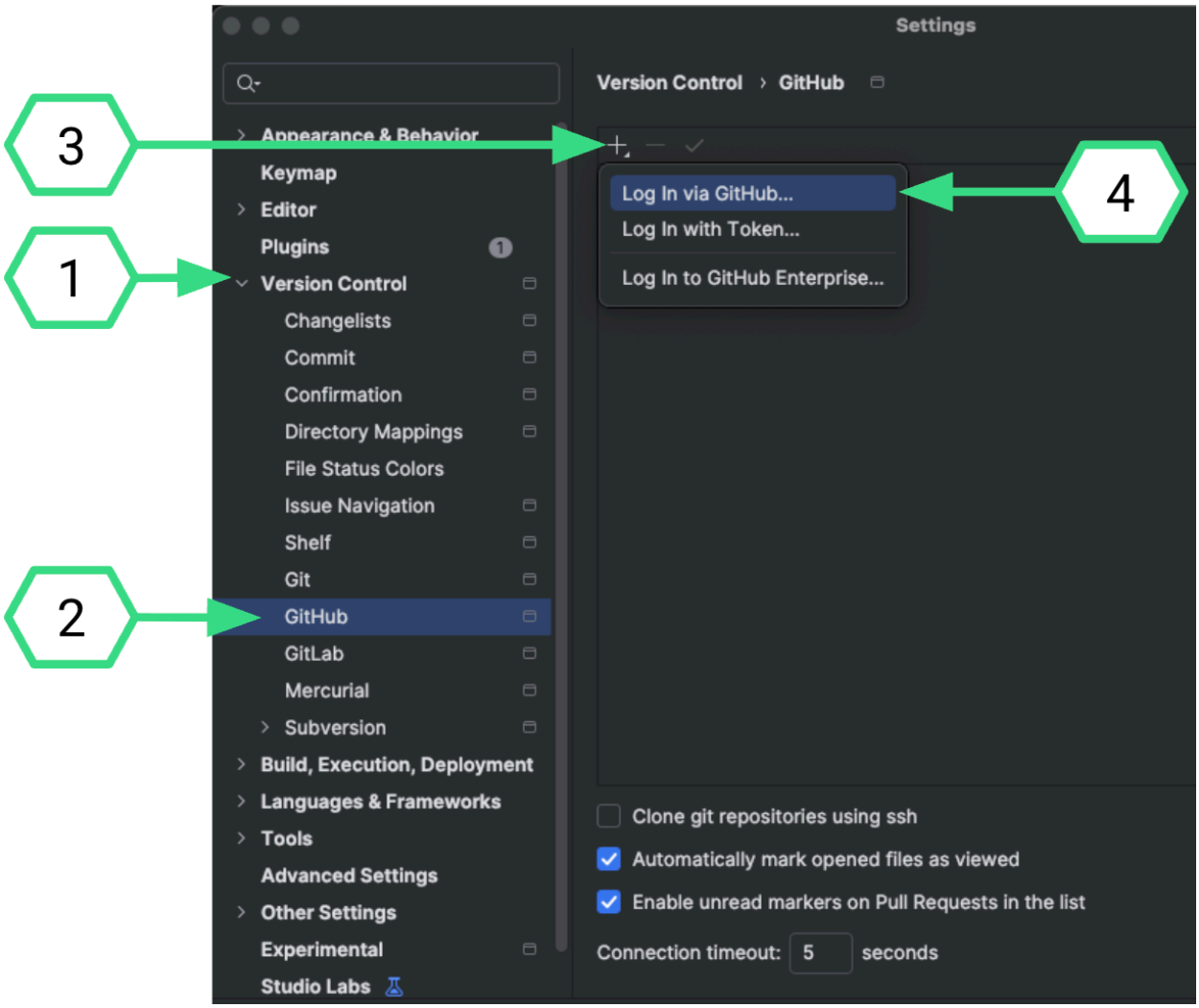


Рисунок 23 - Добавление аккаунта

В окне настроек Android Studio подключение учётной записи выполняется через раздел Version Control → GitHub (аналогично можно выбрать GitLab, если используется другой сервис). Если аккаунт ещё не добавлен, нажмите кнопку «+» и выберите способ авторизации.

- Log in via GitHub… - вход через браузер: Android Studio перенаправляет на страницу GitHub, где вы выполняете авторизацию и подтверждаете доступ для IDE. Этот вариант удобен тем, что не требует вручную создавать и хранить токен, а права доступа выдаются через стандартную процедуру входа и подтверждения.
- Log in with Token… - вход по персональному токену (Personal Access Token): токен заранее создаётся в настройках GitHub и затем вставляется в Android Studio. Этот вариант полезен, когда браузерная авторизация недоступна (например, из-за ограничений сети/политик) или когда требуется явным образом контролировать права и срок действия токена, при этом ответственность за безопасное хранение токена лежит на пользователе.

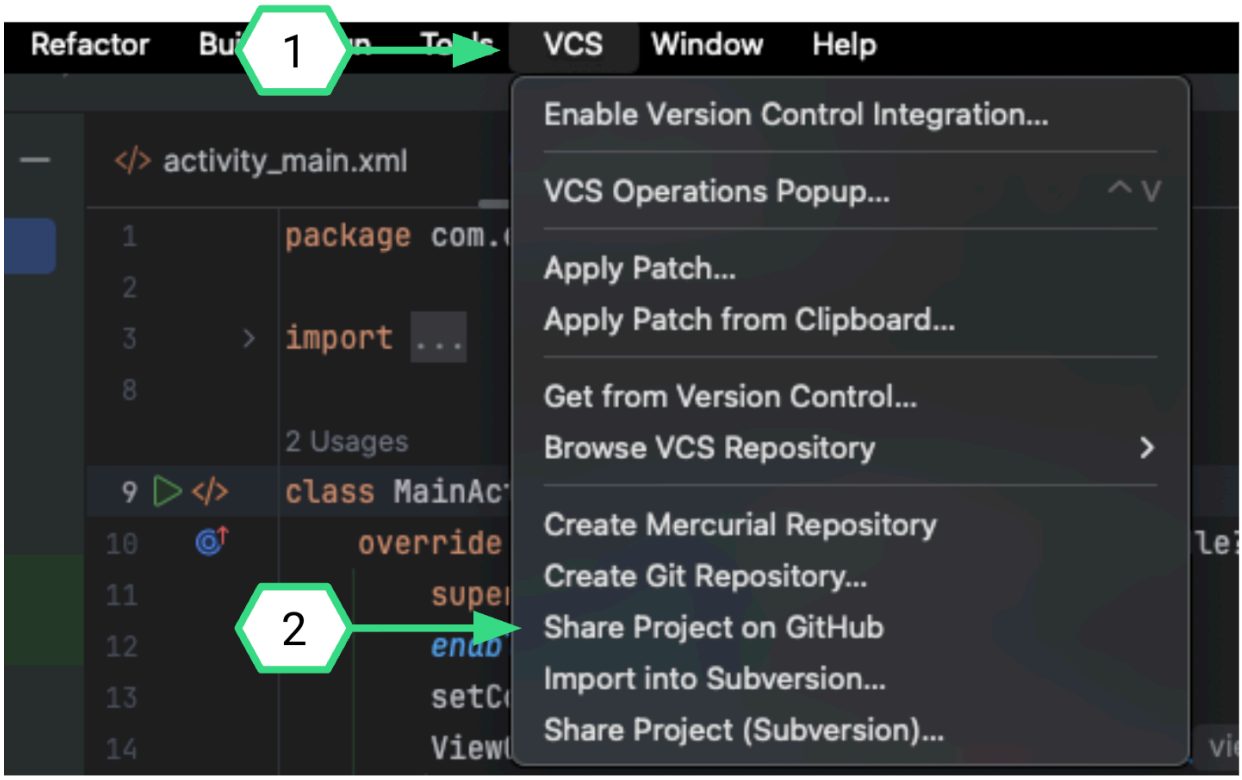


Рисунок 24 - Отправка проекта

Для публикации проекта в удалённый репозиторий используется верхнее меню Android Studio. Строка меню находится в самой верхней части экрана. В этой строке выберите пункт VCS (Version Control System), затем в открывшемся списке нажмите Share Project on GitHub - эта команда создаёт удалённый репозиторий (или связывает проект с репозиторием на GitHub) и отправляет локально созданный проект в GitHub для дальнейшего хранения и совместной работы.

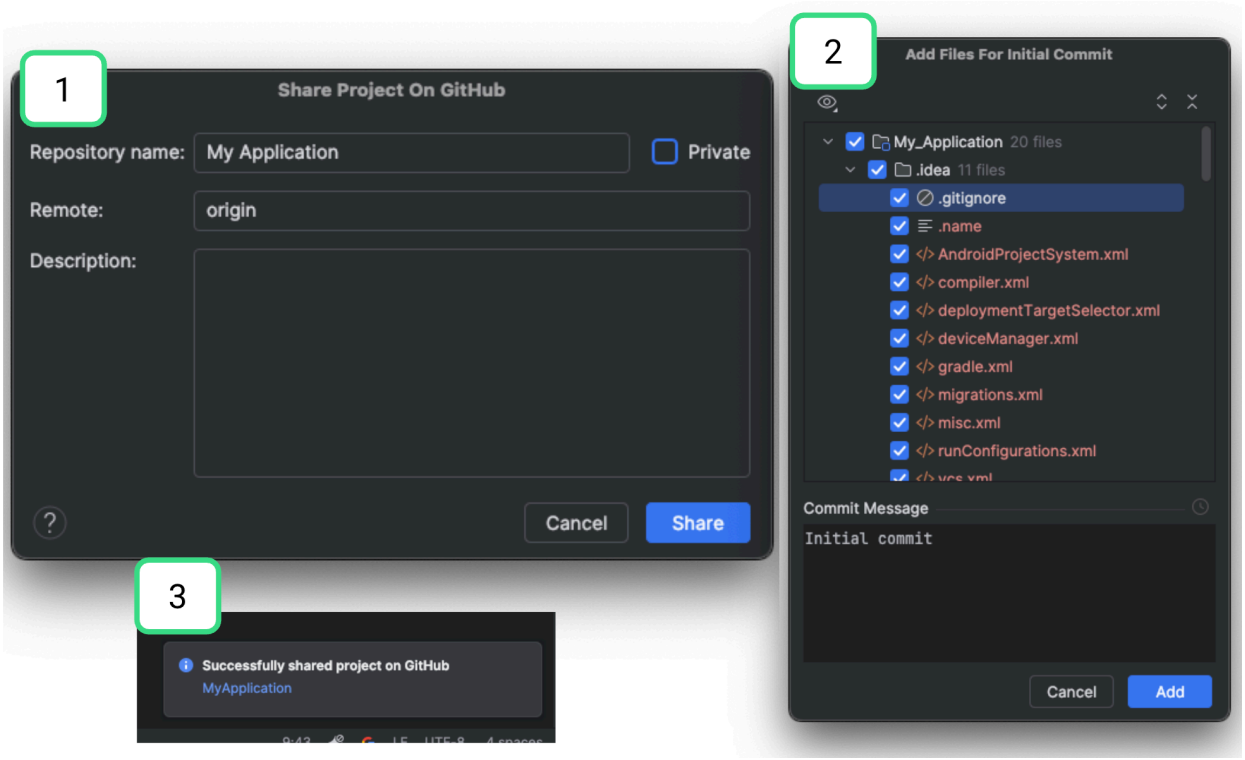


Рисунок 25 - Параметры публикации проекта

В открывшемся окне Share Project on GitHub задаются параметры публикации проекта. Сначала указывается Repository name - имя репозитория, под которым проект будет создан на GitHub. При необходимости включается флаг Private, если репозиторий должен быть доступен только вам (и приглашённым участникам), либо остаётся публичным. Поле Remote задаёт имя удалённого репозитория в локальной конфигурации Git, по умолчанию используется origin - это стандартное имя «основного» удалённого репозитория, но в Git может быть несколько remotes (например, для второго сервера), поэтому они именуются отдельно, чтобы различать, куда отправляются и откуда получаются изменения. В поле Description можно добавить краткое описание проекта - оно будет отображаться на странице репозитория на GitHub.

Далее Android Studio предлагает сформировать первый коммит: в списке выбираются файлы, которые будут включены в начальную отправку (initial commit), а в поле Commit Message задаётся текст первого сообщения коммита (например, Initial commit). После завершения публикации при успешном выполнении Android Studio показывает уведомление в правом нижнем углу о том, что проект успешно отправлен в GitHub (сообщение вида Successfully shared project on GitHub).

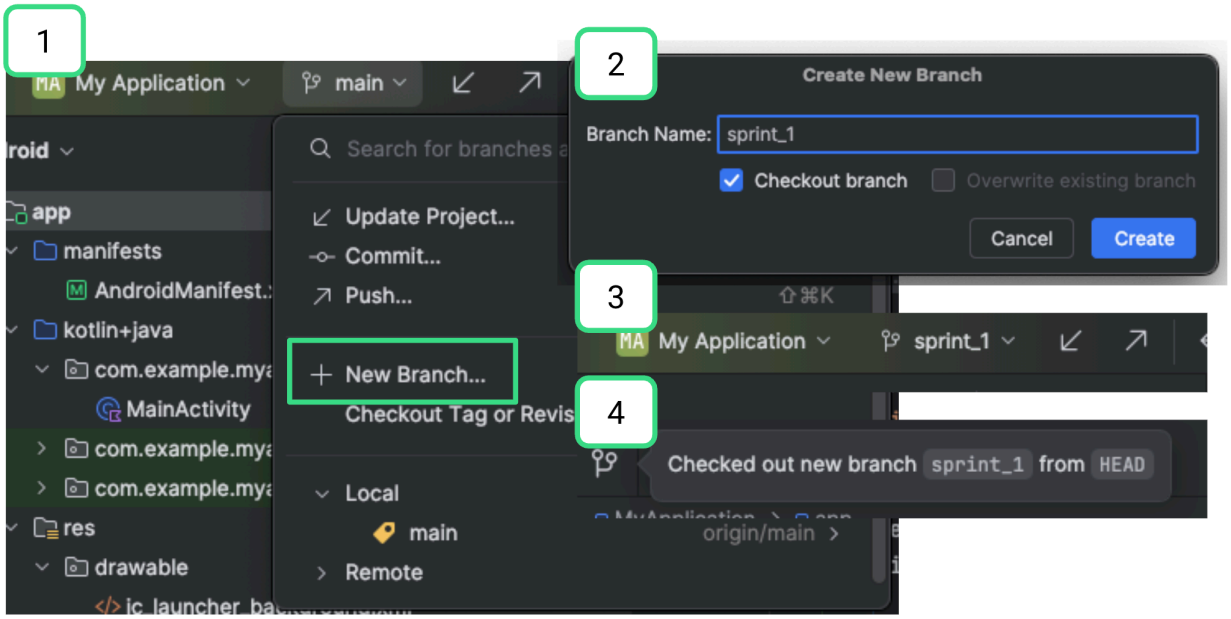


Рисунок 26 - Создание ветки

Также можно создать новую ветку прямо функциональными кнопками в Android Studio. В верхней части окна, в области отображения текущей ветки (на рисунке показана main), нажмите на название ветки - откроется выпадающее меню действий с ветками. В этом меню выберите New Branch… и введите имя ветки (например, sprint_1). Опция Checkout branch означает, что после создания Android Studio сразу переключит рабочее состояние проекта на новую ветку. Если галочку не ставить, ветка будет создана, но активной останется текущая ветка. После нажатия Create в интерфейсе изменится отображаемая текущая ветка, что подтверждает переключение.

Уведомление «Checked out new branch sprint_1 from HEAD» означает, что выполнено переключение (checkout) на новую ветку sprint_1, причём ветка создана от текущего состояния HEAD. В контексте Git HEAD - это ссылка на текущий коммит, на котором находится рабочая копия проекта, то есть новая ветка начинается с того же коммита, который был активен в момент её создания.

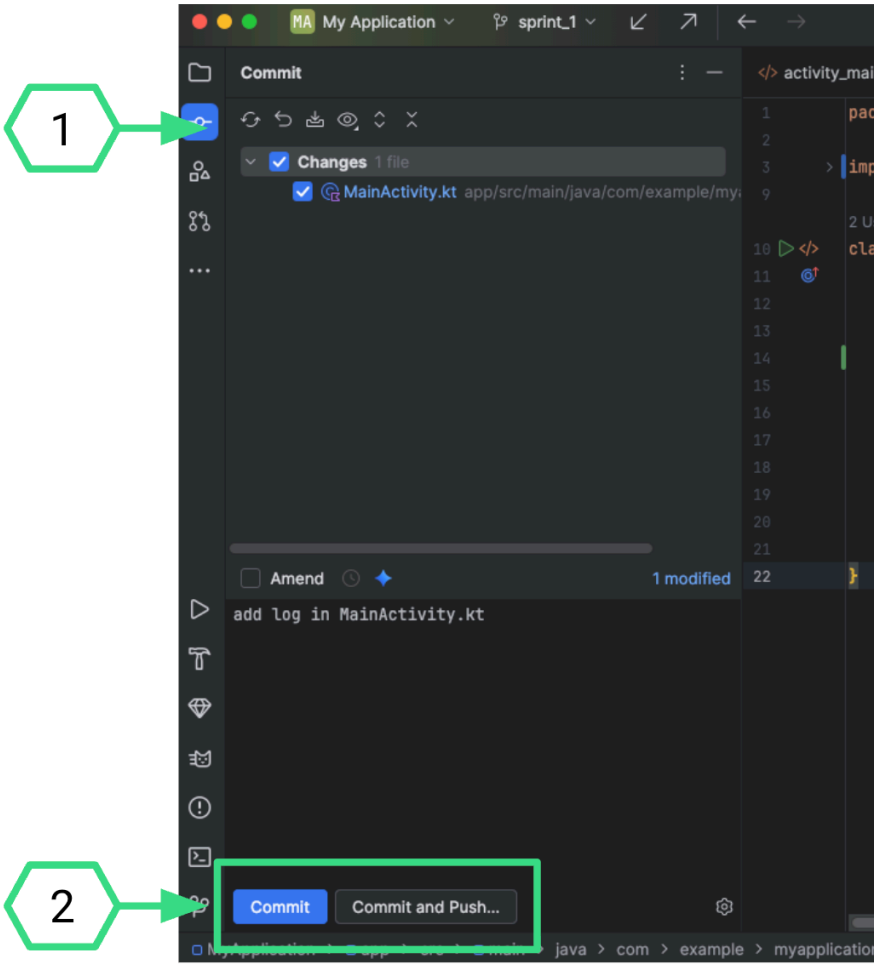


Рисунок 27 - Фиксация изменений

После того как создана новая ветка, в неё вносятся изменения в код или ресурсы проекта. Чтобы эти изменения были зафиксированы в истории Git и появились в удалённом репозитории, их нужно закоммитить и отправить на сервер. На рисунке показано окно Commit, где отображается список изменённых файлов, которые будут включены в коммит. Нажатие кнопки Commit фиксирует выбранные изменения в локальной истории текущей ветки, а команда Commit and Push... дополнительно отправляет этот коммит в удалённый репозиторий. Это нужно, чтобы ветка sprint_1 существовала не только локально, но и в GitHub, и чтобы внесённые изменения были сохранены и доступны для дальнейшей проверки и совместной работы.

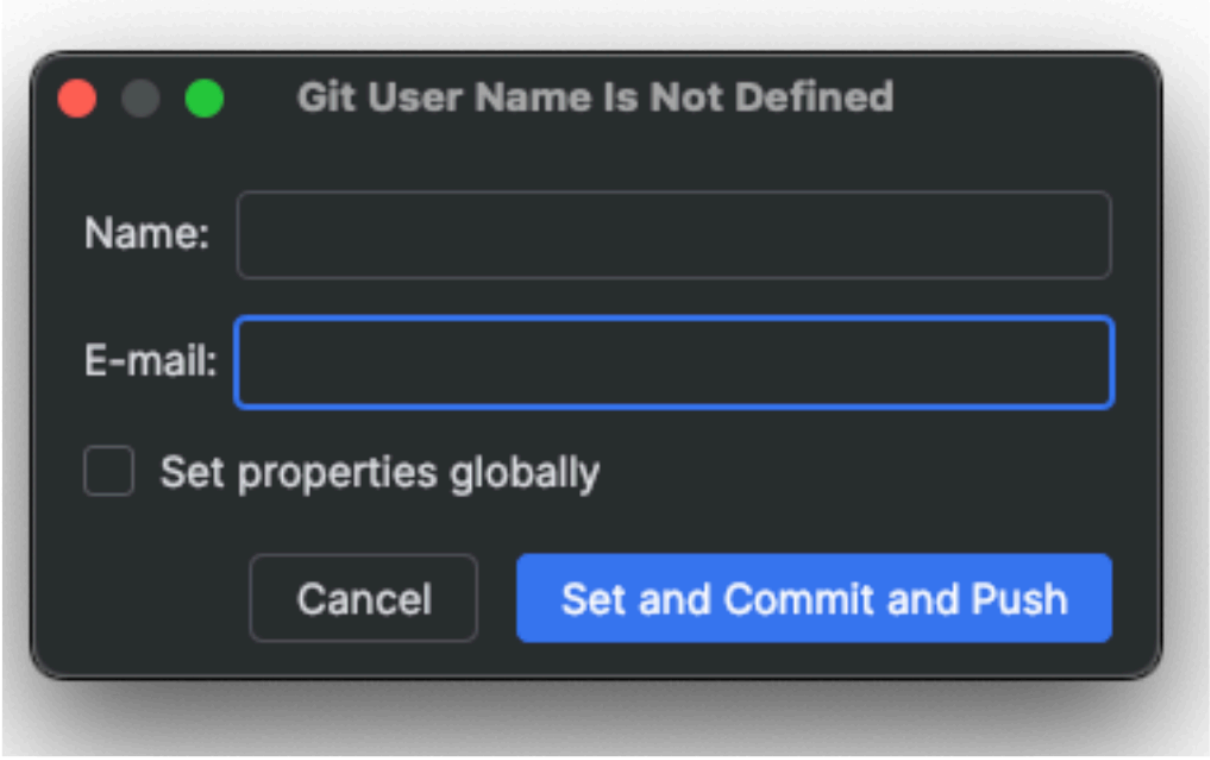


Рисунок 28 - Вход в аккаунт

Такое окно появляется при попытке сделать первый коммит в среде, если в Git ещё не настроены значения user.name и user.email (то есть Git "не знает", от чьего имени создавать коммиты). В полях Name и E-mail указываются имя автора и адрес электронной почты, которые будут записываться в метаданные каждого коммита и затем отображаться в истории репозитория.

Флажок Set properties globally означает, что введённые user.name и user.email будут сохранены как глобальные настройки Git на компьютере и автоматически применятся ко всем другим локальным репозиториям, а не только к текущему проекту.

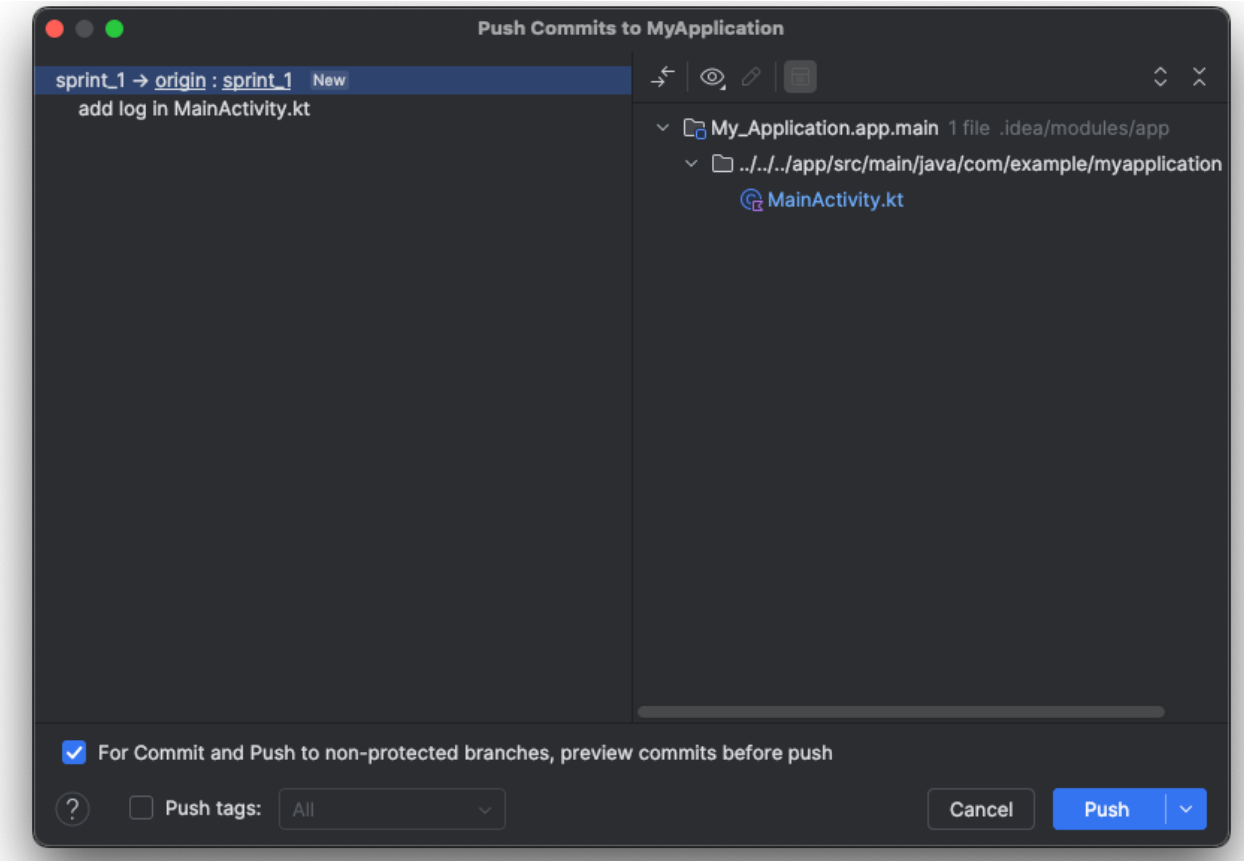


Рисунок 29 - Отправка изменений

В этом окне выполняется финальный шаг отправки изменений в удалённый репозиторий, то есть push. В верхней части показано, какая локальная ветка будет отправляться и к какому удалённому адресу она привязана, например sprint_1 - origin:sprint_1, где origin - имя удалённого репозитория, а sprint_1 - имя ветки на сервере. В левой части отображается список коммитов, которые будут отправлены, поэтому здесь можно увидеть, какие именно локальные изменения ещё не попали в удалённый репозиторий. В правой части показано дерево файлов, затронутых этими коммитами, то есть какие файлы изменялись и будут обновлены на стороне удалённого репозитория после выполнения push.

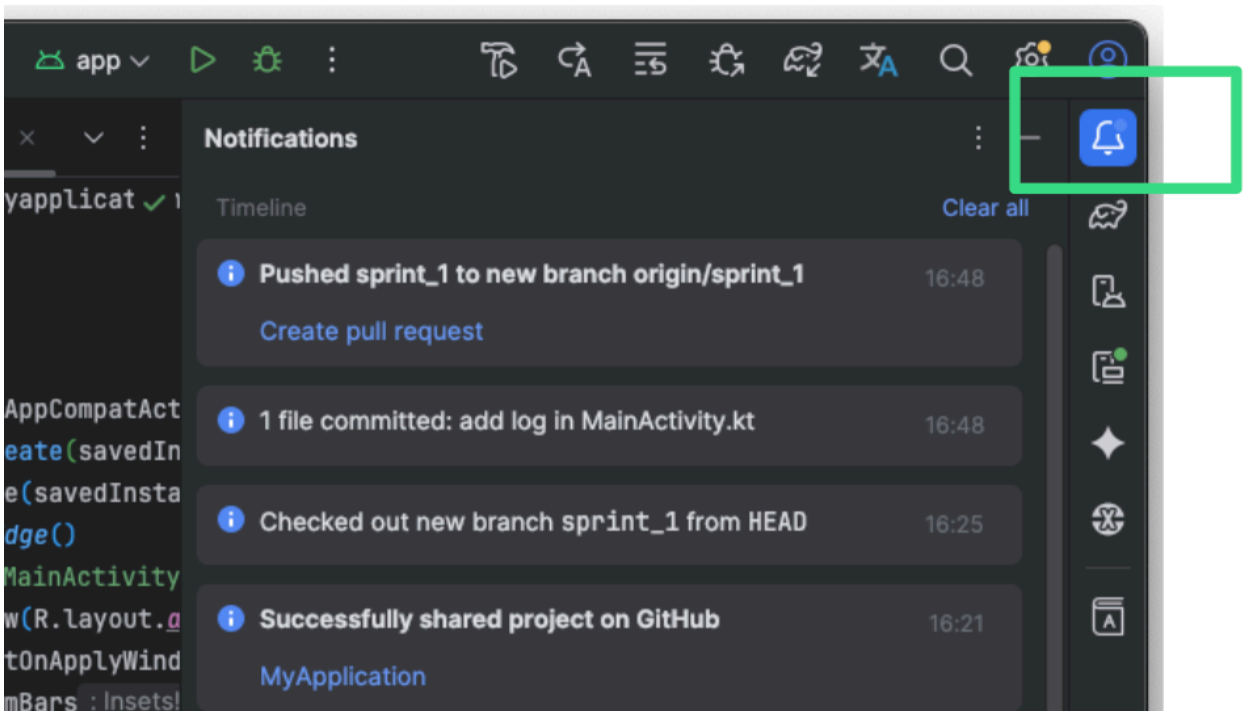


Рисунок 30 - Уведомления

В правой части Android Studio находится значок уведомлений в виде колокольчика. При нажатии на него открывается раздел Notifications, где сохраняются сообщения среды о выполненных действиях и результатах операций, например, о создании проекта, переключении ветки, сборке или отправке изменений в репозиторий.

На рисунке 30 видно, что последнее уведомление связано с отправкой изменений: сообщение вида Pushed sprint_1 to new branch origin/sprint_1 означает, что локальная ветка sprint_1 была успешно отправлена в удалённый репозиторий origin и на сервере создана ветка origin/sprint_1. Это подтверждает, что push выполнен корректно и изменения доступны в удалённом репозитории.

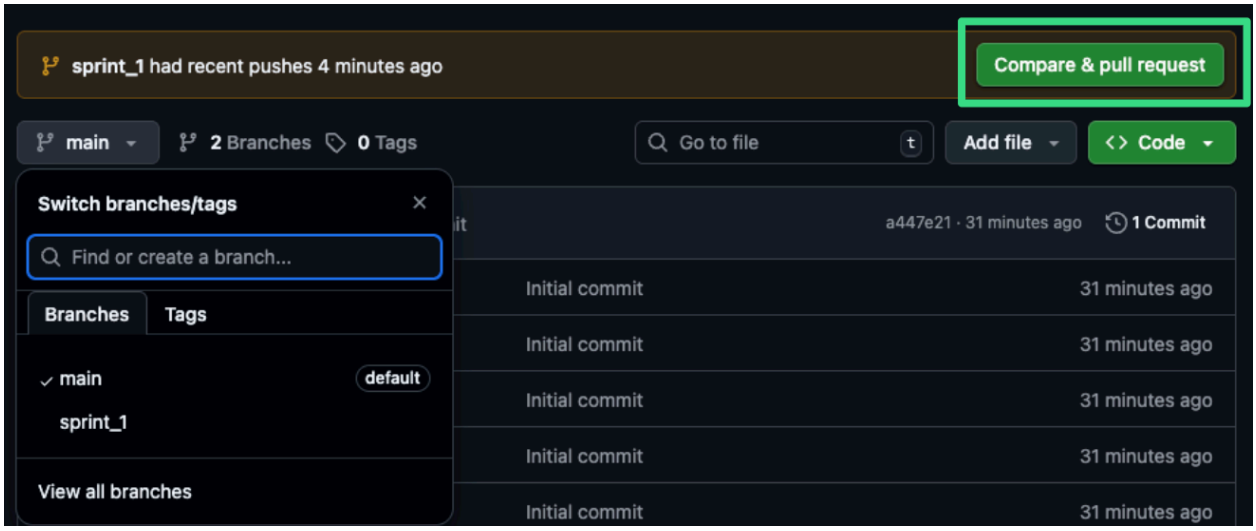


Рисунок 31 - Вкладка проекта

После перехода в репозиторий на GitHub интерфейс показывает уведомление о недавней отправке изменений в новую ветку, что подтверждает успешный push. В левом верхнем блоке можно открыть список веток через выпадающее меню переключения веток

и тегов, где помимо основной ветки main отображается созданная ветка sprint_1. Кнопка Compare & pull request запускает создание pull request.

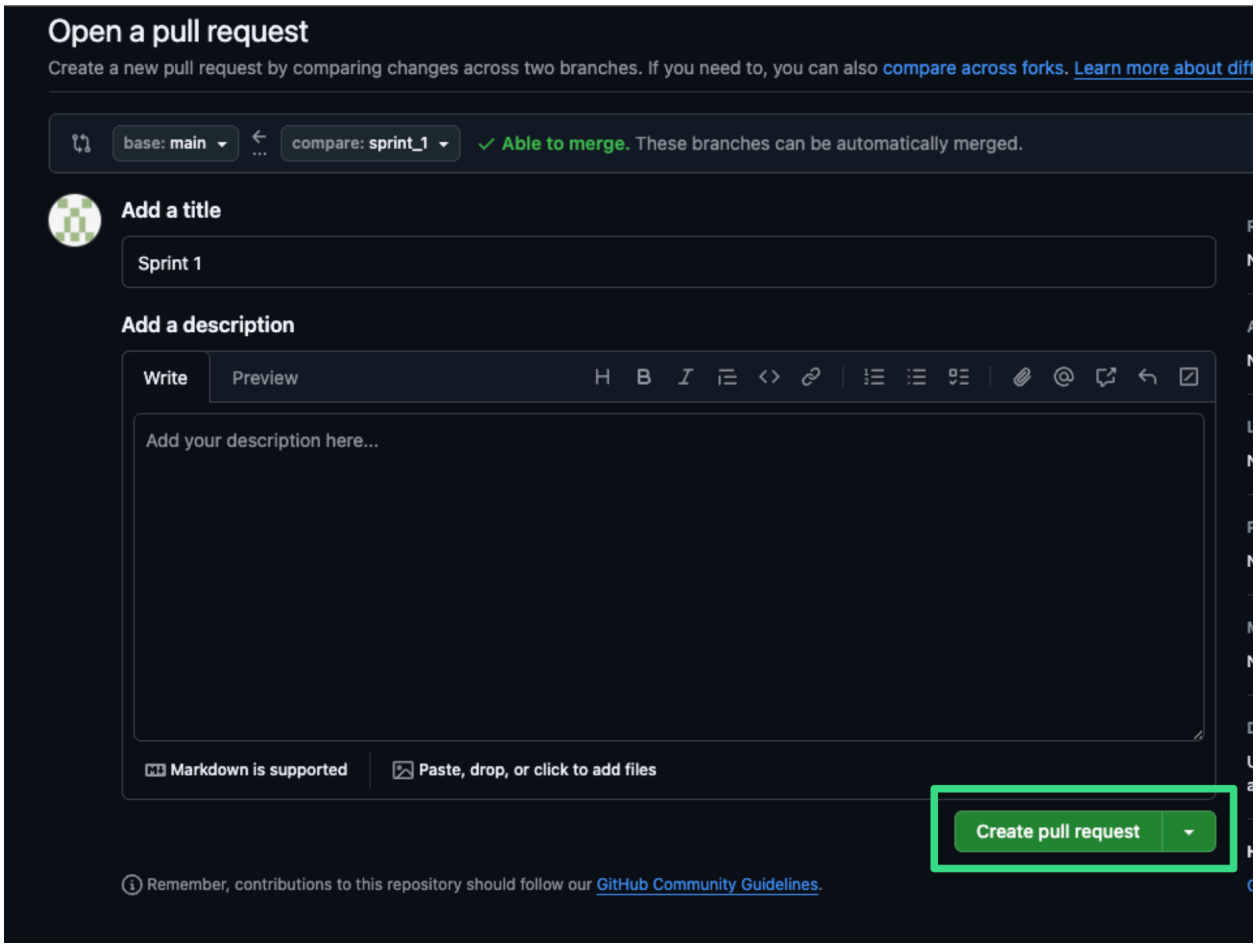


Рисунок 32 - Создание pull request

Здесь открыта форма создания pull request. В верхней части выбираются ветки для слияния, то есть видно, из какой ветки будут взяты изменения и в какую целевую ветку они предлагаются к добавлению. Ниже заполняются основные поля pull request: заголовок (кратко описывает суть изменений) и описание (детали, что именно сделано и почему).

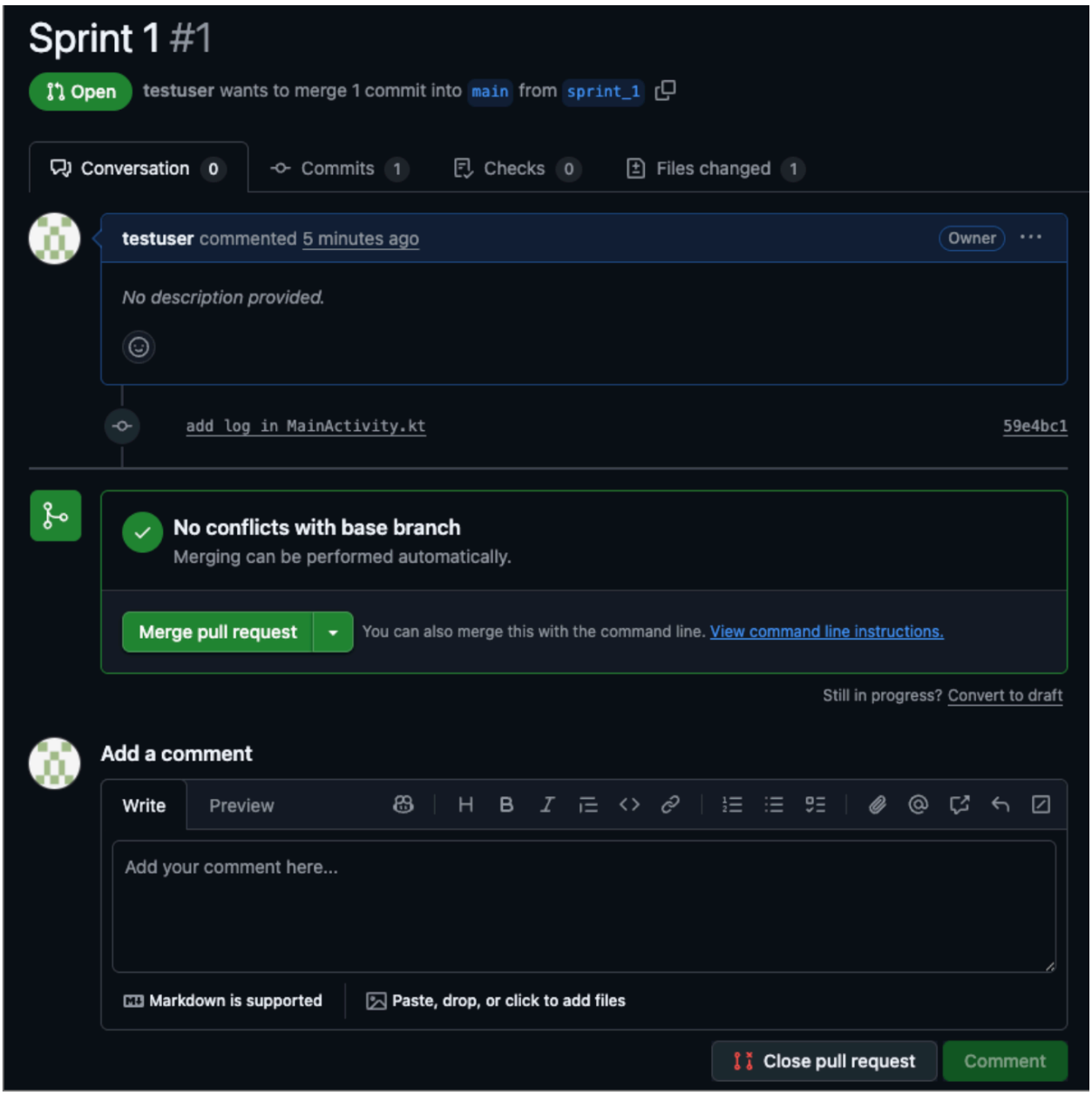


Рисунок 33 - Обсуждение pull request

Это экран уже созданного pull request, где собрана вся информация о предлагаемых изменениях и их проверках. Во вкладке Conversation ведётся обсуждение - здесь оставляют общие комментарии по задаче, уточнения и замечания по коду. Во вкладке Commits отображается список и количество коммитов, которые входят в pull request. Вкладка Checks показывает результаты автоматических проверок, если они настроены в репозитории, например, сборка проекта, запуск тестов, статический анализ. Вкладка Files changed содержит перечень изменённых файлов и построчное сравнение правок, чтобы ревьюеру было удобно просмотреть, что именно изменилось.

Ниже выводится статус возможности слияния. Если GitHub показывает отсутствие конфликтов, значит изменения можно объединить автоматически. Если конфликт есть, Git сообщает, что изменения из разных веток затрагивают одни и те же фрагменты файлов и не могут быть объединены без ручного разрешения (git-конфликт - ситуация, когда системе нужно участие разработчика, чтобы выбрать итоговый вариант). После ревью и успешных проверок используется кнопка Merge pull request, которая выполняет слияние изменений в целевую ветку.

Supabase представляет собой облачную платформу для построения серверной части приложений, ориентированную на упрощение работы с хранением данных и сетевым взаимодействием. В архитектурном смысле Supabase выполняет роль готового бэкенда, предоставляя разработчику набор сервисов, необходимых для работы клиентских приложений, без необходимости самостоятельной настройки серверной инфраструктуры. Платформа основана на реляционной базе данных PostgreSQL и предоставляет к ней безопасный доступ через API, что позволяет использовать стандартные модели данных и привычные принципы работы с таблицами, связями и запросами.

Использование облачного бэкенда, такого как Supabase, позволяет вынести хранение и обработку данных за пределы мобильного приложения, обеспечив централизованное управление информацией, синхронизацию между устройствами и сохранность данных при смене клиента. Для Android-приложений Supabase может использоваться для хранения пользовательских данных, результатов работы, справочной информации и других сущностей предметной области. Такой подход способствует разделению ответственности между клиентской и серверной частью приложения, повышает масштабируемость решения и упрощает дальнейшее сопровождение и развитие программного продукта.

Чтобы начать взаимодействовать с Supabase, создать таблицу для хранения данных и использовать встроенную систему регистрации и авторизации, можно выполнить ряд последовательных шагов:

1. Чтобы создать проект в Supabase, зарегистрируйтесь на платформе и в панели управления нажмите New Project. Укажите имя проекта, выберите регион и задайте пароль для базы данных PostgreSQL - Supabase автоматически развернёт для вас полноценную реляционную базу данных.
2. После создания проекта откройте раздел Table Editor - визуальный интерфейс для управления таблицами. Здесь можно добавлять, удалять и настраивать таблицы и их поля.
3. На вкладке Authentication настройте методы входа пользователей - например, по email и паролю, либо через OAuth-провайдеров. Supabase Auth представляет собой готовую систему аутентификации и авторизации, генерирующую JWT-токены для сессий.
4. Supabase автоматически создаёт схему auth с таблицей auth.users, в которой хранятся email, хеш пароля и другие метаданные. Через API можно регистрировать пользователей и выполнять вход в систему с помощью методов SDK или HTTP-запросов.
5. После создания таблиц можно использовать SQL-редактор или автоматически сгенерированные REST/GraphQL API для выполнения запросов: вставка новых записей, выборка, обновление и удаление.

Supabase Storage предоставляет бакеты (логические контейнеры для файлов), которые используются для хранения медиа-контента: изображений, видео, документов и других ресурсов, аналогично объектному хранилищу S3. Бакеты позволяют организовать файлы в структуре, определять уровни доступа (публичный или приватный) и настраивать правила безопасности, которые управляют возможностью загружать, просматривать или изменять файлы.

Для того чтобы начать работу с изображениями в Supabase Storage нужно создать бакет в панели Supabase в разделе Storage, нажав New Bucket, задав имя (например, `images`) и выбрав режим доступа: публичный или приватный. По умолчанию бакеты приватные - доступ к файлам возможен только через авторизованные запросы или подписанные URL с ограниченным сроком действия. Публичный режим позволяет обращаться к файлам напрямую по URL формата `https://[project_id].supabase.co/storage/v1/object/public/[bucket]/[filename]`. После создания бакета необходимо загрузить изображение через SDK в нужный путь внутри бакета, указав имя файла и корректный MIME-тип. Для получения ссылки используйте getPublicUrl, если бакет публичный; если бакет приватный, создавайте

временную ссылку через createSignedUrl. Полученный URL используйте в UI-компоненте приложения (например, ImageView) для загрузки и отображения изображения. Для приватных бакетов ссылку обычно нужно обновлять по мере истечения срока действия.

Чтобы начать взаимодействие Android-приложения с Supabase, требуется выполнить последовательность шагов:

1. Добавление зависимостей в проект;
2. Настройка разрешений;
3. Инициализация клиента Supabase;
4. Выполнение сетевых запросов (например, чтение/запись данных);
5. Работа с аутентификацией и другими модулями.

В app/build.gradle (Kotlin DSL: build.gradle.kts) необходимо подключить зависимости Supabase и Ktor-клиент для Android. Для управления версиями целесообразно использовать BOM-зависимость Supabase. Значения версий должны быть заменены на актуальные, которые можно посмотреть на странице релизов Supabase-KT на GitHub.

Пример зависимостей:

```
dependencies {
    // BOM для унификации версий модулей Supabase
    implementation(platform("io.github.jan-
tennert.supabase:bom:3.2.6")) // пример версии
    // Клиент для работы с REST API (PostgREST)
    implementation("io.github.jan-tennert.supabase:postgrest-
kt")
    // Модуль для аутентификации
    implementation("io.github.jan-tennert.supabase:auth-kt")
    // Обязательный Ktor-движок для Android
    implementation("io.ktor:ktor-client-android:3.2.1")//
замените на актуальную
    // (опционально) другие модули: storage-kt, realtime-kt и т.п.
}
```

Подключение BOM-платформы позволяет избежать несоответствия версий между модулями.

Для сетевого взаимодействия требуется разрешение доступа в интернет. В файл AndroidManifest.xml (вне тега <application>) добавляется:

<uses-permission android:name="android.permission.INTERNET" />

В коде Android-приложения создаётся клиент Supabase, указывая URL проекта и ключ anon (получаем из панели Supabase → Project Settings → API Keys).

Пример кода на Kotlin:

```
import io.github.jan.supabase.createSupabaseClient
import io.github.jan.supabase.postgrest.Postgrest

val supabase = createSupabaseClient(
    supabaseUrl = "https://YOUR_PROJECT_REF.supabase.co",
    supabaseKey = "YOUR_ANON_PUBLIC_KEY"
) {
    install(Postgrest) // установка PostgREST-модуля
    // при необходимости добавить: install(Auth), install(Storage)
и др.
}
```

Это создаёт глобальный объект клиента, через который осуществляется доступ к базе данных и другим сервисам Supabase.

Для запросов к базе Supabase используется модуль Postgrest, обеспечивающий простую синтаксическую оболочку над REST-API.

Предположим, в Supabase есть таблица products. Запрос на чтение:

```
import kotlinx.coroutines.*

// Запрос выполняется в корутине
CoroutineScope(Dispatchers.IO).launch {
    val response = supabase
        .postgrest["products"]
        .select()
        .decodeList<ProductDto>()

    if (response.isSuccess) {
        val products = response.data
        // обработка списка products
    } else {
        val error = response.error
        // обработка ошибки
    }
}

// Пример DTO(Data Transfer Object - класса для данных
@Serializable
data class ProductDto(
    val id: Int,
    val name: String,
    val price: Double
)
```

Для запуска регистрации и входа используются методы модуля auth-kt.

Пример регистрации по email и паролю:

```
CoroutineScope(Dispatchers.IO).launch {
    val result = supabase.auth.signUpWithEmail(
        email = "user@example.com",
        password = "StrongPassword123"
    )
    // обработка результата регистрации
}

// Вход пользователя
CoroutineScope(Dispatchers.IO).launch {
    val result = supabase.auth.signInWithPassword(
        email = "user@example.com",
        password = "StrongPassword123"
    )
    // если успешно, в result будет информация о сессии
}
```

В Android приложение строится из набора компонентов, которые система может запускать и останавливать независимо друг от друга, формируя пользовательские сценарии и фоновые процессы. Activity отвечает за экран и взаимодействие с пользователем, то есть за отображение интерфейса и обработку действий на конкретном «экране» приложения. Service предназначен для выполнения работы без прямого пользовательского интерфейса, например, для длительных операций или задач, которые должны продолжаться вне активного экрана. BroadcastReceiver используется для реакции на широковещательные сообщения (broadcast) от системы или других приложений, которые сигнализируют о наступлении события, например, о завершении загрузки или изменении состояния устройства. ContentProvider предоставляет стандартизированный интерфейс доступа к данным приложения (внутри приложения или для других приложений при необходимости), позволяя безопасно организовать обмен данными и запросы к ним. Все перечисленные компоненты, которые приложение планирует использовать, должны быть объявлены в AndroidManifest.xml, иначе система не сможет их запускать. Связь между компонентами и запуск сценариев обычно выполняются через Intent, а правила того, какие компоненты могут принимать определённые запросы, задаются intent-filter в манифесте.

Intent в Android - это объект сообщений, который описывает операцию, которую требуется выполнить, и используется для связывания компонентов приложения между собой (а также, при необходимости, для взаимодействия с компонентами других приложений). На практике Intent применяется для запуска Activity через startActivity(...), запуска или связывания с Service, а также для рассылки сообщений BroadcastReceiver. При этом сам объект Intent переносит сведения, необходимые системе для выбора получателя, и данные, необходимые получателю для выполнения действия (например, action, data/Uri, category, дополнительные параметры extras).

Различают явные (explicit) и неявные (implicit) Intent. Явный Intent содержит точное имя целевого компонента (класс Activity/Service/Receiver), поэтому используется в основном для навигации и запуска конкретных экранов или служб внутри собственного приложения, когда получатель заранее известен. Неявный Intent не указывает конкретный компонент, а задаёт «что нужно сделать» через action и, при необходимости, data (например, открыть web-страницу), после чего система подбирает подходящий компонент на устройстве на основе intent-filter, объявленных в AndroidManifest.xml у потенциальных получателей.

Fragment - компонент пользовательского интерфейса, представляющий самодостаточную часть экрана со своей разметкой и логикой, которая размещается внутри Activity и управляется через FragmentManager. Фрагменты используют, когда требуется собирать экран из переиспользуемых модулей, строить адаптивные интерфейсы под разные размеры экранов (например, одновременный показ списка и деталей на планшете), а также организовывать навигацию и смену содержимого внутри одной Activity (в том числе при использовании Navigation Component). По сравнению с отдельными Activity фрагменты дают более гибкую композицию интерфейса: их можно динамически добавлять, заменять и комбинировать в рамках одного окна приложения, сохраняя общий контейнер и общие зависимости, что упрощает переиспользование UI и поддержку согласованной навигации. При этом Activity остается контейнером и точкой входа, а Fragment отвечает за локальную часть UI и связанный с ней участок жизненного цикла.

При вёрстке на XML (eXtensible Markup Language) в Android важно понимать, что интерфейс описывается как иерархия объектов View и ViewGroup: View представляет отдельный отображаемый/интерактивный элемент (например, TextView, Button), а ViewGroup - контейнер, который задаёт правила размещения дочерних элементов и формирует структуру экрана. XML-файлы разметки являются ресурсами и располагаются в res/layout, после чего используются системой и кодом как R.layout.*. При описании элементов в XML ключевыми являются параметры размера и расположения (например,

layout_width, layout_height и атрибуты конкретного контейнера), потому что именно они определяют результат проходов измерения и раскладки и, как следствие, визуальное поведение экрана.

Отдельно следует учитывать адаптацию под разные устройства: размеры в разметке задают преимущественно в dp (плотностно-независимых единицах), чтобы интерфейс корректно масштабировался на экранах с разной плотностью пикселей, а альтернативные XML-разметки под разные размеры/форм-факторы оформляют через квалификаторы ресурсов (например, разные папки layout-*), чтобы система автоматически подбирала подходящий вариант. Наконец, при работе с XML важно следить за переиспользованием и производительностью: крупные повторяющиеся фрагменты интерфейса выносят в отдельные разметки и подключают через <include>, а для уменьшения глубины дерева можно применять <merge>. Рекомендуется избегать избыточной вложенности контейнеров, поскольку сложная иерархия View способна ухудшать производительность. Для обращения к элементам из Kotlin на практике обычно используют View Binding, который генерирует типобезопасные ссылки на View из XML и во многих случаях заменяет findViewById. findViewById используют, чтобы получить ссылку на нужный View по его id после. Метод простой и универсальный, но не даёт типобезопасности и требует аккуратности с null и приведением типов.

LinearLayout - это контейнер ViewGroup, который размещает дочерние элементы последовательно в одном направлении, по вертикали или по горизонтали, порядок определяется их расположением в XML. Направление задаётся через android:orientation, а размеры самого контейнера и каждого дочернего элемента задаются стандартными android:layout_width и android:layout_height. Для выравнивания часто используются android:gravity у контейнера и android:layout_gravity у конкретного дочернего View, для внутренних отступов применяется android:padding, а внешние отступы элементов задаются через android:layout_margin. В вертикальном LinearLayout доступно распределение свободного пространства через android:layout_weight у дочерних элементов и (при необходимости) android:weightSum у контейнера. LinearLayout удобен для простых «линейных» экранов, но при глубокой вложенности нескольких контейнеров (например, LinearLayout внутри LinearLayout внутри LinearLayout) дерево View разрастается, из-за чего усложняются измерение и раскладка, а производительность интерфейса может снижаться.

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="LinearLayout" />
</LinearLayout>
```

ConstraintLayout - это контейнер, в котором элементы размещаются не «линией», а через связи (constraints) между сторонами View и сторонами родителя или других View. Вместо вложенных контейнеров позиционирование описывается ограничениями, например app:layout_constraintTop_toTopOf, app:layout_constraintStart_toStartOf, app:layout_constraintEnd_toEndOf, app:layout_constraintBottom_toBottomOf, благодаря чему разметка часто получается более «плоской» и легче масштабируется под разные размеры экранов. По сравнению с LinearLayout, ConstraintLayout обычно удобнее на сложных экранах, где требуется гибко выравнивать элементы относительно друг друга, выдерживать

интервалы, центрировать блоки и строить адаптивные расположения без лишней вложенности. Для точной настройки используются параметры смещения и поведения, например app:layout_constraintHorizontal_bias и app:layout_constraintVertical_bias, а также подход «0dp» для ширины или высоты (match constraints), когда размер элемента определяется заданными constraints.

```
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/titleText"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="ConstraintLayout"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        android:layout_marginTop="16dp" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

TextView и EditText относятся к базовым элементам текстового интерфейса. TextView используется для отображения статического текста и обычно настраивается через android:text, android:textSize, android:textColor, android:textStyle, android:gravity, а также стандартные параметры размеров android:layout_width и android:layout_height. EditText предназначен для ввода данных пользователем и дополняется атрибутами android:hint, android:inputType (например, textEmailAddress, phone, numberPassword), android:maxLines, android:imeOptions, android:minLines, android:text, при необходимости android:enabled и android:focusable. Для корректной работы с текстовыми полями важно задавать понятные подсказки hint и выбирать подходящий inputType, чтобы система показывала пользователю удобную клавиатуру.

ImageView, Button и CheckBox применяются для отображения графики и выполнения действий. ImageView показывает изображение из ресурсов через android:src или android:background, а масштабирование и обрезка управляются android:scaleType, при этом для доступности желательно указывать android:contentDescription. Button используется для запуска действия по нажатию и чаще всего настраивается через android:text, android:enabled, android:background, а обработка нажатия задаётся в коде через setOnClickListener. CheckBox предназначен для выбора состояния да или нет и использует android:checked для начального значения, android:text для подписи, а изменение состояния обрабатывается слушателем, например setOnCheckedChangeListener.

RadioButton и RadioGroup применяются, когда пользователь должен выбрать ровно один вариант из нескольких. RadioButton представляет отдельный вариант, а RadioGroup объединяет несколько RadioButton и автоматически следит, чтобы был выбран только один элемент. Для RadioGroup важны android:orientation и, при необходимости, android:checkedButton, чтобы задать выбранный вариант по умолчанию. Состояние выбора можно получать и изменять через идентификаторы элементов, поэтому атрибут android:id у RadioButton в группе является практическим требованием.

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
```

```
android:layout_height="wrap_content"
android:orientation="vertical"
android:padding="16dp">

<TextView
    android:id="@+id/titleText"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Профиль пользователя"
    android:textSize="18sp"
    android:textStyle="bold" />

<EditText
    android:id="@+id/emailInput"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Электронная почта"
    android:inputType="textEmailAddress"
    android:layout_marginTop="12dp" />

<ImageView
    android:id="@+id/avatarImage"
    android:layout_width="72dp"
    android:layout_height="72dp"
    android:layout_marginTop="12dp"
    android:src="@mipmap/ic_launcher"
    android:scaleType="centerCrop"
    android:contentDescription="Аватар" />

<CheckBox
    android:id="@+id/agreeCheck"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="12dp"
    android:text="Согласен с условиями"
    android:checked="false" />

<RadioGroup
    android:id="@+id/themeGroup"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="12dp"
    android:orientation="vertical"
    android:checkedButton="@+id/themeLight">

    <RadioButton
        android:id="@+id/themeLight"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Светлая тема" />

    <RadioButton
        android:id="@+id/themeDark"
```

```
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Тёмная тема" />
    </RadioGroup>

    <Button
        android:id="@+id/saveButton"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:text="Сохранить" />
</LinearLayout>
```

Jetpack Compose - современный рекомендованный Google набор инструментов для построения нативного UI в Android на Kotlin, основанный на декларативном подходе: разработчик описывает, каким должен быть интерфейс для текущего состояния данных, вместо пошагового изменения дерева View. UI в Compose строится из функций @Composable, которые формируют иерархию элементов, а при изменении состояния фреймворк автоматически выполняет перерасчёт и обновляет отображение (recomposition). Compose также предоставляет готовые компоненты и темы (включая поддержку Material 3) и инструменты разработки вроде предпросмотра интерфейса.

Clean Architecture - это подход к проектированию приложения, в котором код организуется вокруг устойчивых «правил предметной области» и сценариев использования, а детали реализации (UI-фреймворк, база данных, сеть, конкретные библиотеки) выносятся на периферию. Ключевая идея формулируется через правило направленности зависимостей: зависимости исходного кода должны «смотреть внутрь», то есть высокоуровневые правила и бизнес-логика не должны зависеть от внешних механизмов и инфраструктуры. Это позволяет снижать связанность компонентов, упрощать тестирование и замену технологических деталей без переписывания центральной логики приложения. В Android-разработке Clean Architecture обычно соотносят со слоистой организацией кода, близкой к рекомендациям Google по архитектуре приложений: выделяется слой пользовательского интерфейса (UI), слой данных (data) и при необходимости - промежуточный доменный слой (domain) между ними. Такой подход поддерживает разделение ответственности между частями приложения и помогает выстраивать предсказуемый поток данных, в том числе через принципы однонаправленного data flow, когда UI отправляет события «вниз» (в обработку), а состояние возвращается «вверх» (для отображения).

UI layer (слой пользовательского интерфейса) отвечает за отображение состояния приложения и обработку пользовательских действий, выступая «переходником» между изменениями данных и тем, как эти изменения представлены на экране. В терминах официального руководства Android этот слой можно понимать как конвейер, который преобразует изменения данных приложения в форму, пригодную для отображения, и затем отображает их пользователю. На практике в этот слой относят экраны и их состояние, логику реагирования на события (нажатия, ввод, навигация), а также механизмы, которые обеспечивают предсказуемое обновление UI при изменении состояния.

Domain layer (доменный слой, опциональный) располагается между UI и data и используется не во всех приложениях: документация прямо указывает, что доменный слой является необязательным и имеет смысл, когда требуется инкапсулировать сложную бизнес-логику или переиспользуемые сценарии, которые нужны в нескольких местах (например, в нескольких ViewModel). В общем понимании доменный слой помогает отделить правила и сценарии работы приложения от деталей представления и хранения данных, выступая «центром» прикладной логики, который можно развивать и тестировать независимо от конкретных экранов и источников данных.

Data layer (слой данных) отвечает за получение, хранение и предоставление «данных приложения», а также за связанную с ними бизнес-логику, определяющую, как эти данные создаются, сохраняются и изменяются. Этот слой обычно включает абстракции доступа к данным (например, репозитории) и конкретные источники данных (сеть, локальная база, файловое хранилище и т.п.), а также преобразование форматов и обработку ошибок, чтобы верхние слои работали с согласованной моделью данных.

MVVM (Model–View–ViewModel) в презентационном слое Android применяют как паттерн организации UI-кода, чтобы отделить отображение (View) от логики представления и состояния экрана (ViewModel) и тем самым снизить связанность, повысить тестируемость и упростить сопровождение. В общем определении MVVM направлен на разделение пользовательского интерфейса и логики представления, чтобы UI оставался «тонким», а основная логика работы экрана концентрировалась вне Activity/Fragment/Composable.

– Необходимо разработать приложение для мобильного устройства, удовлетворяющее следующим требованиям:

– Минимальная версия ОС, поддерживаемая приложением, должна быть: Android - 12.0

– В качестве бэкенда будет использован Supabase (https://supabase.com/).

– Данные по пользователям, автомобилям и бронированиям должны находятся в таблицах Supabase.

– В работе необходимо использовать систему контроля версий Git. Репозиторий может быть открытым или закрытым.

– Необходимо следовать предложенному дизайну. Реализация дополнительных экранов и логики, может быть, при желании осуществлена в соответствии с требованиями проекта и архитектурными решениями.

– Необходимо корректно обрабатывать запросы к серверу. В случае получения ошибки от сервера необходимо отобразить соответствующий текст ошибки в диалоговом окне.

– В случае отсутствия доступа в интернет на устройстве, необходимо отображать экран No Connection (Название экрана из Figma).

– Необходимо отображать индикацию выполнения запроса с сервером - loader, progress bar.

– Приложение должно быть разработано с учетом устойчивости к сбоям и предусматривать механизмы обработки исключительных ситуаций, гарантирующие его непрерывную работу при любых обстоятельствах. В случае возникновения непредвиденных ошибок или сбоев, приложение должно сохранять свою функциональность и обеспечивать корректное выполнение всех критически важных процессов, минимизируя влияние этих ситуаций на пользовательский опыт.

– При разработке проекта приложения вам необходимо использовать чистую архитектуру, в которой будут разделены слои бизнес-логики, представлений и данными.

– В слое представления необходимо явно выделить вьюмодель, все взаимодействие с бизнес-логикой должно осуществляться из вьюмодели.

– Файлы проекта распределены по папкам в соответствии с архитектурой.

Вся верстка должна быть адаптивной (следует учитывать разные размеры экранов). Приложение должно корректно отображаться как на устройствах с маленькими экранами смартфонов, так и на устройствах с большими экранами, обеспечивая оптимальный пользовательский интерфейс и удобство взаимодействия независимо от размера экрана.

Необходимо:

– Избегайте появления большого пустого пространства;

– Следите за отсутствием искажения элементов;

– Все элементы должны полностью находится в границах и на месте, указанном в макете;

– Учитывайте расстояние между элементами;

– Используйте шрифты согласно макету.

– Дизайн предложен в Figma: https://www.figma.com/design/bIbRYtg9CLZmGgTea2nh51/DriveNextV2?node-id=3908-2710&p=f&t=hUtUF5k23A8x7WuW-0

ЛАБОРАТОРНАЯ РАБОТА №1

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

SplashScreen – это начальный экран приложения, отображаемый пользователю в момент запуска до загрузки основного интерфейса. Его основная задача заключается в обеспечении визуальной целостности процесса старта приложения, а также в сокрытии задержек, связанных с инициализацией ресурсов, проверкой состояния приложения или подготовкой данных. В современных версиях Android рекомендуется использовать системный API SplashScreen, который обеспечивает единообразное поведение стартового экрана на разных устройствах и снижает вероятность визуальных артефактов. Базовая реализация SplashScreen включает задание темы приложения с параметрами стартового экрана, указание фонового цвета или изображения и последующий переход к основной Activity после завершения инициализации.

Шаги для создания SplashScreen включают:

1. В файле конфигурации зависимостей проекта необходимо добавить библиотеку AndroidX SplashScreen, которая обеспечивает поддержку единого API стартового экрана для разных версий Android.
2. В ресурсах приложения требуется определить отдельную тему, в которой задаются параметры стартового экрана: цвет фона, иконка или изображение, а также тема, применяемая после завершения SplashScreen.
3. Созданную тему SplashScreen необходимо указать для стартовой Activity приложения в манифесте, чтобы система использовала её при запуске приложения.
4. В методе onCreate стартовой Activity следует вызвать инициализацию SplashScreen до установки основного пользовательского интерфейса, чтобы корректно связать его с жизненным циклом Activity.

Для проверки наличия сети нужно в файле AndroidManifest.xml включить разрешения, которые позволяют приложению получать информацию о сети и выполнять сетевые запросы:

```
 <uses-permission android:name="android.permission.INTERNET"/>
     <uses-permission
android:name="android.permission.ACCESS_NETWORK_STATE"/>
```

Разрешение INTERNET требуется для выполнения сетевых запросов, а ACCESS_NETWORK_STATE - для получения статуса сетевого подключения. В коде приложения необходимо получить доступ к системному сервису ConnectivityManager, который предоставляет информацию о текущем состоянии сетевых подключений:

```
val connectivityManager = context.getSystemService(Context.CONNECTIVITY_SERVICE) as ConnectivityManager
```

ConnectivityManager является основным классом, через который осуществляется проверка состояния сети.

Получение информации о текущей сети

С помощью ConnectivityManager можно получить объект, представляющий активную сеть устройства. На современных API рекомендуется использовать методы getActiveNetwork() и getNetworkCapabilities(). Они позволяют определить, доступна ли сеть и какие у нее возможности (например, доступ к интернету):

```
val network = connectivityManager.activeNetwork
     val capabilities = connectivityManager.getNetworkCapabilities(network)
```

```
    val hasInternet = capabilities?.hasCapability(NetworkCapabilities.NET_CAPABILITY_INTERNET) == true
```

ViewPager2 – это компонент пользовательского интерфейса Android, предназначенный для организации горизонтального или вертикального перелистывания страниц, представленных `View` или `Fragment`. Он применяется для реализации онбординга, пошаговых инструкций, галерей и других сценариев, где требуется последовательный просмотр контента. ViewPager2 является современной заменой устаревшего `ViewPager` (androidx.viewpager.widget.ViewPager): новый компонент имеет более предсказуемую работу с жизненным циклом страниц на основе `FragmentStateAdapter`, а также предоставляет более удобные механизмы управления состоянием и обработкой смены страницы. Для реализации онбординга в рамках данной лабораторной работы рекомендуется использовать страницы на основе фрагментов. Минимально требуются следующие классы и элементы:

- `ViewPager2` - компонент, размещаемый в XML-разметке экрана.
- `Fragment` - отдельная «страница» (экран) онбординга, как правило, с собственным макетом и набором UI-элементов.
- `FragmentStateAdapter` - адаптер, который связывает `ViewPager2` со списком фрагментов и отвечает за предоставление нужной страницы по индексу.

Базовая логика работы строится так:

1. В разметке Activity/Fragment размещается `ViewPager2`.
2. В коде создаётся список страниц (набор `Fragment`).
3. Реализуется адаптер на основе `FragmentStateAdapter`, в котором переопределяются два метода:
   - `getItemCount()` - возвращает количество страниц;
   - `createFragment(position: Int)` - возвращает фрагмент для указанной позиции.
4. Адаптер назначается в `viewPager2.adapter`.
5. При необходимости добавляется обработка смены страницы через `registerOnPageChangeCallback(...)`, чтобы, например, обновлять состояние кнопок «Далее/Пропустить» или менять текст на последнем шаге.
6. Для программного переключения страниц используется `setCurrentItem(position, smoothScroll)`, например, при нажатии кнопки «Далее».

Подключение ViewPager2 выполняется через зависимость AndroidX. Актуальную версию следует смотреть в официальных заметках о релизах AndroidX для ViewPager2 (раздел Release notes) или в каталоге артефактов Google Maven. После обновления зависимости требуется синхронизация Gradle.

Пример разметки в XML:

```
<androidx.viewpager2.widget.ViewPager2
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/viewPager"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
```

```
android:orientation="horizontal" />
```

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Экран «Экран загрузки»



Рисунок 34 - Экран загрузки

Общее описание

Реализуйте экран для входа в приложении согласно макету. Данный экран отображается каждый раз при входе в приложение.

Функциональные требования

1. Отображение названия, логотипа и слогана приложения:
    - Название отображается сверху по левому краю экрана.
    - Слоган «Поможем найти твою следующую поездку» отображается под логотипом.
    - Логотип приложения отображается в центре экрана.
2. Время отображения:
    - Экран загрузки отображается на экране в течение 2-3 секунд или до завершения инициализации приложения.

Переходы

- После завершения загрузки автоматически, без дополнительного действия со стороны пользователя, происходит переход на экран входа в приложение, если пользователь открывает его первый раз, либо access токен недействителен. Либо на главный экран приложения.

Экран «Нет подключения к интернету»



Рисунок 35 - Нет подключения к интернету

Общее описание

Реализуйте экран для отображения отсутствия интернета в приложении согласно макету.

Функциональные требования

1. Сообщение об ошибке:
    - Текст «Нет подключения к интернету» отображается в центре экрана.
    - Под основным сообщением отображается текст «Проверьте подключение и повторите попытку», информирующий пользователя о необходимости проверки подключения.
    - Над текстом отображается иконка, символизирующая отсутствие соединения.
2. Кнопки:
    - «Повторить попытку»

        Кнопка становится активной сразу после отображения экрана.

        - По нажатию на кнопку осуществляется повторная попытка подключения к интернету.

- Успех: приложение продолжает работу с переходом на главный экран или текущий процесс, требующий подключения.
- Ошибка: экран остается активным и снова отображает сообщение об ошибке.

Переходы

- В случае успешного подключения приложение переходит на главный экран или возвращается к процессу, который был прерван из-за отсутствия интернета.

Экран «Приветствия»

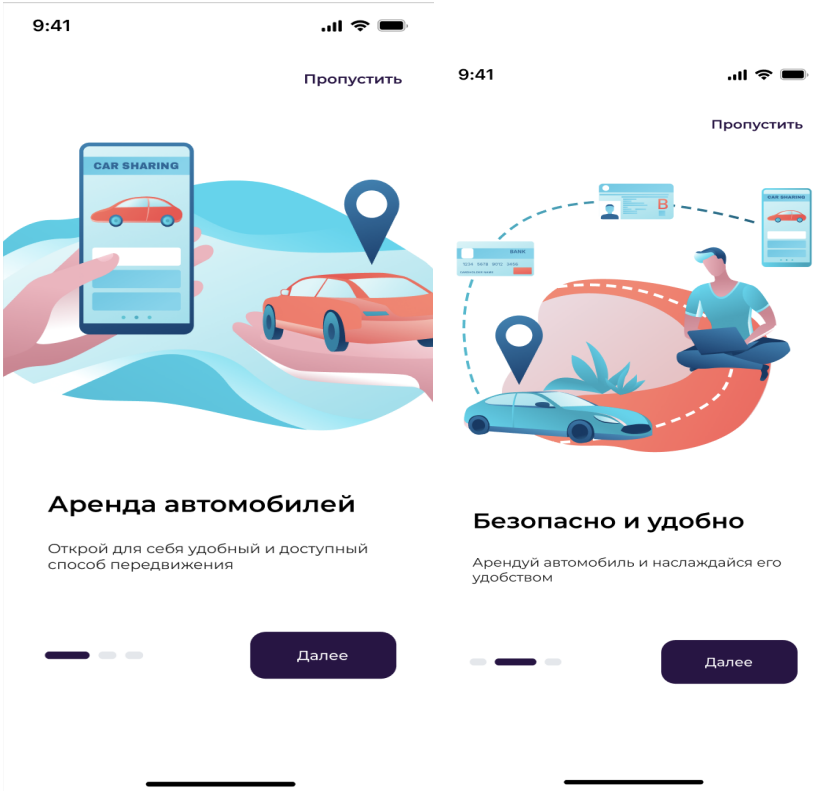




Рисунок 36 - Приветствие

Общее описание

Реализуйте экран приветствия для отображения преимуществ приложения согласно макету. Данный экран отображается при первом открытии приложения.

Функциональные требования

1. Отображение основных преимуществ:
    - Экран приветствия состоит из нескольких последовательных шагов (слайдов), которые пользователь может пролистать, чтобы ознакомиться с основными функциями и преимуществами приложения.
    - Каждый слайд содержит заголовок и краткое описание функциональности, представленной на изображении.
2. Кнопки:
    - «Пропустить»

        Кнопка становится активной сразу после отображения экрана.
        - При нажатии на кнопку пользователь сразу переходит к экрану входа/регистрации.
    - «Далее»

        Кнопка становится активной сразу после отображения экрана.
        - При нажатии на кнопку пользователь видит следующий слайд.
    - «Поехали»

        Кнопка становится активной на последнем слайде.
        - При нажатии на кнопку пользователь переходит к экрану входа/регистрации.

Переходы

- После прохождения приветственных экранов пользователь переходит на экран входа/регистрации.

Хранение данных

- После завершения процесса приветствия (Onboarding) информация о том, что пользователь прошел этот процесс, сохраняется, чтобы не показывать его повторно при последующих запусках приложения.

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Как создать переменную в Kotlin?
2. Что такое Activity?
3. Для чего нужен Intent?

4. Какова основная цель SplashScreen в Android-приложении и какие проблемы пользовательского восприятия он помогает решить?
5. Где и как указывается тема SplashScreen, чтобы она применялась при запуске приложения?
6. В каком месте жизненного цикла стартовой Activity должна происходить инициализация SplashScreen и почему это важно?
7. Где именно в проекте объявляются разрешения uses-permission и что произойдет, если нужного разрешения нет?
8. Для каких задач требуется разрешение android.permission.INTERNET, а для каких - android.permission.ACCESS_NETWORK_STATE?
9. Что такое файл AndroidManifest.xml и для чего он используется в Android-приложении?
10. Что такое Fragment и чем он отличается от Activity?

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Для выбора даты и времени в Android часто используют компоненты Material Components: MaterialDatePicker для даты и MaterialTimePicker для времени. Такой подход даёт единый внешний вид в стиле Material Design и удобную работу через колбэки. Удобно выводить выбранное значение в TextInputEditText, размещённый внутри TextInputLayout, чтобы слева можно было показать иконку календаря и по нажатию открывать диалоги выбора.

Необходимые импорты:

```
import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
import com.google.android.material.datepicker.MaterialDatePicker
import com.google.android.material.timepicker.MaterialTimePicker
import com.google.android.material.timepicker.TimeFormat
import com.google.android.material.textfield.TextInputEditText
import com.google.android.material.textfield.TextInputLayout
import java.text.SimpleDateFormat
import java.util.Calendar
import java.util.Date
import java.util.Locale
```

Пример класса с реализацией выбора даты и времени:

```
class MainActivity: AppCompatActivity() {
    // Форматирование даты для отображения в поле ввода,
например 31.01.2026
    private val dateFormatter = SimpleDateFormat("dd.MM.yyyy",
Locale.getDefault())

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val tilDateTime =
findViewById<TextInputLayout>(R.id.tilDateTime)
        val etDateTime =
findViewById<TextInputEditText>(R.id.etDateTime)

        // Общая функция открытия выбора даты, затем времени,
затем заполнения поля
        fun openDateTimePicker() {
            // Создаём диалог выбора даты и задаём заголовок
окна
            val datePicker =
MaterialDatePicker.Builder.datePicker()
                .setTitleText("Выбор даты")
                .build()
```

```
            // Обрабатываем подтверждение даты и получаем
выбранную дату в миллисекундах UTC
            datePicker.addOnPositiveButtonClickListener {
selectedDateUtcMillis ->
                val calendar = Calendar.getInstance().apply {
                    time = Date(selectedDateUtcMillis)
                }

                // Создаём диалог выбора времени, задаём 24-
часовой формат и стартовые значения
                val timePicker = MaterialTimePicker.Builder()
                    .setTitleText("Выбор времени")
                    .setTimeFormat(TimeFormat.CLOCK_24H)
                    .setHour(calendar.get(Calendar.HOUR_OF_DAY))
                    .setMinute(calendar.get(Calendar.MINUTE))
                    .build()

                // Обрабатываем подтверждение времени, собираем
итоговую дату-время и выводим в поле
                timePicker.addOnPositiveButtonClickListener {
                    calendar.set(Calendar.HOUR_OF_DAY,
timePicker.hour)
                    calendar.set(Calendar.MINUTE,
timePicker.minute)

                    val dateText =
dateFormatter.format(calendar.time)
                    val timeText = String.format(
                        Locale.getDefault(),
                        "%02d:%02d",
                        timePicker.hour,
                        timePicker.minute
                    )

                    etDateTime.setText("$dateText $timeText")
                }

                // Показываем выбор времени после того, как
выбрана дата
                timePicker.show(supportFragmentManager,
"time_picker")
            }

            // Показываем выбор даты
            datePicker.show(supportFragmentManager,
"date_picker")
        }

        // Нажатие на иконку календаря в начале поля
        tilDateTime.setStartIconOnClickListener {
openDateTimePicker() }

        // Нажатие на само поле ввода
```

```
        etDateTime.setOnClickListener { openDateTimePicker() }
    }
}
```

Пример реализации поля для выбора даты и времени в XML:

```
<com.google.android.material.textfield.TextInputLayout
    android:id="@+id/tilDateTime"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Дата и время"
    app:startIconDrawable="@drawable/ic_calendar"
    app:startIconContentDescription="Открыть выбор даты и
времени">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/etDateTime"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:focusable="false"
        android:clickable="true"
        android:inputType="none" />

</com.google.android.material.textfield.TextInputLayout>
```

Один из возможных вариантов реализации выбора одной фотографии из хранилища и создания одной фотографии с камеры - использовать Activity Result API. В обоих случаях приложение получает Uri и дальше работает с ним через contentResolver или отображает изображение в интерфейсе. Для выбора фотографии через системный пикер отдельные разрешения на чтение галереи обычно не требуются, так как доступ выдаётся через Uri. Для снимка с камеры нужен заранее подготовленный Uri, куда камера сохранит изображение, поэтому дополнительно настраивается FileProvider.

Необходимые импорты:

```
import android.net.Uri
import androidx.activity.result.PickVisualMediaRequest
import
androidx.activity.result.contract.ActivityResultContracts
```

Выбор одной фотографии из галереи:

- pickOneImage регистрирует контракт выбора изображения и получает результат в виде Uri;
- openGalleryForOneImage() запускает экран выбора фотографии.

```
  private val pickOneImage = registerForActivityResult(
      ActivityResultContracts.PickVisualMedia()
  ) { uri: Uri? ->
      if (uri != null) {
          selectedImageUri = uri
      }
```

```
    }

    private var selectedImageUri: Uri? = null

    private fun openGalleryForOneImage() {
        pickOneImage.launch(
    PickVisualMediaRequest(ActivityResultContracts.PickVisualMedia
    .ImageOnly)
        )
    }
```

Необходимые импорты для снимка с камеры:

```
import androidx.activity.result.contract.ActivityResultContracts
```

Сделать одну фотографию:

- pendingPhotoUri хранит Uri, куда камера должна записать снимок;
- takeOnePhoto запускает камеру и возвращает true, если снимок успешно сохранён;
- openCameraForOnePhoto(uri) запускает съёмку, передавая подготовленный Uri.

```
private var pendingPhotoUri: Uri? = null

private val takeOnePhoto = registerForActivityResult(
    ActivityResultContracts.TakePicture()
) { success: Boolean ->
    if (success) {
        selectedImageUri = pendingPhotoUri
    }
}

private fun openCameraForOnePhoto(outputUri: Uri) {
    pendingPhotoUri = outputUri
    takeOnePhoto.launch(outputUri)
}
```

Важно: outputUri для камеры должен быть получен через FileProvider, иначе системная камера не сможет безопасно записать файл. Для этого настраивается FileProvider в AndroidManifest.xml и файл путей в res/xml/file_paths.xml.

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Экран «Входа/регистрации»

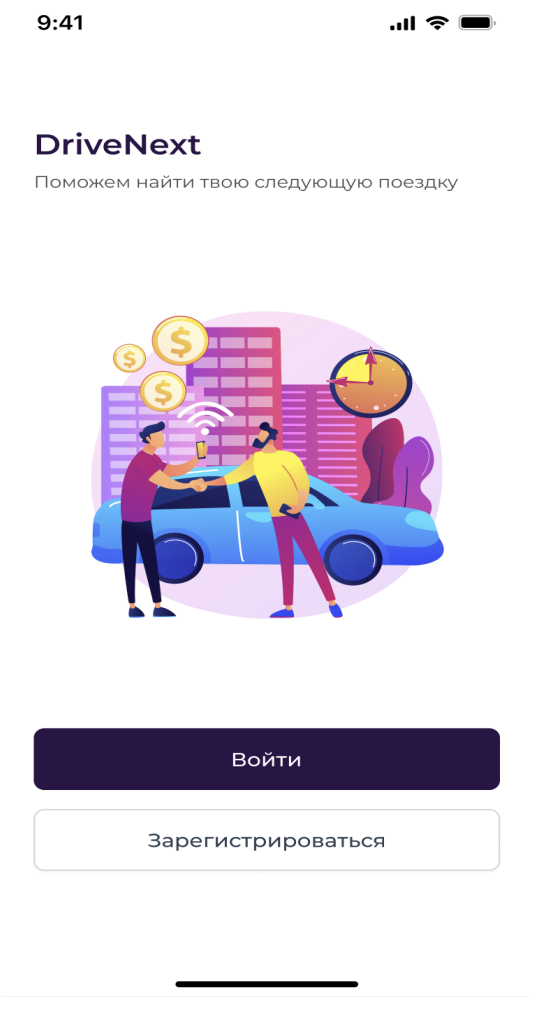


Рисунок 37 - Вход/регистрация

Общее описание

Реализуйте экран выбора действия (входа/регистрации) для отображения основных опций приложения согласно макету. Экран предоставляет пользователю возможность либо войти в существующую учетную запись, либо зарегистрироваться, чтобы начать использовать приложение.

Функциональные требования

1. Отображение названия, логотипа и слогана приложения:
    - Название отображается сверху по левому краю экрана.
    - Слоган «Поможем найти твою следующую поездку» отображается под логотипом.
    - Логотип приложения отображается в центре экрана.
2. Кнопки:
    - «Войти»

Кнопка становится активной сразу после отображения экрана.

- ведет пользователя на экран авторизации, где он может ввести свои учетные данные для входа в приложение.

- «Зарегистрироваться»

  Кнопка становится активной сразу после отображения экрана.

  - ведет пользователя на экран регистрации, где он может создать новый аккаунт.

Переходы

- Пользователь переходит на следующие экраны в зависимости от того, на какую кнопку было произведено нажатие: «Войти» пользователь переходит на экран авторизации, «Зарегистрироваться» пользователь переходит на экран регистрации.

Экран «Вход в аккаунт»

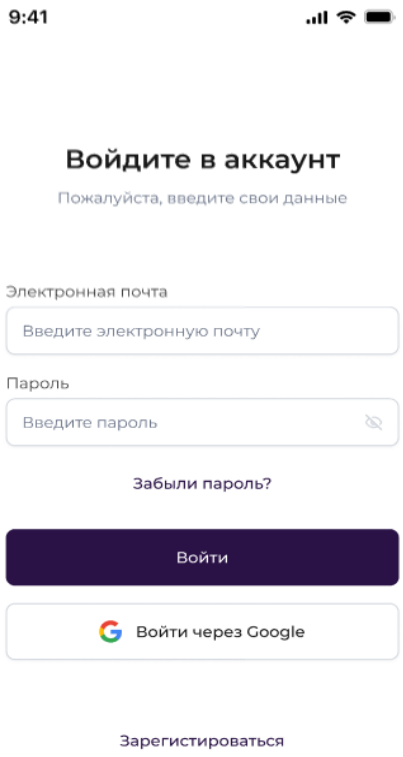


Рисунок 38 – Вход в аккаунт

Общее описание

Реализуйте экран входа в приложения согласно макету. Точкой входа в данный экран является нажатие на кнопку «Войти» на экране входа/регистрации.

Функциональные требования

1. Поля ввода:
    - Электронная почта: обязательное поле, проверка корректности (паттерн «name@domainname.com или .ru»).
    - Пароль: обязательное поле, возможность показать/скрыть пароль при нажатии на иконку глаза.
2. Кнопки:
    - «Войти»

        Кнопка становится активной только после того, как будут заполнены все поля.

        - По нажатию отправить запрос на авторизацию.
        - Успех: сохранить токен, открыть «главный экран».
        - Ошибка: отобразить сообщение об ошибке.

- «Войти через Google»
  - По нажатию инициировать процесс авторизации через Google OAuth.
  - Успех: сохранить токен, открыть пустой «главный экран».
  - Ошибка: отобразить сообщение об ошибке.
- «Зарегистрироваться»
  - По нажатию перейти на экран регистрации.

Переходы

- При успешной авторизации открыть «Главный экран».
- При нажатии на «Зарегистрироваться» перейти на экран регистрации.

Хранение данных

- Сохранить данные входа и активную сессию.

Экран «Регистрация» (шаг 1)

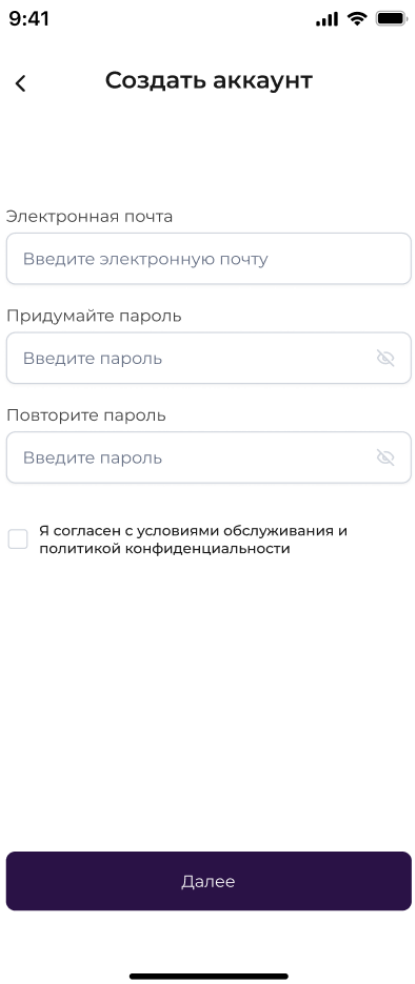


Рисунок 39 – Регистрация (шаг 1)

Общее описание

Реализуйте экран для создания нового аккаунта пользователя в приложении согласно макету. Точкой входа в данный экран является нажатие на кнопку «Зарегистрироваться» на экране входа/регистрации или на экране входа.

Функциональные требования

1. Поля ввода:
    - Электронная почта:
        - Обязательное поле, проверка корректности (паттерн «name@domainname.com или .ru»).
    - Придумайте пароль:
        - Обязательное поле, возможность показать/скрыть пароль при нажатии на иконку глаза.
    - Повторите пароль:

        - Обязательное поле, проверка совпадения с полем «Придумайте пароль», возможность показать/скрыть пароль при нажатии на иконку глаза.

2. Чекбокс:
    - Согласие с условиями обслуживания и политикой конфиденциальности:
        - Обязательный для отметки перед продолжением.

Переходы

1. Кнопка Далее»:
    - Перейти на следующий экран регистрации.
2. Кнопка «Назад»:
    - По нажатию вернуть пользователя на предыдущий экран.

Сообщения об ошибках

Любым образом отобразите сообщения об ошибках

- Некорректный email: «Введите корректный адрес электронной почты.»
- Несовпадающие пароли: «Пароли не совпадают.»
- Неотмеченный чекбокс: «Необходимо согласиться с условиями обслуживания и политикой конфиденциальности.»

Экран «Регистрация» (шаг 2)

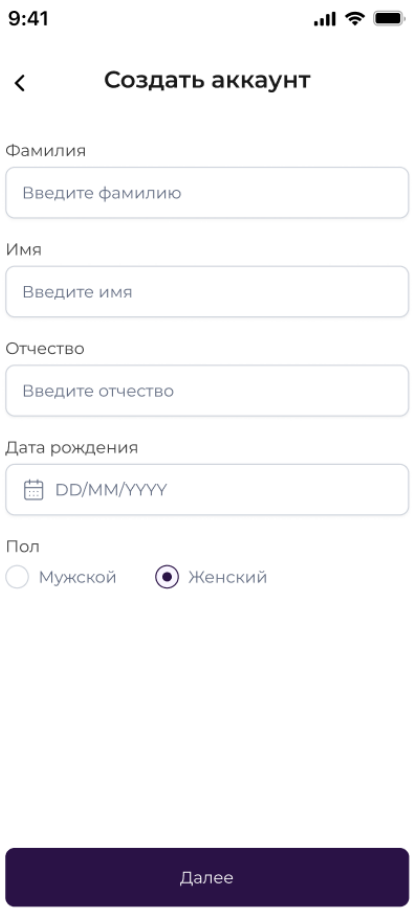


Рисунок 40 – Регистрация (шаг 2)

Общее описание

Реализуйте экран для ввода дополнительных данных при создании нового аккаунта пользователя в приложении согласно макету.

Функциональные требования

1. Поля ввода:
    - Фамилия: обязательное поле.
    - Имя: обязательное поле.
    - Отчество: необязательное поле.
    - Дата рождения: обязательное поле, проверка корректности формата (MM/DD/YYYY).
    - Пол: обязательное поле, выбор между «Мужской» и «Женский».
2. Кнопки:
    - «Далее»:

    Кнопка становится активной только после того, как будут заполнены обязательные поля.

        - Успех: завершить регистрацию и перейти на главный экран.
        - Ошибка: отобразить сообщение об ошибке.
    - «Назад»:
        - По нажатию вернуться на предыдущий экран регистрации (шаг 1), где будут заполненные ранее поля.

Переходы

- При успешной проверке данных перейти на следующий экран регистрации.
- При нажатии «Назад» перейти на первый экран регистрации.

Сообщения об ошибках

- Незаполненные обязательные поля: «Пожалуйста, заполните все обязательные поля.»
- Некорректная дата рождения: «Введите корректную дату рождения.»

Экран «Регистрация» (шаг 3)

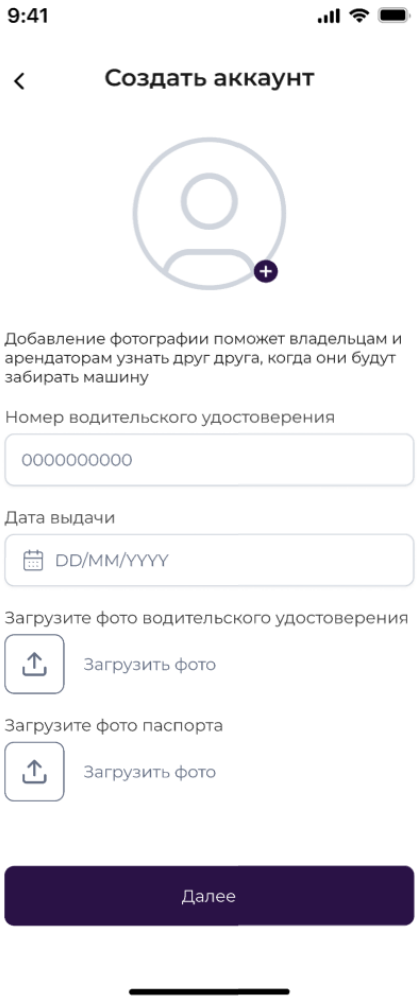


Рисунок 41 – Регистрация (шаг 3)

Общее описание

Реализуйте экран для загрузки документов и фото пользователя при создании нового аккаунта в приложении согласно макету.

Функциональные требования

1. Добавление фотографии:
    - Иконка для добавления фото профиля: необязательно для заполнения.
    - По нажатию на иконку открыть галерею/файлы или камеру для выбора фото.
2. Поля ввода:
    - Номер водительского удостоверения: обязательное поле, необходимое значение 10 символов.
    - Дата выдачи: обязательное поле, проверка корректности формата (DD/MM/YYYY).
3. Загрузка документов:
    - Загрузить фото водительского удостоверения – обязательное действие:

        - Кнопка «Загрузить фото» для открытия галереи или камеры.
    - Загрузить фото паспорта – обязательное действие:
        - Кнопка «Загрузить фото» для открытия галереи или камеры.
4. Кнопки:
    - «Далее»:

    Кнопка становится активной только после того, как будут заполнены обязательные поля.

        - Успех: завершить регистрацию и перейти на экран «Успешная регистрация».
        - Ошибка: отобразить сообщение об ошибке.
    - «Назад»:
        - По нажатию вернуться на предыдущий экран регистрации (шаг 2).

Сообщения об ошибках

- Незаполненные обязательные поля: «Пожалуйста, заполните все обязательные поля.»
- Некорректная дата выдачи: «Введите корректную дату выдачи.»
- Отсутствие загруженных фото: «Пожалуйста, загрузите все необходимые фото.»

Экран «Успешная регистрация»

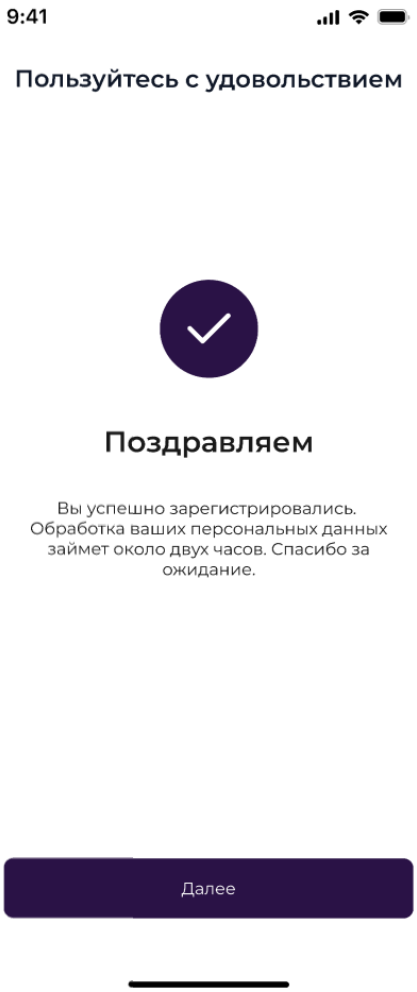


Рисунок 42 – Успешная регистрация

Общее описание

Реализуйте экран поздравления после успешной регистрации пользователя в приложении согласно макету.

Функциональные требования

1. Кнопка:
    - «Далее»: перейти на главный экран приложения и очистка стека навигации.

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Что такое MaterialDatePicker и для чего он применяется в приложении?
2. Какие основные шаги нужны, чтобы создать MaterialDatePicker через Builder и настроить заголовок окна?
3. Как показать MaterialDatePicker на экране с помощью supportFragmentManager?
4. Для чего используется addOnPositiveButtonClickListener у MaterialDatePicker?
5. Что означает значение selectedDateUtcMillis, которое возвращает MaterialDatePicker?

6. Что такое Uri и почему именно его получает приложение при выборе изображения из системного пикера?

7. Для чего используется Activity Result API и зачем регистрируют контракт через registerForActivityResult?

8. Что делает контракт ActivityResultContracts.PickVisualMedia при выборе одной фотографии из галереи?

9. Что делает контракт ActivityResultContracts.TakePicture и какое значение он возвращает?

10. Зачем нужен FileProvider при съёмке фотографии камерой и сохранении результата по Uri?

ЛАБОРАТОРНАЯ РАБОТА №3

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

SearchView - это виджет для ввода поискового запроса и запуска фильтрации данных на экране, например, списка, таблицы или набора карточек. Он предоставляет пользователю привычный сценарий поиска, где слева отображается иконка лупы, а введённый текст передаётся в обработчик изменений, чтобы приложение могло обновлять отображаемые элементы по мере ввода. Чаще всего настраиваются android:queryHint для текста-подсказки, android:iconifiedByDefault для режима отображения, android:inputType для типа ввода, а также android:maxWidth или параметры ширины layout для корректного размещения в интерфейсе. В Kotlin подключают слушатель SearchView.OnQueryTextListener и обрабатывают два события, изменение текста запроса и подтверждение поиска.

Обработка ввода запроса:

```
val searchView = findViewById<SearchView>(R.id.searchView)

searchView.setOnQueryTextListener(object :
SearchView.OnQueryTextListener {
    override fun onQueryTextSubmit(query: String?): Boolean {
        // вызвать поиск по окончательному запросу
        return true
    }

    override fun onQueryTextChange(newText: String?): Boolean {
        // фильтровать данные при изменении текста
        return true
    }
})
```

Пример SearchView:

```
<SearchView
    android:id="@+id/searchView"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:queryHint="Поиск"
    android:iconifiedByDefault="false"
    android:inputType="text" />
```

RecyclerView - это компонент для отображения списков и сеток, когда количество элементов заранее неизвестно или может меняться во время работы приложения. Его применяют для лент, каталогов, списков сообщений, карточек товаров и любых динамических наборов данных. В отличие от статической верстки через LinearLayout с набором заранее размещённых View, RecyclerView рассчитан на большие списки и работает экономно: он создаёт ограниченное число представлений, отображает только те элементы, которые попадают в область экрана, и переиспользует уже созданные View при прокрутке. Это снижает нагрузку на память и ускоряет отрисовку интерфейса. Для управления расположением элементов RecyclerView использует layoutManager, наиболее распространённые варианты - LinearLayoutManager для вертикального или горизонтального списка, GridLayoutManager для сетки, StaggeredGridLayoutManager для «мозаики» с разной высотой элементов.

Пример вёрстки RecyclerView в layout:

```
<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/recyclerView"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

Пример вёрстки ячейки списка item_user.xml:

```
<LinearLayout

xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="12dp">

    <TextView
        android:id="@+id/nameText"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Имя"
        android:textSize="16sp"
        android:textStyle="bold" />

    <TextView
        android:id="@+id/descText"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Описание"
        android:layout_marginTop="4dp" />
</LinearLayout>
```

Обработка и инициализация RecyclerView:

```
val                        recyclerView                              =
findViewById<RecyclerView>(R.id.recyclerView)

// Назначение LayoutManager
recyclerView.layoutManager = LinearLayoutManager(this)

// Создание адаптера и передача списка данных
val items = listOf(
    UserItem("Иван", "Описание 1"),
    UserItem("Мария", "Описание 2")
)

recyclerView.adapter = UserAdapter(items)
```

Пример модели данных:

```
data class UserItem(
```

```
        val name: String,
        val description: String
    )
```

Файл ViewHolder, хранит ссылки на элементы ячейки и ускоряет доступ к ним:

```
    class UserViewHolder(itemView: View:
RecyclerView.ViewHolder(itemView) {
        val nameText: TextView =
itemView.findViewById(R.id.nameText)
        val descText: TextView =
itemView.findViewById(R.id.descText)
    }
```

Файл Adapter, связывает данные и ячейки списка:

```
    class UserAdapter(
        private val items: List<UserItem>,
    ): RecyclerView.Adapter<UserViewHolder>() {

        // Создаёт View для ячейки и возвращает ViewHolder
        override fun onCreateViewHolder(parent: ViewGroup,
viewType: Int): UserViewHolder {
            val view = LayoutInflater.from(parent.context)
                .inflate(R.layout.item_user, parent, false)
            return UserViewHolder(view)
        }

        // Заполняет ячейку данными по позиции
        override fun onBindViewHolder(holder: UserViewHolder,
position: Int) {
            val item = items[position]
            holder.nameText.text = item.name
            holder.descText.text = item.description
        }

        // Возвращает количество элементов списка
        override fun getItemCount(): Int = items.size
    }
```

Нижнее навигационное меню - элемент интерфейса, который постоянно закреплён в нижней части экрана и используется для быстрого переключения между основными разделами приложения. Обычно его применяют, когда в приложении есть 3-5 ключевых экранов, между которыми пользователь часто переключается. В простейшем варианте меню можно сверстать через горизонтальный LinearLayout с распределением ширины вкладок, где каждая вкладка занимает равное пространство и содержит иконку и подпись, а выбранное состояние отображается изменением цвета иконки и текста. Такой подход позволяет реализовать навигацию без сложных компонентов, при этом логика выбора вкладки остаётся полностью управляемой в коде.

Чтобы реализовать это меню на экране, можно организовать разметку так: основную область контента сделать отдельным контейнером, например, FrameLayout или другим корневым контейнером под экран, а внизу закрепить горизонтальный LinearLayout с тремя ячейками вкладок. Каждая ячейка - это кликабельный контейнер, внутри которого

находится ImageView для иконки и TextView для подписи, поэтому визуально получаются три «кнопки» нижнего меню. В коде логика сводится к тому, чтобы хранить текущую выбранную вкладку, на нажатие каждой ячейки менять выбранную вкладку, обновлять состояние оформления всех трёх вкладок, выбранной задавать активные цвета, остальным неактивные, и одновременно подменять содержимое основного контейнера, например, отображая нужный экран через замену фрагмента или переключение видимости нужного блока интерфейса.

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Экран «Настройки»

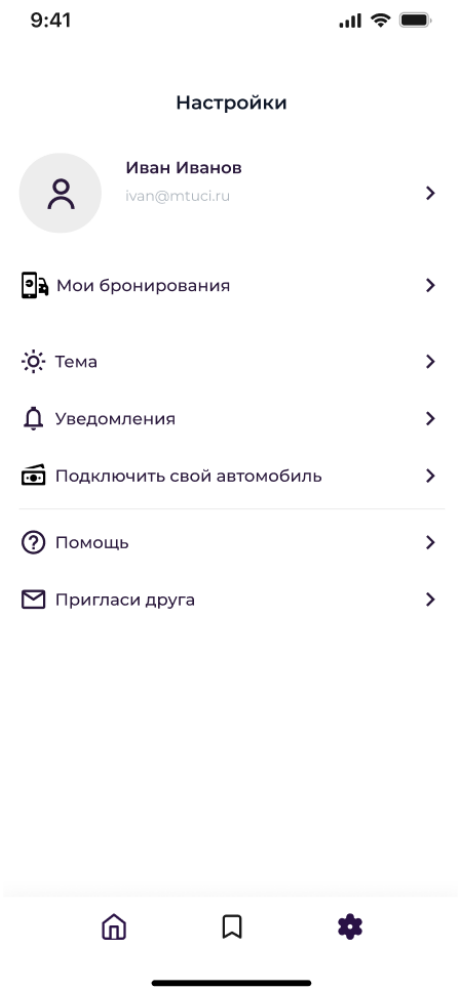


Рисунок 43 – Настройки

Общее описание

Реализуйте экран настроек пользователя в приложении согласно макету. Точкой входа в данный экран является нажатие на иконку шестеренки на нижней панели навигации.

Функциональные требования

1. Профиль пользователя:
   - Отображение аватара пользователя.
   - Отображение имени пользователя.
   - Отображение электронной почты пользователя.
   - По нажатию переход на экран «Профиль».
2. Меню настроек:
   - Мои бронирования: по нажатию переход на экран «Мои бронирования»
   - Тема

- Уведомления
- Подключить свой автомобиль: по нажатию переход на экран добавления автомобиля.
- Помощь
- Пригласи друга

Хранение данных

- Сохранение изменений профиля.

Экран «Профиль»

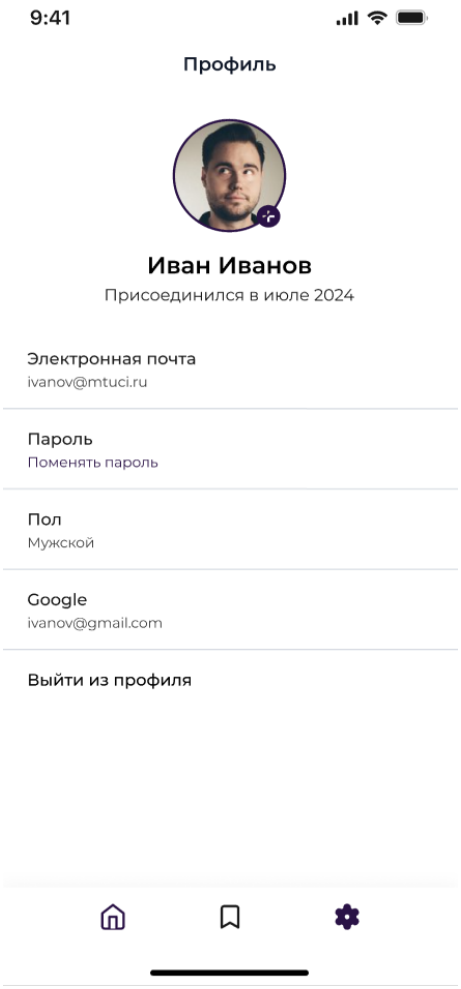


Рисунок 44 – Профиль

Общее описание

Реализуйте экран профиля пользователя в приложении согласно макету. Точкой входа является нажатие на профиль пользователя на экране «Настройки»

Функциональные требования

1. Аватар пользователя:
    - Отображение текущего аватара.
    - По нажатию открытие возможности загрузить новый аватар.
2. Кнопка «Выйти из профиля»:
    - По нажатию выполнение выхода пользователя из аккаунта и возвращение на экран входа в аккаунт и очистка стека навигации.

Переходы

- При нажатии на «Выйти из профиля» выполнение действия выхода и переход на экран входа.

Экран «Главная»

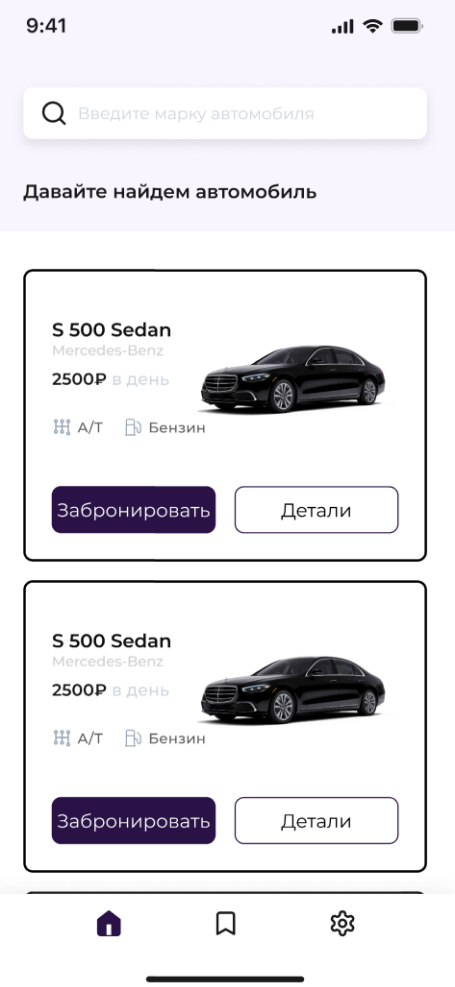


Рисунок 45 – Главная

Общее описание

Реализуйте экран для отображения списка доступных автомобилей для аренды в приложении согласно макету. Точкой входа в данный экран является нажатие на иконку дома на нижней панели навигации. Экран не должен быть статичным, а должен иметь возможность прокрутки вверх или вниз.

Функциональные требования

1. Отображение списка автомобилей:
    - Список всех доступных автомобилей должен быть загружен с сервера при открытии экрана.
    - Каждый автомобиль должен отображаться в виде карточки с информацией о модели, марке, цене и основных характеристиках.
2. Поиск автомобилей:
    - Введите марку или модель автомобиля в поле поиска.
    - После ввода информации должен осуществляться поиск автомобилей с индикатором загрузки и переход на экран «Результаты поиска».

3. Действия с автомобилем:
    - Забронировать автомобиль:
        - По нажатию на кнопку «Забронировать» переход на экран «Оформление аренды».
    - Просмотр деталей:
        - По нажатию на кнопку «Детали» переход на экран «Детали».
4. Загрузка данных:
    - При загрузке данных должен отображаться индикатор загрузки.
    - В случае ошибки загрузки данных показывать сообщение об ошибке и кнопку для повторной попытки загрузки.

Сообщения об ошибках

- Ошибка поиска: «Не удалось выполнить поиск. Попробуйте снова.»
- Ошибка загрузки данных: «Не удалось загрузить данные. Попробуйте снова.»

Экран «Результаты поиска»

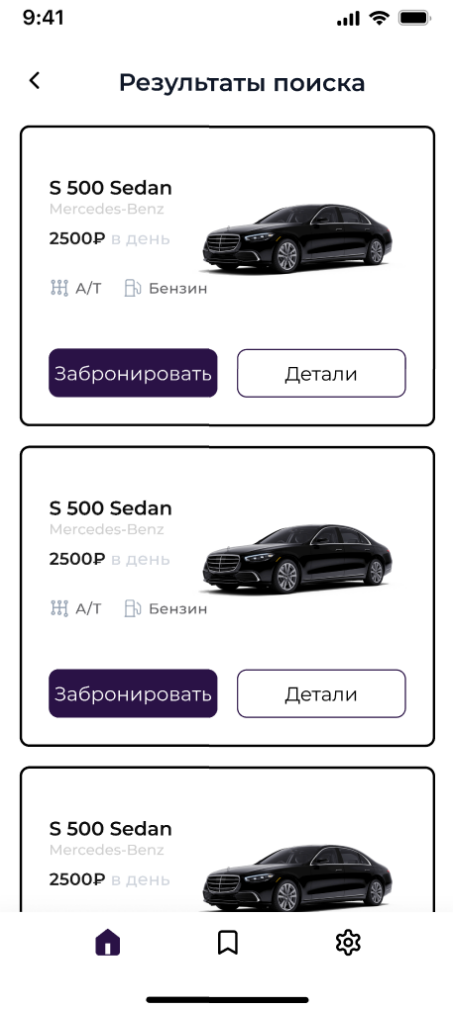


Рисунок 46 – Результаты поиска

Общее описание

Реализуйте экран для отображения результатов поиска автомобилей в приложении согласно макету. Точкой входа в данный экран является ввод запроса в поле поиска на экране «Главная».

Функциональные требования

1. Отображение результатов поиска:
    - Результаты поиска автомобилей должны быть загружены с сервера на основе введенного запроса.
    - Каждый автомобиль должен отображаться в виде карточки с информацией о модели, марке, цене и основных характеристиках.
2. Действия с автомобилем:
    - Забронировать автомобиль:
        - По нажатию на кнопку «Забронировать» переход на экран «Оформление аренды».

- Просмотр деталей:
  - По нажатию на кнопку «Детали» переход на экран «Детали».

3. Загрузка данных:
   - При загрузке данных должен отображаться индикатор загрузки.
   - В случае ошибки загрузки данных показывать сообщение об ошибке и кнопку для повторной попытки загрузки.
4. Обновление данных:
   - Обновление списка автомобилей при изменении поискового запроса.
   - Обновление результатов поиска при перезапуске приложения.

Сообщения об ошибках

Любым способом отобразите сообщения об ошибках в случае их возникновения:

- Ошибка поиска: «Не удалось выполнить поиск. Попробуйте снова.»
- Ошибка загрузки данных: «Не удалось загрузить данные. Попробуйте снова.»

Контрольные вопросы

1. Что такое SearchView и для чего он используется?
2. Как можно обновлять список результатов при изменении текста в SearchView?
3. Для чего нужны методы onQueryTextSubmit и onQueryTextChange в SearchView?
4. Что такое RecyclerView и какие задачи он решает при отображении списков?
5. Что такое Adapter в RecyclerView и за что он отвечает?
6. Какие методы обычно переопределяются в адаптере RecyclerView (onCreateViewHolder, onBindViewHolder, getItemCount) и зачем?
7. Что такое LayoutManager в RecyclerView и какие типы LayoutManager используются чаще всего?
8. Для чего используется нижнее навигационное меню (BottomNavigationView) в приложении?
9. Где обычно описываются пункты меню для BottomNavigationView (в каком ресурсе проекта)?
10. Как обычно организуют переключение экранов по нажатию пунктов BottomNavigationView (например, через фрагменты/Navigation Component)?

ЛАБОРАТОРНАЯ РАБОТА №4

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

SharedPreferences - это встроенный механизм Android для хранения небольших объёмов данных в виде пар ключ значение. Он используется, когда требуется быстро сохранить простые настройки и состояния приложения между запусками, например, выбранную тему, имя пользователя, флаг первого запуска, параметры фильтрации или последний введённый текст. Данные SharedPreferences сохраняются локально на устройстве и доступны приложению, при этом формат хранения ориентирован на примитивные типы и набор строк. Для записи используется объект SharedPreferences.Editor, где изменения фиксируются через apply или commit, а чтение выполняется по ключу с указанием значения по умолчанию, если запись отсутствует.

```
// Получение объекта SharedPreferences по имени хранилища
val prefs = getSharedPreferences("app_prefs", MODE_PRIVATE)

// Чтение значения по ключу, если ключ отсутствует, возвращается значение по умолчанию
val isFirstRun = prefs.getBoolean("is_first_run", true)

// Создание редактора для записи изменений
val editor = prefs.edit()

// Запись значения по ключу
editor.putBoolean("is_first_run", false)

// Применение изменений асинхронно
editor.apply()
```

Room - это библиотека Jetpack для работы с локальной базой данных SQLite на устройстве, которая даёт типобезопасный доступ к данным и снижает количество ручного кода по сравнению с прямыми SQL вызовами. Room подключается как зависимость в модуле приложения через implementation в файле build.gradle уровня модуля app. База данных в Room описывается через сущности и интерфейсы доступа, а запросы выполняются через аннотации и методы DAO. Такой подход применяется, когда требуется хранить структурированные данные дольше одного запуска приложения и выполнять выборки, вставки, обновления и удаления без использования удалённого сервера.

Сущность Entity - класс, который описывает таблицу базы данных, где поля класса становятся колонками, а объект соответствует строке таблицы. В Entity задаётся имя таблицы, первичный ключ и типы данных, которые Room сможет сохранять.

```
import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity(tableName = "notes")
data class NoteEntity(
    @PrimaryKey(autoGenerate = true) val id: Long = 0L,
    val title: String,
    val text: String
)
```

DAO (Data Access Object) - интерфейс или абстрактный класс, который описывает операции доступа к данным, то есть методы для чтения и изменения таблиц. В DAO

задаются запросы через аннотации, а Room генерирует реализацию этих методов во время сборки.

```
import androidx.room.Dao
import androidx.room.Insert
import androidx.room.Query

@Dao
interface NoteDao {

    // Выполняется чтение всех записей таблицы notes,
сортировка выполняется по id по убыванию
    @Query("SELECT * FROM notes ORDER BY id DESC")
    fun getAll(): List<NoteEntity>

    // Выполняется вставка новой записи в таблицу notes
    @Insert
    fun insert(note: NoteEntity)
}
```

Класс базы данных Room - это абстрактный класс, помеченный @Database, который объединяет список сущностей и предоставляет доступ к DAO. Экземпляр базы данных обычно создаётся как singleton, то есть как единственный объект на всё приложение. Для потокобезопасного создания singleton применяется synchronized блок, то есть участок кода, в который одновременно может войти только один поток, поэтому два потока не создадут базу данных параллельно.

```
import android.content.Context
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase

@Database(entities = [NoteEntity::class], version = 1)
abstract class AppDatabase : RoomDatabase() {

    // Предоставляется доступ к DAO для выполнения операций
с таблицей notes
    abstract fun noteDao(): NoteDao

    companion object {
        @Volatile
        private var INSTANCE: AppDatabase? = null

        // Возвращается единственный экземпляр базы данных,
при отсутствии выполняется создание
        fun getInstance(context: Context): AppDatabase {
            return INSTANCE ?: synchronized(this) {
                // Создаётся база данных через builder,
задаётся имя файла базы данных app_db
                val instance = Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "app_db"
```

```
                    ).build()
                    INSTANCE = instance
                    instance
                }
            }
        }
    }
```

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Экран «Детали»

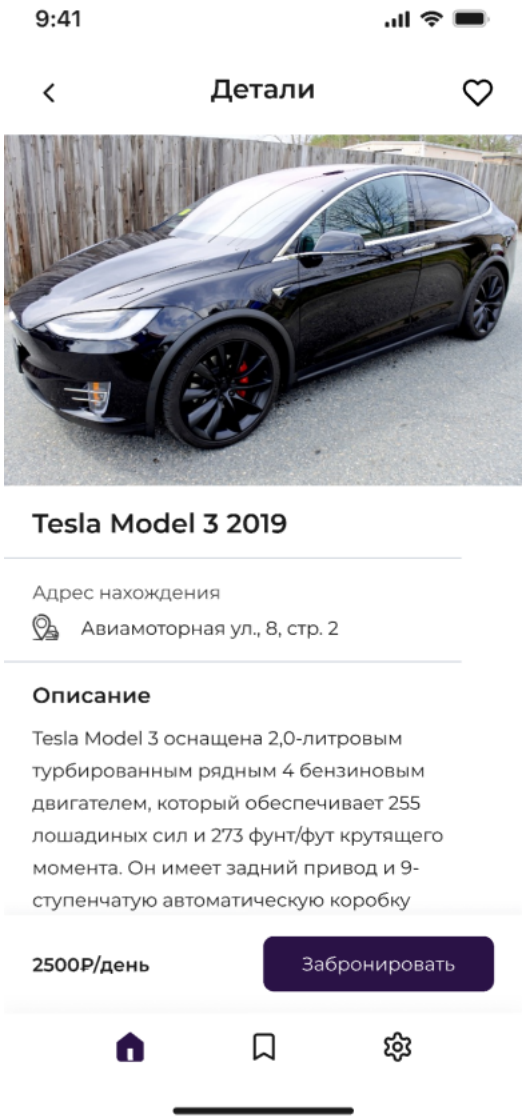


Рисунок 47 – Детали

Общее описание

Реализуйте экран для отображения детальной информации об автомобиле в приложении согласно макету. Точкой входа в данный экран является нажатие на кнопку «Детали» на экране результатов поиска или каталога автомобилей.

Функциональные требования

1. Действия с автомобилем:

- Забронировать автомобиль:
  - По нажатию на кнопку «Забронировать» переход на экран «Оформление аренды».
- Добавить в избранное:
  - По нажатию на иконку «сердце» добавление/удаление автомобиля в список избранного и изменение цвета данной иконки.

2. Загрузка данных:
   - При загрузке данных должен отображаться индикатор загрузки.
   - В случае ошибки загрузки данных показывать сообщение об ошибке.
3. Навигация:
   - Кнопка «Назад»: по нажатию возврат на предыдущий экран.

Сообщения об ошибках

Любым способом отобразите сообщения об ошибках в случае их возникновения:

- Ошибка загрузки данных: «Не удалось загрузить данные. Попробуйте снова.»
- Ошибка добавления в избранное: «Не удалось добавить в избранное. Попробуйте снова.»

Экран «Оформление аренды»

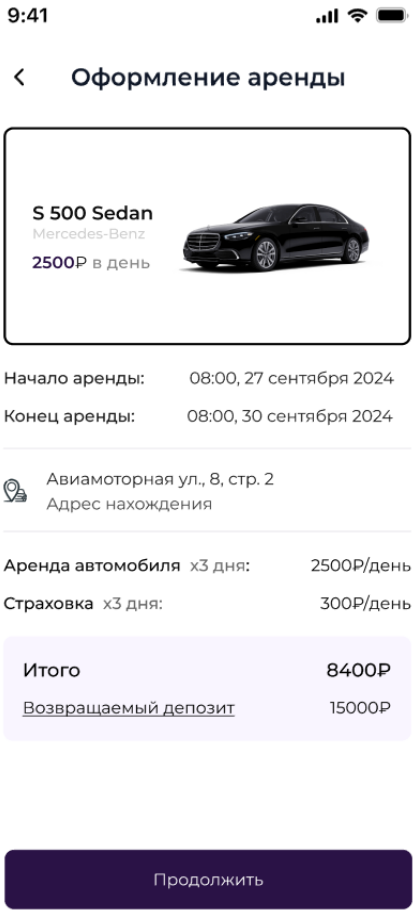


Рисунок 48 – Оформление аренды

Общее описание

Реализуйте экран для оформления бронирования автомобиля в приложении согласно макету. Точкой входа в данный экран является нажатие на кнопку «Забронировать» на экране результатов поиска, каталога автомобилей или экране деталей.

Функциональные требования

1. Отображение информации о бронировании:
    - Название автомобиля, модель, марка и стоимость аренды в день.
    - Даты начала и конца аренды с возможностью их выбора.
    - Адрес местоположения автомобиля.
2. Стоимость:
    - Отображение стоимости аренды автомобиля за весь период.
    - Отображение стоимости страховки за весь период.
    - Отображение итоговой суммы (с учетом аренды и страховки).
    - Отображение возвращаемого депозита.
3. Кнопка «Продолжить»:

- При нажатии на кнопку происходит переход на следующий шаг оформления аренды и очистка стека навигации.

Переходы

- Кнопка «Продолжить»: Переход на следующий экран, уведомляющий пользователя об успешном бронировании.

Экран «Удачной поездки»

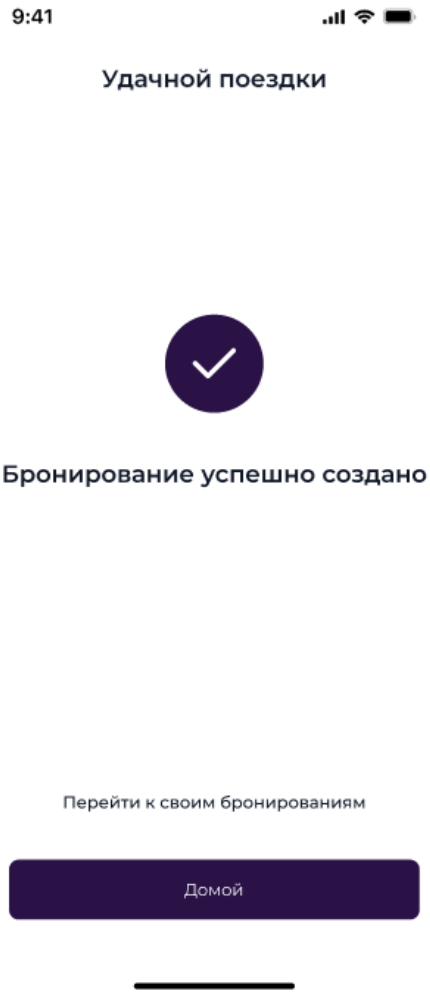


Рисунок 49 – Удачной поездки

Общее описание

Реализуйте экран для отображения успешного бронирования автомобиля в приложении согласно макету. Точкой входа в данный экран является нажатие на кнопку «Далее» на экране оформления бронирования.

Функциональные требования

1. Сообщение об успешном бронировании:
    - Отображение сообщения о том, что бронирование было успешно создано.
2. Кнопки навигации:
    - «Перейти к своим бронированиям»: По нажатию на эту кнопку пользователь переходит на экран с перечнем его активных бронирований.
    - «Домой»: По нажатию на эту кнопку пользователь возвращается на главный экран приложения.

Экран «Мои бронирования»

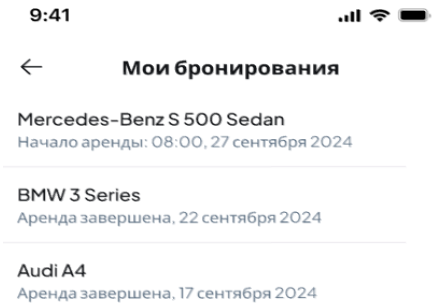


Рисунок 50 – Мои бронирования

Общее описание

Реализуйте экран для отображения списка активных и завершенных бронирований автомобилей в приложении согласно макету. Точкой входа в данный экран является нажатие на кнопку «Перейти к своим бронированиям» с экрана успешного бронирования или выбор пункта «Мои бронирования» на экране настроек.

Функциональные требования

1. Отображение списка бронирований:
    - Список включает активные и завершенные бронирования.
    - Для каждого активного бронирования отображается марка, модель автомобиля, а также дата и время начала аренды.
    - Для завершенных бронирований отображается марка, модель автомобиля, а также дата завершения аренды.

Переходы

- Нажатие на активное бронирование: Переход на экран с детальной информацией о бронировании.
- Кнопка «Назад»: Возвращение на предыдущий экран.

Экран «Детальная информация о бронировании»



Рисунок 51 – Детальная информация о бронировании

Общее описание

Реализуйте экран для отображения детальной информации о конкретном бронировании автомобиля в приложении согласно макету. Точкой входа в данный экран является нажатие на активное бронирование на экране списка бронирований.

Функциональные требования

1. Отображение информации о бронировании:
    - Номер бронирования.
    - Название автомобиля, марка, модель и стоимость аренды в день.
    - Адрес местоположения автомобиля.
    - Даты начала и окончания аренды.
    - ФИО водителя.
    - Номер водительского удостоверения.
    - Статус аренды (например, «Одобрено»).
2. Отображение стоимость:
    - Отображение стоимости аренды автомобиля за весь период.

- Отображение стоимости страховки за весь период.
- Отображение общей суммы бронирования.

Переходы

- Кнопка «Отменить бронирование» в случае активного бронирования: Выполнение действия отмены бронирования и переход на экран списка бронирований.
- Кнопка «Назад»: Возвращение на предыдущий экран без изменений.

Экран «Избранное»

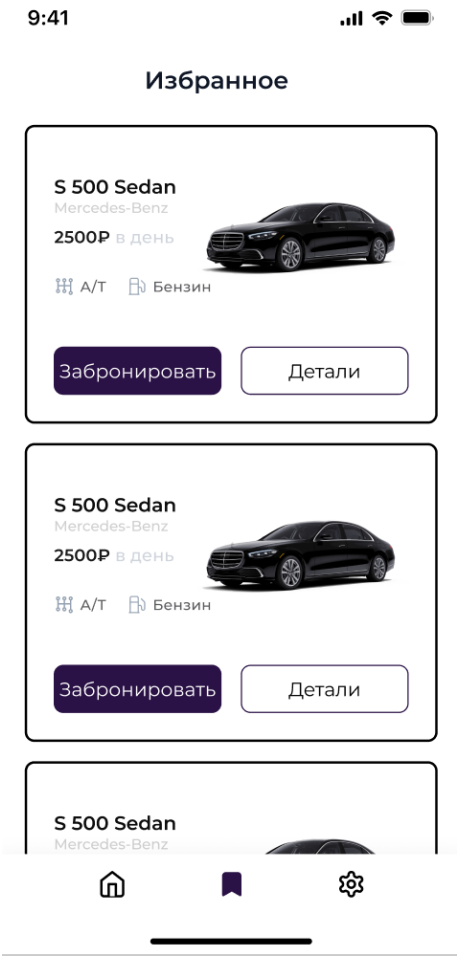


Рисунок 52 – Избранное

Общее описание

Реализуйте экран для отображения избранных автомобилей в приложении согласно макету. Точкой входа в данный экран является нажатие на иконку «Избранное» в нижней навигации.

Функциональные требования

1. Отображение списка избранных автомобилей:
    - Каждый автомобиль отображается в виде карточки с информацией о модели, марке, цене и основных характеристиках.
    - В каждой карточке доступны две кнопки: «Забронировать» и «Детали».
2. Действия с автомобилем:
    - Забронировать автомобиль:
        - По нажатию на кнопку «Забронировать» происходит переход на экран оформления аренды.
    - Просмотр деталей:

- По нажатию на кнопку «Детали» происходит переход на экран с детальной информацией об автомобиле.

Переходы

- Кнопка «Забронировать»: Переход на экран оформления аренды.
- Кнопка «Детали»: Переход на экран с детальной информацией об автомобиле.

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Что такое SharedPreferences и для чего он используется?
2. Какие типы данных поддерживает SharedPreferences?
3. Какие основные методы используются для записи данных в SharedPreferences через Editor (например, put*, apply, commit)?
4. В чём различие между apply() и commit() при сохранении данных в SharedPreferences?
5. Что такое Room и какую задачу он решает при работе с локальной базой данных в Android?
6. Что такое Entity в Room и как она связана с таблицей базы данных?
7. Что такое DAO в Room и для чего он нужен?
8. Какие основные аннотации используются в Room для сущностей и DAO (например, @Entity, @Dao, @Query, @Insert)?
9. Как обычно создают экземпляр базы данных Room через Room.databaseBuilder(...)?
10. Зачем используют synchronized-блок при создании singleton-экземпляра базы данных Room?

ЛАБОРАТОРНАЯ РАБОТА №5

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Spinner - это элемент интерфейса Android, представляющий выпадающий список, в котором пользователь выбирает одно значение из набора. Он используется, когда требуется компактно показать перечень вариантов, например, категорию, сортировку, город или тип фильтра. Данные для Spinner задаются через адаптер, который связывает список значений с отображением в выпадающем меню, а выбранный элемент обрабатывается через слушатель, чтобы приложение могло реагировать на выбор пользователя.

Обработка выбора элемента Spinner:

```
import android.widget.AdapterView
import android.widget.ArrayAdapter
import android.widget.Spinner

val spinner = findViewById<Spinner>(R.id.categorySpinner)

val items = listOf("Все", "Работа", "Учёба", "Личное")

// Создаётся адаптер, который связывает список строк с отображением элементов Spinner
val adapter = ArrayAdapter(this, android.R.layout.simple_spinner_item, items).apply {
    // Задаётся шаблон отображения элементов в выпадающем списке

setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item)
}
spinner.adapter = adapter
// Назначается слушатель выбора элемента из списка
spinner.onItemSelectedListener = object : AdapterView.OnItemSelectedListener {
    // Обрабатывается выбор элемента пользователем
    override fun onItemSelected(…) {
        val selectedValue = items[position]
        // Здесь выполняется реакция на выбранное значение
    }
    // Обрабатывается состояние, когда выбор отсутствует
    override fun onNothingSelected(parent: AdapterView<*>?) {…}
}
```

Пример Spinner в XML:

```
<Spinner
    android:id="@+id/categorySpinner"
    android:layout_width="match_parent"
    android:layout_height="wrap_content" />
```

Для выбора до пяти изображений из галереи за один раз подход с Activity Result API сохраняется, но вместо контракта для одного изображения используется контракт множественного выбора. В этом сценарии системный пикер возвращает не один Uri, а

список Uri выбранных изображений, поэтому результат обрабатывается как набор элементов: при успешном выборе каждый Uri добавляется в коллекцию выбранных изображений (с ограничением не более пяти), после чего интерфейс обновляется и отображает миниатюры добавленных фотографий. Доступ к выбранным файлам предоставляется через возвращённые Uri, поэтому работа с изображениями далее выполняется через contentResolver, а отдельные разрешения на чтение галереи обычно не требуются, так как приложение получает доступ только к тем объектам, которые выбрал пользователь.

Для добавления до пяти фотографий, сделанных с камеры, отличие заключается в том, что системный контракт съёмки сохраняет результат только в заранее подготовленный Uri и за один запуск создаёт один снимок, возвращая признак успешного сохранения. Поэтому серия до пяти фотографий организуется как последовательность шагов: заранее подготавливаются несколько outputUri (например, до пяти) через FileProvider, затем камера запускается для первого Uri, после успешного сохранения Uri добавляется в коллекцию и при необходимости предлагается выполнить следующий снимок, который запускается уже со следующим подготовленным Uri, пока не будет достигнут лимит в пять изображений или пользователь не отменит действие. Неиспользованные подготовленные Uri (для которых съёмка была отменена) не должны попадать в список добавленных фотографий. Требование использования FileProvider для безопасной передачи outputUri остаётся тем же, меняется только то, что таких Uri подготавливается и обрабатывается несколько.

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Экран «Стать арендодателем»

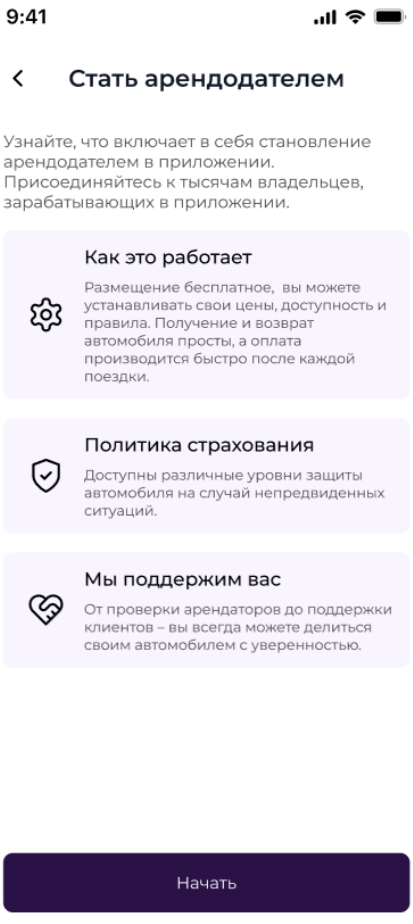


Рисунок 53 – Стать арендодателем

Общее описание

Реализуйте экран согласно макету. Точкой входа в данный экран является нажатие на поле «Подключить свой автомобиль» на экране настроек.

Функциональные требования

1. Отображение информационных блоков.
2. Кнопка «Начать»:
    - При нажатии на кнопку пользователь переходит на следующий экран, где начинается процесс добавления автомобиля для аренды.

Переходы

- Кнопка «Начать»: Переход на экран добавления автомобиля в приложение.
- Кнопка «Назад»: Возвращение на экран настроек и очистка стека навигации.

Экран «Добавление автомобиля (шаг 1)»



Рисунок 54 – Добавление автомобиля (шаг 1)

Общее описание

Реализуйте экран добавления автомобиля в приложении согласно макету. Точкой входа в данный экран является нажатие на кнопку «Начать» на экране «Стать арендодателем».

Функциональные требования

1. Поле ввода адреса:
    - Пользователь должен ввести адрес, где расположен автомобиль.
2. Кнопка «Далее»:
    - Неактивное состояние: Кнопка «Далее» должна быть неактивной, пока поле ввода адреса не заполнено.
    - Активное состояние: После ввода адреса кнопка становится активной, и по нажатию на неё осуществляется переход на следующий шаг добавления автомобиля.

Переходы

- Кнопка «Далее» (активная): Переход на следующий экран добавления автомобиля после ввода адреса.
- Кнопка «Назад»: Возвращение на предыдущий экран («Стать арендодателем»).

Экран «Добавление автомобиля (шаг 2)»

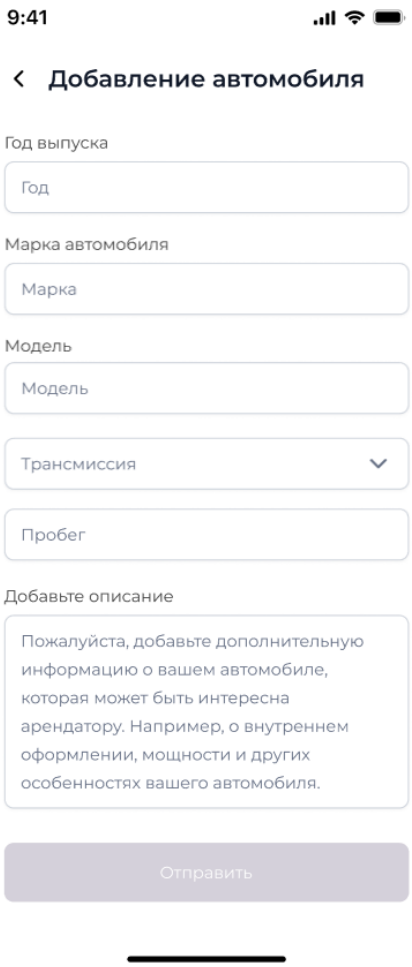


Рисунок 55 – Добавление автомобиля (шаг 2)

Общее описание

Реализуйте экран добавления информации об автомобиле в приложении согласно макету. Точкой входа в данный экран является нажатие на кнопку «Далее» на предыдущем экране добавления автомобиля.

Функциональные требования

1. Поля ввода:
    - Год выпуска автомобиля.
    - Марка автомобиля.
    - Модель автомобиля.
    - Тип трансмиссии (с выпадающим списком).
    - Пробег автомобиля.
    - Описание автомобиля (например, информация о внутреннем оформлении, мощности, особенностях).
2. Кнопка «Отправить»:

- Неактивное состояние: Кнопка «Отправить» должна быть неактивной, пока все поля не заполнены.
- Активное состояние: После заполнения всех полей кнопка становится активной, и по нажатию на неё происходит отправка данных и переход на следующий шаг.

Переходы

- Кнопка «Отправить» (активная): После заполнения всех полей и нажатия кнопки данные отправляются, и пользователь переходит на следующий экран процесса добавления автомобиля.
- Кнопка «Назад»: Возвращение на предыдущий экран добавления автомобиля (ввода адреса).

Экран «Добавление фотографий автомобиля»

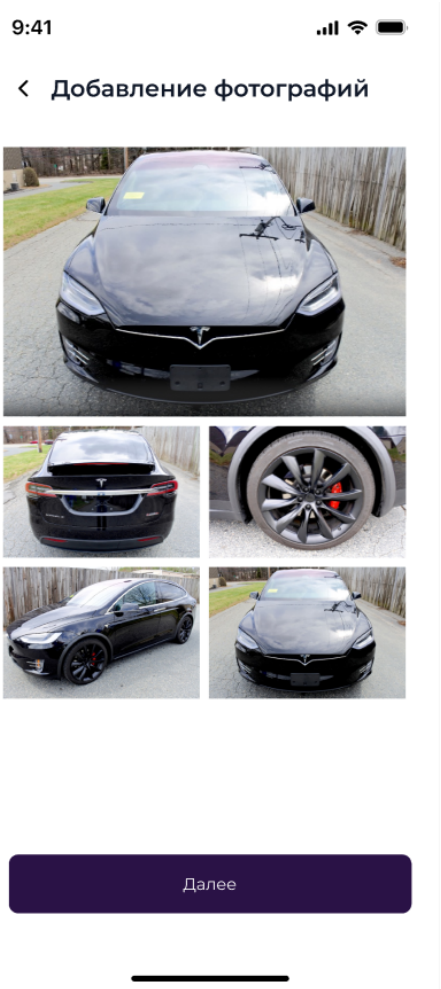


Рисунок 56 – Добавление фотографий автомобиля

Общее описание

Реализуйте экран добавления фотографий автомобиля в приложении согласно макету. Точкой входа в данный экран является нажатие на кнопку «Отправить» на предыдущем экране добавления информации об автомобиле.

Функциональные требования

1. Отображение загруженных фотографий:
    - Пользователь может добавить до 5 фотографий автомобиля за один раз при открытии галереи/файлов или камеры, которые будут отображаться на этом экране.
2. Кнопка «Далее»:
    - Кнопка становится активной только после того, как будут добавлены фотографии.

Переходы

- Кнопка «Далее»: Переход на финальный экран процесса добавления автомобиля после добавления фотографий и очистка стека навигации.

- Кнопка «Назад»: Возвращение на предыдущий экран добавления информации об автомобиле.

Экран «Успешное добавление автомобиля»

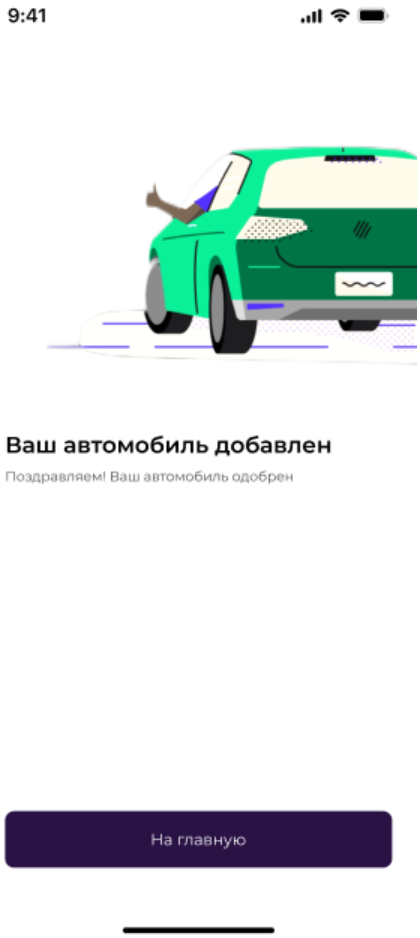


Рисунок 57 – Успешное добавление автомобиля

Общее описание

Реализуйте экран подтверждения добавления автомобиля в приложении согласно макету. Точкой входа в данный экран является успешное завершение процесса добавления автомобиля.

Функциональные требования

1. Сообщение об успешном добавлении:
    - Отображение сообщения, подтверждающего, что автомобиль был успешно добавлен и одобрен.
2. Кнопка «На главную»:
    - По нажатию на кнопку пользователь возвращается на главный экран приложения.

Переходы

- Кнопка «На главную»: Переход на главный экран приложения.

Контрольные вопросы

1. Что такое Spinner и для чего он используется?

2. В каких случаях целесообразно применять Spinner вместо набора кнопок или списка?

3. Каким XML-тегом добавляется Spinner в разметку экрана?

4. Что такое Adapter для Spinner и зачем он нужен?

5. Для чего используется OnItemSelectedListener у Spinner и какие методы он содержит?

6. Что такое множественный выбор фотографий из галереи пользователя и для чего он применяется?

7. Какой контракт Activity Result API используется для выбора нескольких изображений (например, PickMultipleVisualMedia)?

8. Что возвращает множественный выбор изображений (какой тип результата) и что такое Uri выбранной фотографии?

9. Как обработать список выбранных Uri и отобразить выбранные изображения в интерфейсе?

10. Зачем может понадобиться ограничение максимального числа выбираемых фотографий и как оно задаётся в контрактах выбора?